TRƯỜNG .....

KHOA .....

LỚP .....

HƯỚNG DẪN CÀI ĐẶT VÀ SỬ DỤNG WEBSITE GIẢI QUY HOẠCH TUYẾN TÍNH

Kèm toàn bộ mã nguồn sản phẩm

Tên nhóm: .....

Thành viên tham gia nhóm

- Thành viên 1: .....
Mã sinh viên: .....

- Thành viên 2: .....
Mã sinh viên: .....

- Thành viên 3: .....
Mã sinh viên: .....

- Thành viên 4: .....
Mã sinh viên: .....

- Thành viên 5: .....
Mã sinh viên: .....

Giảng viên: .....

Ngày nộp: ..... / ..... / .....

1 Giới thiệu sản phẩm

Website QHTT dùng để nhập và giải bài toán quy hoạch tuyến tính. Tài liệu hướng dẫn mở chương trình trên máy tính, nhập bài toán, xem lời giải, lưu dữ liệu và in kết quả. Phần cuối chứa toàn bộ mã nguồn của sản phẩm.

- Đơn hình: hỗ trợ bài toán Max hoặc Min, từ 1 đến 20 biến và tối đa 40 ràng buộc.

- Hình học: dùng cho bài toán có đúng 2 biến và miền nghiệm bị chặn.

- Cho phép nhập số nguyên, số thập phân, phân số và chọn miền của từng biến. Không hỗ trợ biến nguyên, biến nhị phân hoặc hàm phi tuyến.

2 Cài đặt và mở chương trình

- Bước 1: Sao chép hoặc giải nén toàn bộ thư mục DonHinhWeb vào máy tính.

+ Giữ nguyên tệp index.html và hai thư mục css, js cùng cấp. Không chỉ sao chép riêng tệp index.html.

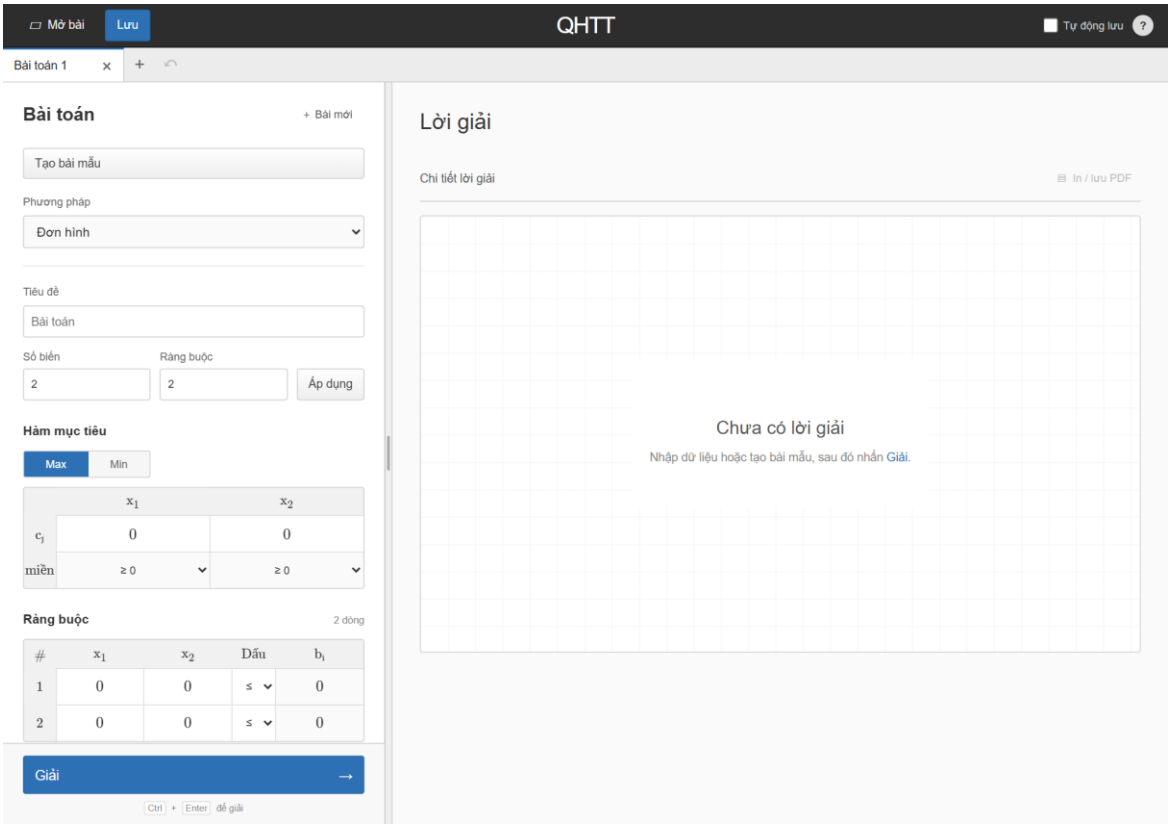
- Bước 2: Nhấp chuột phải vào index.html, chọn Open with và chọn Microsoft Edge hoặc Google Chrome.

- Bước 3: Kiểm tra trang hiển thị khung Bài toán và khung Lời giải. Trình duyệt phải cho phép JavaScript hoạt động.

- Chương trình chạy trực tiếp trong trình duyệt, không cần cài cơ sở dữ liệu, máy chủ hoặc thư viện bổ sung. Các tài nguyên dùng để chạy đã nằm trong thư mục sản phẩm.

- Sau khi đã có đủ tệp, có thể sử dụng ngoại tuyến. Khi chuyển sang máy khác, sao chép cả thư mục sản phẩm.

3 Làm quen với giao diện



Hình 1 Giao diện khi mở chương trình

- Bên trái là vùng nhập dữ liệu: phương pháp, tiêu đề, số biến, số ràng buộc, hàm mục tiêu và điều kiện ràng buộc.

- Bên phải là vùng lời giải và nút In / lưu PDF. Thanh trên cùng có Mở bài, Lưu và Tự động lưu.

- Nhấn dấu + hoặc Bài mới để mở một tab bài toán trống. Nhấn Tạo bài mẫu nếu muốn thử nhanh; sau đó nhấn Giải.

- Có thể kéo thanh ngăn giữa hai vùng để thay đổi độ rộng. Nhấp đúp thanh ngăn để đặt lại.

4 Nhập và giải một bài toán

Ví dụ: Tìm Max f = 3x1 + 2x2 với x1 + x2 ≤ 4, x1 ≤ 2, x2 ≤ 3 và x1, x2 ≥ 0.

- Bước 1: Chọn phương pháp Đơn hình, nhập tiêu đề Ví dụ hướng dẫn.

- Bước 2: Đặt Số biến là 2, Ràng buộc là 3 rồi nhấn Áp dụng.

- Bước 3: Chọn Max. Nhập hàng hệ số mục tiêu lần lượt là 3 và 2. Chọn miền của cả hai biến là ≥ 0.

- Bước 4: Nhập các dòng ràng buộc theo thứ tự hệ số x1, hệ số x2, dấu và vế phải b_i.

+ Dòng 1: 1, 1, ≤, 4.

+ Dòng 2: 1, 0, ≤, 2.

+ Dòng 3: 0, 1, ≤, 3.

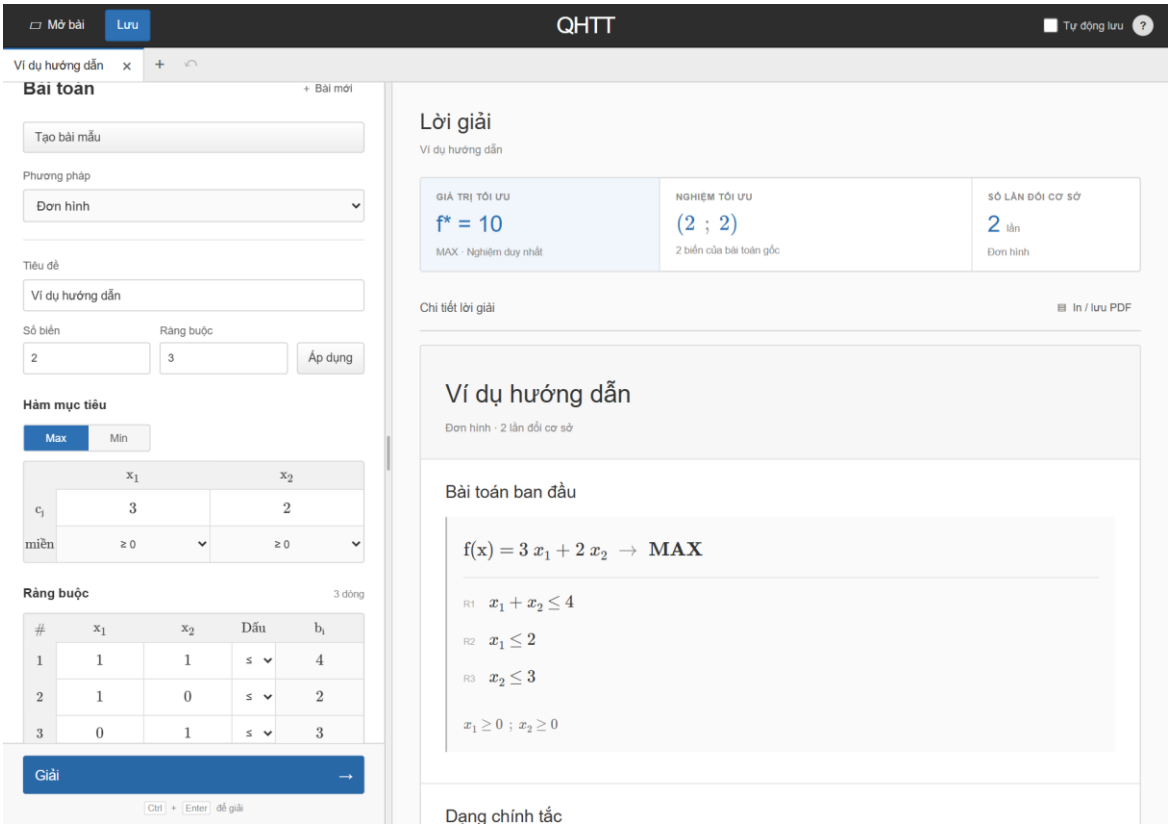
- Bước 5: Nhấn Giải hoặc Ctrl + Enter. Kết quả đúng của ví dụ là x1 = 2, x2 = 2 và f tối ưu = 10.

- Dùng 0 cho hệ số không có. Phân số nhập dạng 3/7; số thập phân nhập dạng 0.5. Không nhập tên biến vào ô hệ số.

- Nếu cần, chọn Min, thay dấu ràng buộc hoặc đổi miền biến thành ≤ 0 hay Tự do theo đề bài. Có thể thêm ràng buộc hoặc xóa dòng cuối bằng nút tương ứng.

- Khi sửa dữ liệu, nhấn Giải lần nữa để cập nhật lời giải. Nhấn Dừng nếu muốn ngừng phép tính đang chạy.

5 Đọc kết quả đơn hình



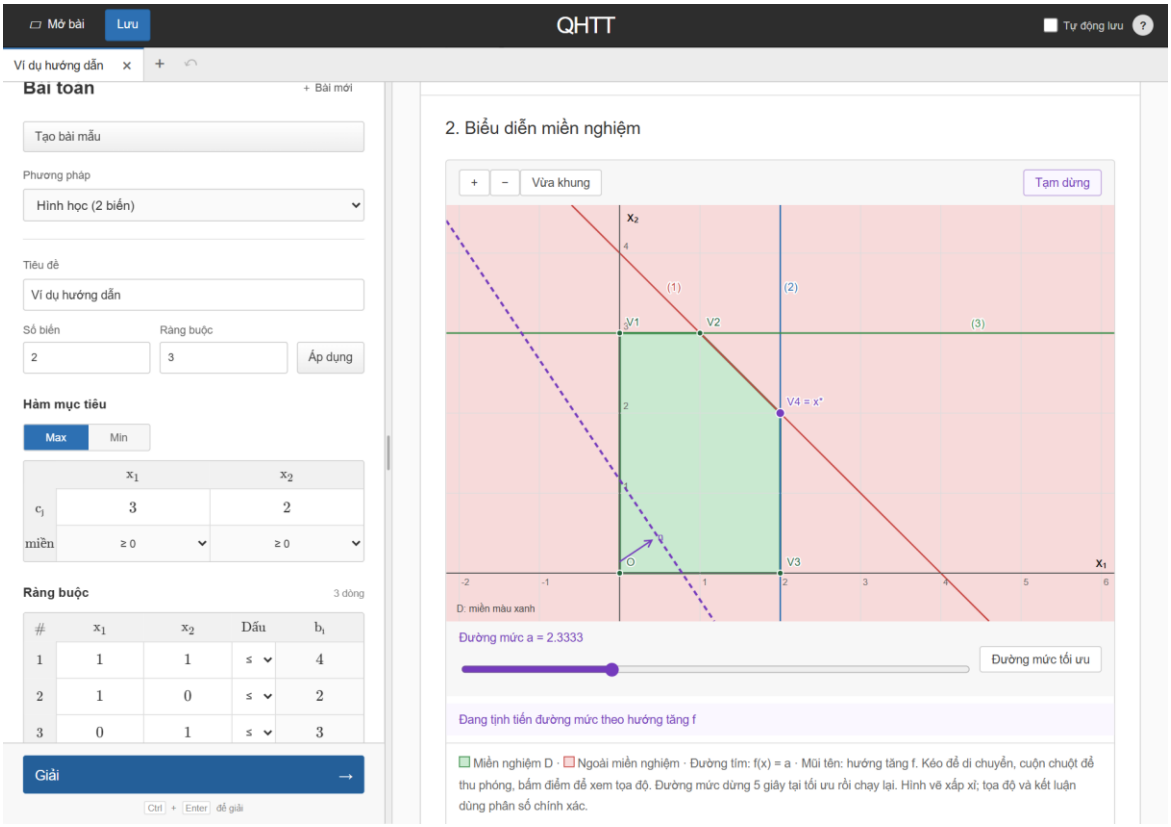
Hình 2 Nhập ví dụ và xem kết quả đơn hình

- Phần đầu hiển thị giá trị tối ưu, nghiệm tối ưu và số lần đổi cơ sở. Cuộn vùng lời giải để xem phần chuẩn hóa, các bảng tính và kết luận.

- Nếu kết quả là vô nghiệm, kiểm tra lại ràng buộc và điều kiện dấu. Nếu hàm mục tiêu không bị chặn, bài toán không có nghiệm tối ưu hữu hạn.

- Nếu chương trình thông báo chạm giới hạn tính toán, chưa thể kết luận bài toán tối ưu, vô nghiệm hay không bị chặn. Hãy kiểm tra hoặc đơn giản hóa dữ liệu.

6 Sử dụng phương pháp hình học



Hình 3 Đồ thị miền nghiệm của bài toán ví dụ

- Giữ dữ liệu ví dụ, chọn Hình học (2 biến), nhấn Giải rồi cuộn đến đồ thị. Kết quả vẫn là (2; 2), giá trị tối ưu 10.

- Cuộn chuột để thu phóng; kéo đồ thị để di chuyển; bấm điểm hoặc đường thẳng để xem thông tin. Nhấn Vừa khung để đặt lại vùng nhìn.

- Nhấn Tạm dừng hoặc kéo thanh a để điều khiển đường mức. Nhấn Đường mức tối ưu để xem vị trí tối ưu.

- Phương pháp này yêu cầu đúng 2 biến. Nếu miền nghiệm không bị chặn, chương trình thông báo và không hiển thị lời giải hình học; có thể chuyển sang Đơn hình để giải.

7 Lưu mở và in kết quả

- Lưu bài: nhấn Lưu để tải tệp JSON của tab hiện tại về máy. Tệp lưu đề bài và phương pháp, không phải bản in lời giải.

- Mở lại: nhấn Mở bài, chọn tệp JSON đã lưu. Bài toán được mở trong tab mới; nhấn Giải để tính kết quả.

- Tự động lưu: đánh dấu Tự động lưu để giữ các tab và dữ liệu nhập trên trình duyệt hiện tại.

+ Mở lại bằng cùng trình duyệt và cùng địa chỉ ứng dụng. Lời giải cần được tính lại bằng nút Giải.

+ Tắt tùy chọn sẽ xóa bản lưu tự động nhưng giữ các tab đang mở. Nên dùng Lưu để có bản JSON riêng khi cần chuyển máy hoặc sao lưu.

- In hoặc lưu PDF: giải bài toán trước, nhấn In / lưu PDF, chọn máy in hoặc Save as PDF trong hộp thoại in rồi xác nhận.

- Quản lý tab: bấm tên tab để chuyển bài; nhấn × để đóng tab; nhấn ↶ để mở lại tab vừa đóng. Chương trình giữ tối đa 10 tab đã đóng để mở lại.

8 Xử lý một số lỗi thường gặp

- Trang mất bố cục hoặc nút không chạy: kiểm tra thư mục css, js có đi kèm index.html hay không, rồi mở lại bằng trình duyệt.

- Báo dữ liệu không hợp lệ: kiểm tra ô trống, phân số có mẫu bằng 0, dấu quan hệ, số biến và số ràng buộc.

- Không tạo được bộ tính toán nền: mở bằng trình duyệt có hỗ trợ Web Worker và cho phép chạy JavaScript.

- Không khôi phục được bài: kiểm tra đã bật Tự động lưu và đang dùng đúng trình duyệt, đúng địa chỉ. Nếu có tệp JSON, dùng Mở bài để nạp lại.

- Nút In / lưu PDF chưa dùng được: nhấn Giải và đợi lời giải hoàn tất trước khi in.

9 Toàn bộ mã nguồn sản phẩm

Mã nguồn được chép theo từng tệp bên dưới, gồm HTML, CSS và JavaScript. Tệp math-fonts.js chứa cả dữ liệu phông chữ nhúng. Các ký tự, nội dung và thứ tự dòng của từng tệp được giữ nguyên; dòng dài được Word tự xuống dòng khi hiển thị.

- index.html
- css/style.css
- js/app.js
- js/examples.js
- js/geometry-view.js
- js/geometry.js
- js/layout.js
- js/math-fonts.js
- js/report.js
- js/session.js
- js/solver.js

index.html

```
<!doctype html>
<html lang="vi">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-
width,initial-scale=1" />
    <meta
      name="description"
      content="Giải quy hoạch tuyến tính"
    />
    <title>Quy hoạch tuyến tính</title>
    <link
      rel="icon"
      href="data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' viewBox='0 0 64
64'%3E%3Crect width='64' height='64' rx='16'
fill='%232d70b3'/%3E%3Ctext x='32' y='46' font-size='44' text-
anchor='middle' fill='white'%3EΣ%3C/text%3E%3C/svg%3E"
    />
    <link rel="stylesheet" href="css/style.css" />
    <script defer src="js/math-fonts.js"></script>
    <script defer src="js/solver.js"></script>
    <script defer src="js/geometry.js"></script>
    <script defer src="js/examples.js"></script>
    <script defer src="js/report.js"></script>
    <script defer src="js/geometry-view.js"></script>
    <script defer src="js/session.js"></script>
    <script defer src="js/app.js"></script>
    <script defer src="js/layout.js"></script>
  </head>
  <body>
    <div class="app-shell">
      <header class="app-header">
        <div class="header-actions">
          <button id="open-btn" class="btn subtle"
title="Mở tệp bài toán JSON">
            <span aria-hidden="true">▱</span> Mở
bài</button
          ><button id="save-btn" class="btn
primary">Lưu</button>
        </div>
        <a class="brand" href="#" aria-label="QHTT"
```

```
              ><strong>QHTT</strong></a
            >
            <div class="header-right">
              <label
                class="autosave-option"
                id="autosave-label"
                title="Tự động lưu các tab và dữ liệu nhập
trên trình duyệt này"
              >
                <input id="autosave-toggle" type="checkbox"
aria-describedby="autosave-help" />
                <span>Tự động lưu</span>
              </label>
              <button id="help-btn" class="help-button" aria-
label="Hướng dẫn sử dụng">?</button>
            </div>
          </header>
          <nav class="tab-bar" aria-label="Các bài toán đang
mở">
            <div id="problem-tabs" class="problem-tabs"
role="tablist" aria-label="Bài toán"></div>
            <button
              id="add-tab"
              class="tab-action"
              type="button"
              title="Tạo tab mới"
              aria-label="Tạo tab mới"
            >
              +
            </button>
            <button
              id="reopen-tab"
              class="tab-action"
              type="button"
              title="Mở lại tab vừa đóng"
              aria-label="Mở lại tab vừa đóng"
              disabled
            >
              ↶
            </button>
          </nav>
          <div id="storage-notice" class="storage-notice"
role="status" hidden></div>
          <main id="workspace-panel" class="workspace"
```

```
role="tabpanel">
            <aside id="input-panel" class="input-panel" aria-
label="Nhập bài toán">
              <form id="problem-form" novalidate>
                <div class="input-scroll">
                  <div class="panel-heading">
                    <h1>Bài toán</h1>
                    <button id="new-btn" type="button"
class="btn subtle" title="Tạo bài toán trống">
                      ＋ Bài mới
                    </button>
                  </div>
                  <button
                    id="example"
                    type="button"
                    class="btn secondary sample-button"
                    title="Nạp ngẫu nhiên một bài toán"
                  >
                    Tạo bài mẫu
                  </button>
                  <label class="field-label method-label"
for="solve-method">Phương pháp</label>
                  <select id="solve-method" class="method-
select">
                    <option value="bigm">Đơn hình</option>
                    <option value="geometry">Hình học (2
biến)</option>
                    <option value="two" hidden>Đơn hình hai
pha</option>
                  </select>
                  <div class="form-divider"></div>
                  <label class="field-label" for="problem-
title">Tiêu đề</label
                  ><input
                    id="problem-title"
                    type="text"
                    maxlength="150"
                    autocomplete="off"
                    placeholder="Bài toán"
                  />
                  <div class="dimensions">
                    <label>Số biến <input id="n"
type="number" min="1" max="20" value="2" /></label
                    ><label>Ràng buộc <input id="m"
```

```
type="number" min="0" max="40" value="2" /></label
                      ><button id="resize-btn" type="button"
class="btn secondary">Áp dụng</button>
                    </div>
                    <div class="section-heading">
                      <h2>Hàm mục tiêu</h2>
                    </div>
                    <div class="objective-options">
                      <div class="segmented" aria-label="Chiều
tối ưu">
                        <label
                          ><input type="radio" name="sense"
value="max" checked /><span>Max</span></label
                        ><label><input type="radio"
name="sense" value="min" /><span>Min</span></label>
                      </div>
                    </div>
                    <div id="objective-input" class="input-
table-wrap"></div>
                    <div class="section-heading">
                      <h2>Ràng buộc</h2>
                      <span id="constraint-count" class="count-
chip">2 dòng</span>
                    </div>
                    <div id="constraint-input" class="input-
table-wrap constraints-input"></div>
                    <div class="table-actions">
                      <button id="add-row" class="text-button"
type="button">＋ Thêm ràng buộc</button
                      ><button id="remove-row" class="text-
button muted" type="button">
                        − Xóa dòng cuối
                      </button>
                    </div>
                    <p class="field-hint">Nhập số hoặc phân số,
ví dụ 0.5, 3/7. Hỗ trợ dán từ Excel.</p>
                    <div id="input-error" class="input-error"
role="alert" hidden></div>
                  </div>
                  <div class="solve-bar">
                    <button id="solve-btn" class="btn primary"
type="submit">
                      <span>Giải</span><span aria-
hidden="true">→</span></button
```

```
                  ><button id="cancel-btn" type="button"
class="btn secondary" hidden>Dừng</button>
                  <p><kbd>Ctrl</kbd> + <kbd>Enter</kbd> để
giải</p>
                </div>
              </form>
            </aside>
            <div
              id="panel-resizer"
              class="panel-resizer"
              role="separator"
              tabindex="0"
              aria-label="Điều chỉnh độ rộng panel nhập bài
toán"
              aria-orientation="vertical"
              aria-controls="input-panel"
              aria-valuemin="300"
              aria-valuemax="800"
              aria-valuenow="420"
              title="Kéo để chỉnh độ rộng · Nhấp đúp để đặt
lại · Phím ← → để điều chỉnh"
            >
              <span aria-hidden="true"></span>
            </div>
            <section class="solution-panel" aria-label="Lời
giải">
              <div class="solution-intro">
                <div>
                  <h2>Lời giải</h2>
                  <p id="solution-subtitle" hidden></p>
                </div>
              </div>
              <div id="stale-banner" class="stale-banner"
hidden>
                Dữ liệu đã thay đổi. Nhấn <b>Giải</b> để cập
nhật kết quả.
              </div>
              <div id="busy-banner" class="busy-banner"
role="status" hidden>
                <span class="spinner"></span> Đang giải…
              </div>
              <div id="result-summary" class="result-summary"
hidden aria-live="polite"></div>
              <div class="result-toolbar">
```

```
                <span>Chi tiết lời giải</span>
                <div class="export-actions">
                  <button id="print-btn" class="btn subtle"
disabled>
                    🖶 <span>In / lưu PDF</span>
                  </button>
                </div>
              </div>
              <article id="report" class="report is-empty"
aria-live="off">
                <div class="empty-state">
                  <h2>Chưa có lời giải</h2>
                  <p>Nhập dữ liệu hoặc tạo bài mẫu, sau đó
nhấn <b>Giải</b>.</p>
                </div>
              </article>
            </section>
          </main>
          <div id="toast" class="toast" role="status"
hidden></div>
        </div>
        <div id="print-output" class="report"></div>
        <input id="file-input" type="file"
accept=".json,application/json" hidden />
        <dialog id="help-dialog">
          <div class="dialog-header">
            <h2>Hướng dẫn</h2>
            <button id="close-help" class="icon-button" aria-
label="Đóng hướng dẫn">×</button>
          </div>
          <div class="help-content">
            <h3>Nhập bài toán</h3>
            <p>
              Chọn số biến và số ràng buộc, nhấn Áp dụng.
Nhập hệ số mục tiêu, điều kiện dấu và các ràng
              buộc. Dùng 0 cho hệ số không có.
            </p>
            <p>Tạo bài mẫu nạp ngẫu nhiên một đề. Nhấn Giải
hoặc Ctrl+Enter để tính.</p>
            <h3>Định dạng dữ liệu</h3>
            <p>
              Chấp nhận số nguyên, số thập phân và phân số:
−2, 0.5, 3/7. Có thể dán vùng dữ liệu từ
              Excel; dấu quan hệ dùng &amp;lt;=, =, &amp;gt;=; miền
```

```
biến dùng pos, neg, free.
            </p>
            <h3>Lưu và xuất</h3>
            <p>
              Lưu tải đề của tab hiện tại dưới dạng JSON; Mở
bài nạp lại đề trong tab mới. JSON giữ
              nguyên hệ số, điều kiện dấu và phương pháp
giải. In / lưu PDF mở hộp thoại in.
            </p>
            <h3>Các tab bài toán</h3>
            <p>
              Nhấn + hoặc Bài mới để mở tab trống. Mỗi tab
giữ riêng đề và lời giải. Nhấn × để đóng; ↶
              mở lại tab vừa đóng (giữ tối đa 10 tab). Khi
thanh tab được chọn, dùng ← / →, Home hoặc
              End để chuyển tab. Các tab tạm khóa khi đang
tính; nhấn Dừng để thao tác tiếp.
            </p>
            <h3>Tự động lưu phiên làm việc</h3>
            <p id="autosave-help">
              Tùy chọn Tự động lưu mặc định tắt. Bật để lưu
các tab, dữ liệu nhập và tab đang chọn vào
              localStorage trên trình duyệt này. Khi mở lại,
nhấn Giải để tính lại lời giải. Tắt tùy
              chọn sẽ xóa bản lưu tự động, giữ nguyên các tab
đang mở.
            </p>
            <p>
              Phiên được khôi phục khi dùng cùng trình duyệt
và địa chỉ mở ứng dụng. Dùng Lưu để tải
              JSON và chuyển đề sang máy khác hoặc giữ bản
sao riêng.
            </p>
            <h3>Bố cục</h3>
            <p>
              Kéo thanh ngăn để đổi độ rộng. Nhấp đúp để đặt
lại. Có thể chọn thanh bằng Tab và điều
              chỉnh bằng phím ← / →.
            </p>
            <h3>Phương pháp hình học</h3>
            <p>
              Chọn Hình học (2 biến), nhập đúng hai biến và
các ràng buộc tạo miền nghiệm bị chặn rồi nhấn Giải. Miền
không bị chặn sẽ được thông báo và không hiển thị lời giải hay
```

```
animation. Đồ thị hiển thị các đường biên,
              miền nghiệm và đường mức. Đường mức tự chạy,
dừng 5 giây tại tối ưu rồi lặp lại. Nhấn Tạm
              dừng hoặc kéo thanh a để điều khiển. Vùng đỏ ở
ngoài miền nghiệm; vùng xanh lục thuộc miền
              nghiệm. Cuộn chuột để thu phóng tại con trỏ,
kéo hình để di chuyển và bấm điểm để xem tọa
              độ. Bấm đường thẳng để xem phương trình. Vừa
khung đặt lại vùng nhìn. Lời giải gồm dựng
              miền D, các đỉnh, đường mức và kết luận. In /
lưu PDF dùng hình tổng quan cùng lời giải.
            </p>
            <h3>Quy ước tính toán</h3>
            <div class="math-block">
              Δ_j = Σ c^B_i a_ij − c_j<br />Max: Δ_j ≥ 0 tại tối
ưu.<br />Min: Δ_j ≤ 0 tại tối ưu.
            </div>
            <p>
              Biến giả được xử lý bằng M tượng trưng. Dấu ×
chỉ cột đã loại. Các tệp JSON hai pha vẫn
              được hỗ trợ.
            </p>
            <h3>Phạm vi</h3>
            <p>
              Quy hoạch tuyến tính với biến liên tục: 1–20
biến, tối đa 40 ràng buộc. Hỗ trợ Max/Min,
              các dấu ≤, =, ≥ và biến tự do. Không hỗ trợ
biến nguyên, nhị phân hoặc hàm phi tuyến.
            </p>
          </div>
        </dialog>
        <noscript>Ứng dụng cần bật JavaScript trong trình
duyệt.</noscript>
      </body>
    </html>
```

css/style.css

```
@charset "UTF-8";
:root {
  --workspace-top: 86px;
  --ink: #333;
  --muted: #777;
  --blue: #2d70b3;
  --line: #d8d8d8;
  --math: 'Computer Modern', 'Cambria Math', serif;
  --sans: Arial, 'Segoe UI', sans-serif;
}
* {
  box-sizing: border-box;
}
body {
  margin: 0;
  background: #fafafa;
  color: var(--ink);
  font: 14px/1.5 var(--sans);
}
button,
input,
select {
  font: inherit;
}
button,
select,
summary {
  cursor: pointer;
}
button:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}
[hidden] {
  display: none !important;
}
h1,
h2,
h3,
p {
  margin: 0;
}
```

```
h1,
h2,
h3 {
  font-weight: 600;
}
button:focus-visible,
a:focus-visible,
summary:focus-visible {
  outline: 3px solid #8ab8e6;
  outline-offset: 2px;
}
input:focus,
select:focus {
  outline: 0;
  border-color: var(--blue);
  box-shadow: 0 0 0 2px #2d70b322;
}
input[aria-invalid='true'] {
  border-color: #c74440 !important;
  background: #fff0ef !important;
}
.muted {
  color: var(--muted);
}

/* Thanh đầu trang và panel nhập liệu */
.app-header {
  height: 48px;
  background: #2d2d2d;
  color: #eee;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 16px;
  position: sticky;
  top: 0;
  z-index: 20;
  border-bottom: 1px solid #111;
}
.header-actions,
.header-right {
  display: flex;
  align-items: center;
  gap: 10px;
```

```
  flex: 1;
}
.header-right {
  justify-content: flex-end;
  font-size: 12px;
  color: #ccc;
}
.brand {
  text-decoration: none;
  color: #fff;
  line-height: 1;
}
.brand strong {
  font-size: 25px;
  font-weight: 400;
  letter-spacing: -1px;
}
.brand span {
  font-size: 13px;
  letter-spacing: 0.7px;
  margin-left: 7px;
  color: #bbb;
}
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  padding: 7px 12px;
  border: 1px solid transparent;
  border-radius: 3px;
  background: transparent;
  font-size: 13px;
  line-height: 1.4;
  transition: background 0.12s;
}
.btn.subtle {
  color: #555;
}
.btn.subtle:hover:not(:disabled) {
  background: #e8e8e8;
}
.btn.primary {
  background: var(--blue);
```

```
      border-color: #24639f;
      color: #fff;
    }
    .btn.primary:hover:not(:disabled) {
      background: #245f99;
    }
    .btn.secondary {
      background: linear-gradient(#fff, #eee);
      border-color: #ccc;
      color: #444;
    }
    .btn.secondary:hover:not(:disabled) {
      background: #e5e5e5;
    }
    .app-header .subtle {
      color: #eee;
    }
    .app-header .subtle:hover {
      background: #444;
    }
    .help-button {
      border: 0;
      border-radius: 50%;
      width: 23px;
      height: 23px;
      background: #ddd;
      color: #333;
      font-weight: bold;
    }
    .workspace {
      display: grid;
      grid-template-columns: var(--sidebar-width, 420px) 8px
minmax(0, 1fr);
      min-height: calc(100vh - var(--workspace-top));
    }
    .input-panel {
      background: #fff;
      border-right: 1px solid #bbb;
      height: calc(100vh - var(--workspace-top));
      position: sticky;
      top: var(--workspace-top);
      min-height: 440px;
      box-shadow: 2px 0 5px #00000006;
    }
```

```
#problem-form {
  height: 100%;
  display: flex;
  flex-direction: column;
}
.input-scroll {
  overflow: auto;
  padding: 19px 22px 20px;
  scrollbar-width: thin;
  scrollbar-color: #ccc transparent;
}
.panel-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 20px;
}
.panel-heading h1 {
  font-size: 20px;
}
.panel-heading .btn {
  font-size: 12px;
}
.field-label {
  display: block;
  font-size: 12px;
  color: #666;
  margin-bottom: 6px;
}
input,
select {
  background: #fff;
  border: 1px solid #ccc;
  border-radius: 3px;
  color: #333;
  min-height: 35px;
  padding: 6px 8px;
}
select:not(.input-table select),
#problem-title {
  width: 100%;
}
select {
  padding-right: 23px;
```

```
}
.sample-button {
  width: 100%;
  justify-content: flex-start;
  background: #f7f7f7;
  font-size: 13px;
}
#problem-title {
  font-size: 13px;
}
.form-divider {
  height: 1px;
  background: #e2e2e2;
  margin: 19px 0;
}
.dimensions {
  display: grid;
  grid-template-columns: 1fr 1fr auto;
  gap: 10px;
  align-items: end;
  margin-top: 12px;
}
.dimensions label {
  font-size: 12px;
  color: #666;
}
.dimensions input {
  display: block;
  width: 100%;
  margin-top: 4px;
  height: 34px;
}
.dimensions .btn {
  height: 35px;
}
.section-heading {
  display: flex;
  align-items: center;
  gap: 9px;
  margin: 23px 0 11px;
}
.section-heading > span:not(.count-chip) {
  font-size: 11px;
  color: #888;
```

```
  border-right: 1px solid #ddd;
  padding-right: 8px;
}
.section-heading h2 {
  font-size: 14px;
}
.count-chip {
  margin-left: auto;
  color: #888;
  font-size: 11px;
}
.objective-options {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 10px;
  gap: 8px;
}
.segmented {
  display: flex;
  border: 1px solid #ccc;
  border-radius: 3px;
  overflow: hidden;
}
.segmented input {
  position: absolute;
  opacity: 0;
  width: 1px;
  height: 1px;
  min-height: 0;
}
.segmented span {
  display: block;
  padding: 5px 24px;
  background: #f6f6f6;
  color: #666;
  font-size: 12px;
  cursor: pointer;
}
.segmented input:checked + span {
  background: var(--blue);
  color: #fff;
}
.segmented input:focus-visible + span {
```

```
  outline: 2px solid #222;
  outline-offset: -3px;
}
.input-table-wrap {
  overflow: auto;
  border: 1px solid #ccc;
  border-radius: 3px;
  scrollbar-width: thin;
}
.input-table {
  border-collapse: collapse;
  width: 100%;
  table-layout: fixed;
  text-align: center;
}
.input-table th {
  background: #f1f1f1;
  height: 31px;
  font-family: var(--math);
  font-size: 15px;
  font-weight: 400;
}
.input-table th:first-child {
  width: 35px;
}
.input-table td {
  border-top: 1px solid #ddd;
  border-left: 1px solid #ddd;
  padding: 0;
  height: 39px;
}
.input-table input,
.input-table select {
  border: 0;
  border-radius: 0;
  width: 100%;
  height: 39px;
  min-height: 39px;
  padding: 4px 2px;
  text-align: center;
  font-family: var(--math);
  font-size: 17px;
  background: transparent;
}
```

```
.input-table select {
  font-family: var(--sans);
  font-size: 12px;
  padding-right: 0;
}
.input-table input:focus,
.input-table select:focus {
  box-shadow: inset 0 0 0 2px var(--blue);
  background: #f3f8ff;
}
.input-table .relation {
  width: 48px;
}
.domain-row,
.rhs-input {
  background: #fafafa;
}
.constraints-input {
  max-height: 260px;
}
.field-hint {
  font-size: 11px;
  color: #888;
  line-height: 1.6;
  margin-top: 8px;
}
.field-hint b {
  font-weight: 400;
  color: #555;
}
.table-actions {
  display: flex;
  justify-content: space-between;
  gap: 8px;
  margin-top: 7px;
}
.text-button {
  background: none;
  border: 0;
  padding: 4px 0;
  color: var(--blue);
  font-size: 12px;
}
.text-button.muted {
```

```
  color: #888;
}
#method {
  font-size: 13px;
}
.solve-bar {
  margin-top: auto;
  border-top: 1px solid #ccc;
  padding: 13px 22px 10px;
  background: #f8f8f8;
  flex-shrink: 0;
}
.solve-bar .primary {
  width: 100%;
  height: 42px;
  justify-content: space-between;
  font-size: 15px;
}
.solve-bar .secondary {
  width: 100%;
  margin-top: 8px;
}
.solve-bar p {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  margin-top: 8px;
  color: #888;
  font-size: 10px;
}
kbd {
  font: 10px var(--sans);
  background: #fff;
  border: 1px solid #ddd;
  border-radius: 2px;
  padding: 1px 3px;
}
.input-error {
  background: #fff1f0;
  color: #ad3430;
  border: 1px solid #ecc3c1;
  border-radius: 3px;
  padding: 10px;
```

```
  margin-top: 12px;
  font-size: 12px;
}

/* Khu vực lời giải và trạng thái */
.solution-panel {
  min-width: 0;
  padding: 25px 30px 0;
}
.solution-intro {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 15px;
  margin-bottom: 21px;
}
.solution-intro h2 {
  font-size: 24px;
  font-weight: 400;
  margin-bottom: 4px;
}
.solution-intro p {
  font-size: 12px;
  color: #888;
}
.exact-badge {
  font-size: 11px;
  color: #666;
  padding: 5px 9px;
  border: 1px solid #ddd;
  background: #fff;
  border-radius: 3px;
  white-space: nowrap;
}
.result-summary {
  display: grid;
  grid-template-columns: 1fr 1.3fr 0.7fr;
  border: 1px solid #ccc;
  background: #fff;
  border-radius: 3px;
  margin-bottom: 20px;
}
.summary-card {
  padding: 13px 17px;
```

```
  min-width: 0;
  overflow: auto;
  border-right: 1px solid #ddd;
}
.summary-card:last-child {
  border-right: 0;
}
.summary-card.primary-card {
  background: #f0f6fc;
}
.summary-label {
  color: #888;
  font-size: 10px;
  font-weight: 600;
  letter-spacing: 0.5px;
}
.summary-value {
  font-size: 25px;
  color: var(--blue);
  margin-top: 4px;
  white-space: nowrap;
}
.summary-value.math {
  font-size: 24px;
}
.summary-foot {
  font-size: 11px;
  color: #888;
  margin-top: 3px;
  white-space: nowrap;
}
.warning-card .summary-value {
  color: #c74440;
  font-size: 18px;
}
.result-toolbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  padding: 5px 0 10px;
  border-bottom: 1px solid #ccc;
  margin-bottom: 15px;
  font-size: 13px;
```

```
  color: #555;
}
.export-actions {
  display: flex;
  gap: 4px;
}
.export-actions .btn {
  font-size: 12px;
  padding: 5px 9px;
}
.workspace-footer {
  display: flex;
  justify-content: space-between;
  gap: 15px;
  padding: 17px 0 22px;
  font-size: 10px;
  color: #999;
}
.footer-dot {
  margin: 0 7px;
}
.stale-banner,
.busy-banner {
  padding: 11px 14px;
  margin-bottom: 14px;
  font-size: 12px;
  border: 1px solid #e9d7ad;
  border-radius: 3px;
  background: #fffaed;
  color: #8b6b2e;
}
.busy-banner {
  display: flex;
  align-items: center;
  gap: 10px;
  background: #eff6fd;
  border-color: #c9ddef;
  color: var(--blue);
}
.spinner {
  width: 16px;
  height: 16px;
  border: 2px solid #bfd6ed;
  border-top-color: var(--blue);
```

```
      border-radius: 50%;
      animation: spin 1s linear infinite;
    }
    @keyframes spin {
      to {
        transform: rotate(360deg);
      }
    }
    .toast {
      position: fixed;
      bottom: 25px;
      left: 50%;
      transform: translateX(-50%);
      padding: 12px 20px;
      background: #333;
      color: #fff;
      border-radius: 3px;
      box-shadow: 0 4px 18px #0002;
      z-index: 100;
      font-size: 13px;
      max-width: 90vw;
    }
    .report {
      background: #fff;
      border: 1px solid #d0d0d0;
      border-radius: 3px;
      overflow: hidden;
    }
    .report.is-empty {
      min-height: 480px;
      display: grid;
      place-items: center;
      background-color: #fff;
      background-image: linear-gradient(#eee8 1px,
transparent 1px),
        linear-gradient(90deg, #eee8 1px, transparent 1px);
      background-size: 30px 30px;
    }
    .empty-state {
      text-align: center;
      padding: 45px 20px;
      color: #888;
      background: #ffffffed;
    }
```

```
.empty-state h2 {
  font-size: 20px;
  font-weight: 400;
  color: #555;
  margin: 13px 0 8px;
}
.empty-state p {
  font-size: 13px;
}
.empty-state b {
  font-weight: 400;
  color: var(--blue);
}

/* Công thức, bảng đơn hình và kết luận */
.math,
.tableau,
.simple-table {
  font-family: var(--math);
}
.math {
  font-size: 19px;
}
.math-var {
  font-family: 'Computer Modern Math', var(--math);
  font-weight: 400;
  font-style: italic;
}
.solution-section {
  padding: 22px 27px;
}
.solution-section + .solution-section {
  border-top: 1px solid #ddd;
}
.solution-section h2 {
  font-size: 19px;
  font-weight: 500;
  margin: 0 0 12px;
}
.solution-section h3 {
  font-size: 14px;
  margin: 16px 0 8px;
}
.solution-section p {
```

```
  font-size: 13px;
  line-height: 1.8;
  margin: 8px 0;
}
.solution-section p.math {
  font-size: 18px;
}
.solution-section p.muted {
  font-size: 12px;
}
.step-heading {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
}
.badge {
  font: 11px var(--sans);
  padding: 4px 8px;
  background: #f2f2f2;
  color: #666;
  border: 1px solid #ddd;
  border-radius: 3px;
  white-space: nowrap;
}
.math-block {
  font: 19px/2.1 var(--math);
  overflow: auto;
  margin: 12px 0;
  padding: 13px 18px;
  background: #fafafa;
  border-left: 2px solid #bbb;
}
.objective-line {
  font-size: 21px;
  padding-bottom: 7px;
}
.arrow {
  margin: 0 7px;
  color: #777;
}
.constraints-list {
  border-top: 1px solid #e3e3e3;
  padding: 7px 0;
```

```
}
.row-tag {
  font: 10px var(--sans);
  color: #aaa;
  margin-right: 15px;
}
.domain-line {
  font-size: 16px;
  color: #666;
  margin-top: 7px;
}
.initial-basis {
  margin-top: 13px;
  padding-top: 10px;
  border-top: 1px solid #eee;
}
.table-wrap {
  overflow: auto;
  margin: 14px 0 10px;
  border: 1px solid #ccc;
  border-radius: 2px;
  scrollbar-width: thin;
}
.tableau,
.simple-table {
  border-collapse: collapse;
  width: 100%;
  text-align: center;
  font-size: 18px;
  white-space: nowrap;
}
.tableau td,
.tableau th {
  padding: 11px 10px;
  border-right: 1px solid #ddd;
  border-bottom: 1px solid #ddd;
  min-width: 45px;
  font-weight: 400;
}
.tableau th:last-child,
.tableau td:last-child {
  border-right: 0;
}
.tableau thead {
```

```
  background: #f3f3f3;
}
.tableau thead tr:first-child th:not([rowspan]) {
  border-bottom: 0;
  padding-bottom: 5px;
}
.tableau thead tr:nth-child(2) th {
  padding-top: 4px;
}
.tableau small {
  display: block;
  font: 10px var(--sans);
  color: #888;
  margin-top: 3px;
}
.tableau .rhs {
  background: #fafafa;
  border-right: 2px solid #ccc;
}
.tableau .enter-col {
  background: #eaf3fc;
}
.tableau tbody .leaving-row td {
  background: #fff0ef;
}
.tableau tbody .leaving-row .enter-col {
  background: #e3edf9;
}
.tableau tbody .pivot {
  background: var(--blue) !important;
  color: #fff;
}
.tableau .inactive {
  color: #bbb;
  background: #f8f8f8 !important;
}
.tableau tfoot td,
.tableau tfoot th {
  background: #f3f6fa;
  border-bottom: 0;
  color: #244e78;
}
.table-legend {
  display: flex;
```

```
  gap: 17px;
  flex-wrap: wrap;
  color: #888;
  font-size: 10px;
  margin: 9px 0 12px;
}
.table-legend > span {
  display: flex;
  gap: 5px;
  align-items: center;
}
.dot {
  display: inline-block;
  width: 8px;
  height: 8px;
  border-radius: 1px;
}
.dot.teal {
  background: var(--blue);
}
.dot.pale {
  background: #c6ddf5;
}
.dot.amber {
  background: #f6ceca;
}
.fraction {
  display: inline-flex;
  flex-direction: column;
  vertical-align: middle;
  text-align: center;
  line-height: 1.2;
  font-family: var(--math);
  margin: 0 0.12em;
  min-width: 1em;
}
.fraction > span:first-child {
  border-bottom: 1px solid currentColor;
  padding: 0 0.12em 0.12em;
}
.fraction > span:last-child {
  padding: 0.12em 0.12em 0;
}
.vector {
```

```
  white-space: nowrap;
}
.separator {
  margin: 0 0.4em;
  color: #999;
}
.notice {
  padding: 12px 16px;
  background: #f7f7f7;
  border: 1px solid #ddd;
  border-radius: 3px;
  margin: 15px 0;
  font-size: 13px;
}
.notice.success {
  background: #f1f7fd;
  border-color: #d3e3f3;
}
.notice.warning {
  background: #fff6ee;
  border-color: #ebdac8;
}
.notice .answer {
  font-size: 24px;
  line-height: 1.9;
  color: var(--blue);
  margin-top: 10px;
}
.simple-table td,
.simple-table th {
  padding: 10px 13px;
  border-bottom: 1px solid #ddd;
  font-weight: 400;
}
.simple-table thead th {
  background: #f4f4f4;
  font: 12px var(--sans);
}
.check {
  font: 12px var(--sans);
  color: var(--blue);
}
.report details {
  border-top: 1px solid #ddd;
```

```
  padding-top: 12px;
  margin-top: 16px;
}
.report summary {
  font-size: 12px;
  color: var(--blue);
}
.report details[open] summary {
  margin-bottom: 10px;
}
.report-cover {
  background: #f7f7f7;
  padding: 25px 27px;
  border-bottom: 1px solid #ddd;
}
.section-label,
.eyebrow {
  font-size: 10px;
  letter-spacing: 1px;
  color: #888;
}
.report-cover h1 {
  font-size: 27px;
  font-weight: 400;
  margin: 8px 0;
}
.report-cover p {
  font-size: 12px;
  color: #888;
}
.report-footer {
  background: #fafafa;
  border-top: 1px solid #ddd;
  padding: 17px 27px;
  font-size: 11px;
  color: #888;
}
.report sub {
  font-size: 0.7em;
  vertical-align: sub;
  line-height: 0;
}
.report sup {
  font-size: 0.7em;
```

```
}
.exported {
  padding: 30px;
}
.exported .report {
  max-width: 1200px;
  margin: auto;
}

/* Hộp hướng dẫn */
dialog {
  padding: 0;
  border: 1px solid #bbb;
  border-radius: 4px;
  width: calc(100% - 36px);
  max-width: 760px;
  max-height: 85vh;
  color: #333;
  box-shadow: 0 20px 80px #0003;
}
dialog::backdrop {
  background: #0006;
}
.dialog-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 25px;
  border-bottom: 1px solid #ddd;
  background: #fff;
  position: sticky;
  top: 0;
}
.dialog-header h2 {
  font-size: 22px;
  margin-top: 4px;
}
.icon-button {
  border: 1px solid #ccc;
  border-radius: 3px;
  background: #f4f4f4;
  color: #555;
  width: 29px;
  height: 29px;
```

```
      font-size: 20px;
    }
    .help-content {
      padding: 20px 25px;
      font-size: 13px;
      color: #666;
    }
    .help-content h3 {
      font-size: 15px;
      color: #333;
      margin: 20px 0 8px;
    }
    .help-content p {
      margin: 8px 0;
      line-height: 1.8;
    }
    .help-steps {
      display: grid;
      grid-template-columns: repeat(3, 1fr);
      gap: 12px;
    }
    .help-steps > div {
      background: #f5f5f5;
      padding: 13px;
    }
    .help-steps b {
      color: #999;
      font-size: 12px;
    }
    .help-steps h3 {
      margin: 6px 0;
    }
    .help-steps p {
      font-size: 12px;
    }
    #print-output {
      display: none;
    }

    /* Bố cục theo kích thước màn hình */
    @media (min-width: 1600px) {
      .workspace {
        grid-template-columns: var(--sidebar-width, 455px)
8px minmax(0, 1fr);
```

```
      }
      .input-scroll {
        padding: 22px 27px;
      }
      .solve-bar {
        padding: 14px 27px;
      }
      .solution-panel {
        padding: 29px 40px 0;
      }
    }
    @media (max-width: 1100px) {
      .workspace {
        grid-template-columns: var(--sidebar-width, 360px)
8px minmax(0, 1fr);
      }
      .input-scroll {
        padding: 18px;
      }
      .solve-bar {
        padding: 12px 18px;
      }
      .solution-panel {
        padding: 22px 18px 0;
      }
      .exact-badge {
        display: none;
      }
      .summary-card {
        padding: 12px;
      }
      .summary-value.math {
        font-size: 20px;
      }
      .solution-section {
        padding: 20px;
      }
      .header-right > span {
        display: none;
      }
    }
    @media (max-width: 850px) {
      .workspace {
        display: block;
```

```
}
.input-panel {
  position: relative;
  height: auto;
  top: auto;
  min-height: 0;
  border-right: 0;
  border-bottom: 1px solid #bbb;
}
#problem-form {
  height: auto;
}
.input-scroll {
  padding: 22px;
}
.solve-bar {
  padding: 13px 22px;
}
.solution-panel {
  padding: 24px 22px 0;
}
.dimensions {
  grid-template-columns: 1fr 1fr 100px;
}
.report.is-empty {
  min-height: 340px;
}
.app-header {
  padding: 0 12px;
}
.header-actions {
  gap: 4px;
}
.brand strong {
  font-size: 23px;
}
.header-right {
  flex: 0.3;
}
.help-steps {
  grid-template-columns: 1fr;
}
.help-steps p {
  margin-bottom: 0;
```

```
  }
}
@media (max-width: 480px) {
  .brand strong {
    font-size: 20px;
  }
  .brand span {
    display: none;
  }
  .header-actions .btn {
    padding: 6px 8px;
    font-size: 12px;
  }
  .header-right {
    flex: 0.2;
  }
  .solution-panel {
    padding: 20px 12px 0;
  }
  .input-scroll {
    padding: 18px;
  }
  .result-summary {
    grid-template-columns: 1fr 1fr;
  }
  .summary-card:last-child {
    grid-column: span 2;
    border-top: 1px solid #ddd;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 8px;
  }
  .summary-card:nth-child(2) {
    border-right: 0;
  }
  .summary-card:last-child .summary-value {
    margin: 0;
    font-size: 19px;
  }
  .summary-foot {
    font-size: 10px;
  }
  .summary-label {
```

```
    font-size: 9px;
  }
  .result-toolbar {
    font-size: 12px;
  }
  .export-actions .btn {
    padding: 5px;
    font-size: 11px;
  }
  .solution-section {
    padding: 18px 15px;
  }
  .math-block {
    padding: 11px 13px;
    font-size: 18px;
  }
  .step-heading {
    align-items: flex-start;
    flex-wrap: wrap;
  }
  .report-cover {
    padding: 20px 15px;
  }
  .report-cover h1 {
    font-size: 23px;
  }
  .workspace-footer {
    font-size: 9px;
  }
  .dialog-header,
  .help-content {
    padding: 18px;
  }
}
@media (prefers-reduced-motion: reduce) {
  * {
    animation: none !important;
    transition: none !important;
  }
}
@media print {
  @page {
    size: A4 landscape;
    margin: 12mm;
```

```
}
body {
  background: #fff;
  color: #222;
}
.app-shell,
dialog,
.toast {
  display: none !important;
}
#print-output {
  display: block !important;
}
.report {
  border: 0;
  overflow: visible;
}
.solution-section {
  padding: 12px 0;
}
.solution-section h2,
.solution-section h3 {
  break-after: avoid;
}
.table-wrap,
.math-block {
  overflow: visible;
}
.tableau {
  font-size: 12pt;
}
.tableau th,
.tableau td {
  padding: 7px 5px;
  min-width: 0;
}
.tableau tr,
.simple-table tr {
  break-inside: avoid;
}
.tableau thead {
  display: table-header-group;
}
.tableau tfoot {
```

```
        display: table-row-group;
      }
      .notice {
        break-inside: avoid;
      }
      .report details > * {
        display: block !important;
      }
      .tableau,
      .notice {
        -webkit-print-color-adjust: exact;
        print-color-adjust: exact;
      }
      .report-cover {
        padding: 10px 0 20px;
        background: #fff;
      }
      .report-footer {
        padding: 15px 0;
      }
      .exported {
        padding: 0;
      }
    }

    /* Draggable separator: a stable grab target, also
operable with a keyboard. */
    .input-panel {
      min-width: 0;
    }
    .panel-resizer {
      position: sticky;
      top: var(--workspace-top);
      height: calc(100vh - var(--workspace-top));
      align-self: start;
      cursor: col-resize;
      touch-action: none;
      background: #eee;
      border-right: 1px solid #d0d0d0;
      display: flex;
      align-items: center;
      justify-content: center;
      z-index: 5;
    }
```

```
.panel-resizer span {
  width: 3px;
  height: 38px;
  border-radius: 3px;
  background: #aaa;
  transition: background 0.12s;
}
.panel-resizer:hover,
.panel-resizer:focus-visible,
.resizing-panels .panel-resizer {
  background: #dceaf8;
  outline: none;
}
.panel-resizer:hover span,
.panel-resizer:focus-visible span,
.resizing-panels .panel-resizer span {
  background: var(--blue);
}
body.resizing-panels,
body.resizing-panels * {
  cursor: col-resize !important;
  user-select: none !important;
}
@media (max-width: 850px) {
  .panel-resizer {
    display: none;
  }
}
@media print {
  .panel-resizer {
    display: none;
  }
}

/* Thanh tab bài toán */
.tab-bar {
  position: sticky;
  top: 48px;
  z-index: 19;
  display: flex;
  height: 38px;
  background: #ececec;
  border-bottom: 1px solid #ccc;
}
```

```
.problem-tabs {
  display: flex;
  min-width: 0;
  overflow-x: auto;
  scrollbar-width: thin;
}
.problem-tab {
  display: flex;
  flex: 0 0 auto;
  align-items: center;
  max-width: 240px;
  background: #e5e5e5;
  border-right: 1px solid #ccc;
  border-top: 2px solid transparent;
}
.problem-tab.active {
  background: #fff;
  border-top-color: var(--blue);
}
.tab-label {
  border: 0;
  background: transparent;
  color: #555;
  min-width: 100px;
  max-width: 206px;
  padding: 5px 12px;
  text-align: left;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  font-size: 13px;
}
.active .tab-label {
  color: #222;
}
.tab-close,
.tab-action {
  border: 0;
  background: transparent;
  color: #666;
  border-radius: 3px;
  width: 28px;
  height: 28px;
  flex: 0 0 auto;
```

```
  font-size: 20px;
  line-height: 1;
}
.tab-close {
  margin-right: 4px;
}
.tab-action {
  margin: 4px 3px;
}
.tab-close:hover,
.tab-action:hover {
  background: #d5d5d5;
  color: #222;
}
.tab-label:focus-visible {
  outline-offset: -3px;
}
.export-actions {
  flex-wrap: wrap;
}
@media print {
  .tab-bar {
    display: none;
  }
}

.autosave-option {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  white-space: nowrap;
  cursor: pointer;
  font-size: 12px;
}
.autosave-option input {
  width: 16px;
  height: 16px;
  margin: 0;
  accent-color: var(--blue);
  cursor: pointer;
}
.storage-notice {
  padding: 8px 20px;
  background: #fff3dd;
```

```
  color: #684b16;
  border-bottom: 1px solid #e4c998;
  font-size: 13px;
}
@media (max-width: 600px) {
  .app-header {
    height: auto;
    min-height: 48px;
    flex-wrap: wrap;
    gap: 6px;
    padding: 6px 10px;
    position: relative;
  }
  .app-header .brand {
    order: -1;
    flex: 0 0 100%;
    text-align: center;
  }
  .header-right {
    flex: 1;
  }
  .tab-bar {
    top: 0;
  }
}

/* Phương pháp hình học hai biến */
.method-label {
  margin-top: 16px;
}
.method-select {
  width: 100%;
  padding: 8px 10px;
  border: 1px solid #ccc;
  border-radius: 3px;
  background: #fafafa;
  color: var(--ink);
}
.geometry-table {
  width: 100%;
  border-collapse: collapse;
  font-size: 14px;
  margin: 16px 0;
}
```

```
.geometry-table th,
.geometry-table td {
  padding: 12px 14px;
  border: 1px solid #ddd;
  text-align: center;
}
.geometry-table th {
  background: #f4f4f4;
  font-weight: 500;
}
.geometry-table .math {
  font-size: 18px;
}
.geo-best td,
.geo-best th {
  background: #f1eafa;
  color: #6031a0;
}
.geo-swatch {
  display: inline-block;
  width: 18px;
  height: 3px;
  vertical-align: middle;
  margin-right: 7px;
}
.geo-figure {
  border: 1px solid #d4d4d4;
  border-radius: 4px;
  overflow: hidden;
  margin: 20px 0;
}
.geo-toolbar,
.geo-level {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 10px 14px;
  background: #f7f7f7;
  flex-wrap: wrap;
  font: 13px var(--sans);
}
.geo-toolbar button,
.geo-level button {
```

```
  border: 1px solid #ccc;
  background: white;
  border-radius: 3px;
  padding: 6px 12px;
  color: #444;
}
.geo-toolbar button:hover,
.geo-level button:hover {
  background: #eaf2fc;
}
.geo-toolbar label {
  display: flex;
  align-items: center;
  gap: 6px;
}
.geo-canvas {
  cursor: grab;
  touch-action: none;
  background: white;
}
.geo-canvas:active {
  cursor: grabbing;
}
.geo-canvas svg {
  width: 100%;
  height: auto;
  display: block;
  overflow: hidden;
}
.geo-point-label,
.geo-line-label {
  font: 15px var(--sans);
  paint-order: stroke;
  stroke: white;
  stroke-width: 3px;
  stroke-linejoin: round;
}
.geo-level label {
  flex: 1;
  min-width: 180px;
  color: #753bbd;
}
.geo-level input {
  display: block;
```

```
  width: 100%;
  margin-top: 10px;
  accent-color: #753bbd;
}
.geo-caption {
  font: 12px/1.6 var(--sans);
  color: #666;
  padding: 10px 14px;
  border-top: 1px solid #ddd;
}
.geo-key {
  display: inline-block;
  width: 12px;
  height: 12px;
  background: #388c4633;
  border: 1px solid #388c46;
}
.geo-system {
  display: flex;
  align-items: center;
  gap: 28px;
  padding: 16px 22px;
  margin: 18px 0;
  border-left: 2px solid #aaa;
  background: #fafafa;
  line-height: 2.1;
}
.solution-section > p {
  margin: 12px 0;
}
@media print {
  .geo-toolbar,
  .geo-level {
    display: none;
  }
  .geo-figure {
    break-inside: avoid;
  }
  .geo-canvas {
    max-width: 740px;
    margin: auto;
  }
  .geometry-table {
    font-size: 11px;
```

```
  }
  .geometry-table th,
  .geometry-table td {
    padding: 6px;
  }
}

.geo-plot {
  position: relative;
}
.geo-probe {
  position: absolute;
  z-index: 2;
  pointer-events: none;
  max-width: 230px;
  padding: 7px 11px;
  border: 1px solid #bbb;
  border-radius: 5px;
  background: #fff;
  color: #222;
  box-shadow: 0 3px 12px #0002;
  font: 14px var(--math);
  overflow-wrap: anywhere;
}
.geo-probe-dot {
  position: absolute;
  z-index: 2;
  width: 9px;
  height: 9px;
  border: 2px solid white;
  border-radius: 50%;
  background: #222;
  transform: translate(-50%, -50%);
  pointer-events: none;
  box-shadow: 0 0 0 1px #555;
}
.geo-animation-status {
  font: 12px/1.5 var(--sans);
  color: #753bbd;
  padding: 6px 14px;
  min-height: 30px;
  background: #faf7ff;
}
.geo-animation-status:empty {
```

```
  display: none;
}
.geo-key {
  background: #c9e9d0;
}
.geo-key.outside {
  background: #f7dada;
  border-color: #c74440;
}
.geo-toolbar button[aria-pressed='true'] {
  color: #753bbd;
  border-color: #bda1dc;
  background: #faf7ff;
}
@media print {
  .geo-probe,
  .geo-probe-dot,
  .geo-animation-status {
    display: none;
  }
}

.geo-probe b {
  display: block;
  font: 11px var(--sans);
  color: #777;
  margin-bottom: 4px;
}
.geo-probe .math {
  display: block;
  white-space: normal;
  font-size: 17px;
}
.geo-probe small {
  display: block;
  font: 11px var(--sans);
  color: #777;
  margin-top: 5px;
}
```

js/app.js

```
'use strict';
(() => {
  const $ = (s) => document.querySelector(s),
    $$ = (s) => Array.from(document.querySelectorAll(s));
  const { esc, num, vector } = Report;
  let n = 2,
    m = 2,
    result = null,
    worker = null,
    working = false,
    dirty = false,
    job = 0;
  let toastTimer,
    activeMethod = 'bigm',
    activeConstant = '0';
  let lastExampleIndex = -1,
    disposeGraph = null;
  const tabs = [],
    closedTabs = [];
  let activeTabId = null,
    nextTabId = 1;
  let autosave = false,
    sessionReady = false,
    saveTimer;

  // Dựng các bảng nhập hệ số và điều kiện dấu
  function inputs(raw) {
    n = raw.c.length;
    m = raw.a.length;
    $('#n').value = n;
    $('#m').value = m;
    $('#constraint-count').textContent = m + ' dòng';
    const wide = n > 3 ? `style="min-width:${(n + 1) *
65}px"` : '';
    $('#objective-input').innerHTML = /* HTML */ `<table
class="input-table" ${wide}>
        <thead>
          <tr>
            <th aria-label="Loại hệ số"></th>
            ${raw.c.map((v, j) => /* HTML */ `<th
scope="col">x<sub>${j + 1}</sub></th>`).join('')}
          </tr>
```

```
</thead>
<tbody>
  <tr>
    <th scope="row">c_j</th>
    ${raw.c
      .map(
        (v, j) =>
          /* HTML */ `<td>
            <input
              type="text"
              inputmode="text"
              data-table="objective"
              data-row="0"
              data-col="${j}"
              data-c="${j}"
              value="${esc(v)}"
              aria-label="Hệ số mục tiêu x${j + 1}"
              autocomplete="off"
              spellcheck="false"
            />
          </td>`
      )
      .join('')}
  </tr>
  <tr class="domain-row">
    <th scope="row">miền</th>
    ${raw.domains
      .map(
        (v, j) =>
          /* HTML */ `<td>
            <select
              data-table="objective"
              data-row="1"
              data-col="${j}"
              data-domain="${j}"
              aria-label="Miền biến x${j + 1}"
            >
              ${[
                ['pos', '≥ 0'],
                ['neg', '≤ 0'],
                ['free', 'Tự do'],
              ]
                .map(
                  ([val, label]) =>
```

```
                                /* HTML */ `<option
value="${val}" ${v === val ? 'selected' : ''}>
                                  ${label}
                                </option>`
                            )
                            .join('')}
                        </select>
                      </td>`
                  )
                  .join('')}
              </tr>
            </tbody>
          </table>`;
          $('#constraint-input').innerHTML = m
            ? /* HTML */ `<table
                class="input-table"
                ${n > 2 ? `style="min-width:${(n + 2) * 58 +
35}px"` : ''}
              >
                <thead>
                  <tr>
                    <th scope="col">#</th>
                    ${raw.c
                      .map((v, j) => /* HTML */ `<th
scope="col">x<sub>${j + 1}</sub></th>`)
                      .join('')}
                    <th class="relation" scope="col">Dấu</th>
                    <th scope="col">b<sub>i</sub></th>
                  </tr>
                </thead>
                <tbody>
                  ${raw.a
                    .map(
                      (row, i) =>
                        /* HTML */ `<tr>
                          <th scope="row">${i + 1}</th>
                          ${row
                            .map(
                              (v, j) =>
                                /* HTML */ `<td>
                                  <input
                                    type="text"
                                    data-table="constraint"
                                    data-row="${i}"
```

```
                                data-col="${j}"
                                data-a="${i},${j}"
                                value="${esc(v)}"
                                aria-label="Ràng buộc ${i +
1}, hệ số x${j + 1}"
                                autocomplete="off"
                                spellcheck="false"
                              />
                            </td>`
                        )
                        .join('')}
                      <td>
                        <select
                          data-table="constraint"
                          data-row="${i}"
                          data-col="${n}"
                          data-rel="${i}"
                          aria-label="Quan hệ ràng buộc ${i
+ 1}"
                        >
                          ${['<=', '=', '>=']
                            .map(
                              (s) =>
                                /* HTML */ `<option
                                  value="${esc(s)}"
                                  ${s === raw.rel[i] ?
'selected' : ''}
                                >
                                  ${Report.rel(s)}
                                </option>`
                            )
                            .join('')}
                        </select>
                      </td>
                      <td class="rhs-input">
                        <input
                          type="text"
                          data-table="constraint"
                          data-row="${i}"
                          data-col="${n + 1}"
                          data-b="${i}"
                          value="${esc(raw.b[i])}"
                          aria-label="Vế phải ràng buộc ${i
+ 1}"
```

```
                            autocomplete="off"
                            spellcheck="false"
                          />
                        </td>
                      </tr>`
                  )
                  .join('')}
              </tbody>
            </table>`
          : '<div class="field-hint"
style="padding:14px;margin:0">Chưa có ràng buộc.</div>';
        $('#add-row').disabled = m >= 40;
        $('#remove-row').disabled = m === 0;
      }
      // Thu thập, nạp và kiểm tra đề bài
      function collect() {
        return {
          title: $('#problem-title').value,
          sense: $('[name=sense]:checked').value,
          constant: activeConstant,
          c: $$('[data-c]').map((e) => e.value),
          domains: $$('[data-domain]').map((e) => e.value),
          a: Array.from({ length: m }, (_, i) =>
            Array.from({ length: n }, (_, j) => $(`[data-
a="${i},${j}"]`).value)
          ),
          b: $$('[data-b]').map((e) => e.value),
          rel: $$('[data-rel]').map((e) => e.value),
        };
      }
      function load(raw, method) {
        $('#problem-title').value = raw.title;
        $(`[name=sense][value=${raw.sense}]`).checked = true;
        activeConstant = raw.constant ?? '0';
        if (method) activeMethod = method;
        $('#solve-method').value = activeMethod;
        inputs(raw);
        clearError();
        markDirty();
      }
      function resize() {
        const nn = Number($('#n').value),
          mm = Number($('#m').value);
        if (!Number.isInteger(nn) || nn < 1 || nn > 20 ||
```

```
!Number.isInteger(mm) || mm < 0 || mm > 40)
          throw new Error('Chọn 1–20 biến và 0–40 ràng
buộc.');
        if (nn === n && mm === m) return;
        const p = collect();
        p.c = Array.from({ length: nn }, (_, j) => p.c[j] ??
'0');
        p.domains = Array.from({ length: nn }, (_, j) =>
p.domains[j] ?? 'pos');
        p.a = Array.from({ length: mm }, (_, i) =>
          Array.from({ length: nn }, (_, j) => p.a[i]?.[j] ??
'0')
        );
        p.b = Array.from({ length: mm }, (_, i) => p.b[i] ??
'0');
        p.rel = Array.from({ length: mm }, (_, i) => p.rel[i]
?? '<=');
        inputs(p);
        markDirty();
      }
      function clearError() {
        $('#input-error').hidden = true;
        $$('[aria-invalid]').forEach((e) =>
e.removeAttribute('aria-invalid'));
      }
      function error(e) {
        $('#input-error').textContent = e.message ||
String(e);
        $('#input-error').hidden = false;
        $('#input-error').scrollIntoView({ block: 'nearest',
behavior: 'smooth' });
      }
      function read() {
        resize();
        clearError();
        for (const el of [...$$('[data-c]'), ...$$('[data-
a]'), ...$$('[data-b]')]) {
          try {
            LP.Q.parse(el.value);
          } catch (e) {
            el.setAttribute('aria-invalid', 'true');
            el.focus();
            throw new Error((el.getAttribute('aria-label') ||
'Hệ số') + ': ' + e.message);
```

```
          }
        }
        const raw = collect();
        LP.parseProblem(raw);
        return raw;
      }
      function toast(text) {
        clearTimeout(toastTimer);
        $('#toast').textContent = text;
        $('#toast').hidden = false;
        toastTimer = setTimeout(() => ($('#toast').hidden =
true), 4000);
      }
      function markDirty() {
        dirty = true;
        $('#stale-banner').hidden = !result;
        $('#print-btn').disabled = true;
        updateTabLabel();
        scheduleSave();
      }
      function setBusy(v) {
        working = v;
        $('#busy-banner').hidden = !v;
        $('#cancel-btn').hidden = !v;
        $('#solve-btn').hidden = v;
        $$('#problem-form input,#problem-form
select,#problem-form button').forEach((e) => {
          if (e.id !== 'cancel-btn') e.disabled = v;
        });
        $('#new-btn').disabled = v;
        $('#open-btn').disabled = v;
        $('#save-btn').disabled = v;
        renderTabs();
        $('#print-btn').disabled = v || !result || dirty;
        if (!v) {
          $('#add-row').disabled = m >= 40;
          $('#remove-row').disabled = m === 0;
        }
      }
      // Chạy bộ giải trong Web Worker
      function startSolve() {
        if (working) return;
        let raw;
        try {
```

```
          raw = read();
          if (activeMethod === 'geometry' && raw.c.length !==
2)
            throw new Error('Phương pháp hình học yêu cầu
đúng 2 biến.');
        } catch (e) {
          error(e);
          return;
        }
        const method = activeMethod,
          id = ++job;
        setBusy(true);
        const complete = (r) => {
          if (job !== id) return;
          worker?.terminate();
          worker = null;
          setBusy(false);
          showResult(r);
        };
        const fail = (e) => {
          if (job !== id) return;
          worker?.terminate();
          worker = null;
          setBusy(false);
          if (method === 'geometry') {
            clearResult();
            dirty = true;
          }
          error(e);
        };
        // A self-contained Blob worker works when index.html
is opened directly from disk.
        try {
          const source =
`(${installLP.toString()})(self);(${installGeometry.toString()
})(self);self.onmessage=function(e){try{const
r=e.data.method==='geometry'?Geometry.solveBounded(e.data.prob
lem):new
LP.Solver().solve(e.data.problem,e.data.method);self.postMessa
ge({result:JSON.stringify(r)});}catch(err){self.postMessage({e
rror:err.message});}};`;
          const url = URL.createObjectURL(new Blob([source],
{ type: 'text/javascript' }));
          worker = new Worker(url);
```

```
          URL.revokeObjectURL(url);
          worker.onmessage = (e) =>
            e.data.error
              ? fail(new Error(e.data.error))
              : complete(JSON.parse(e.data.result,
LP.revive));
          worker.onerror = (e) => {
            e.preventDefault();
            fail(
              new Error(
                'Trình duyệt không chạy được bộ tính toán
nền. Hãy mở index.html bằng Chrome, Edge hoặc Firefox phiên
bản có hỗ trợ Web Worker.'
              )
            );
          };
          worker.postMessage({ problem: raw, method });
        } catch (e) {
          fail(new Error('Không khởi tạo được Web Worker: ' +
e.message));
        }
      }
      // Hiển thị kết quả và chuẩn bị tệp tải về
      function renderReport(r, interactive = true) {
        return r.method === 'geometry' ?
GeometryView.report(r, interactive) : Report.all(r);
      }
      function showResult(r) {
        disposeGraph?.();
        disposeGraph = null;
        result = r;
        dirty = false;
        $('#stale-banner').hidden = true;
        $('#print-btn').disabled = false;
        $('#solution-subtitle').textContent = r.p.title;
        $('#solution-subtitle').hidden = false;
        const ok = r.status === 'optimal';
        $('#result-summary').innerHTML = /* HTML */ `<div
            class="summary-card ${ok ? 'primary-card' :
'warning-card'}"
          >
            <div class="summary-label">${ok ? 'GIÁ TRỊ TỐI
ƯU' : 'TRẠNG THÁI BÀI TOÁN'}</div>
            <div class="summary-value">
```

```
              ${ok
                ? 'f* = ' + num(r.value)
                : { infeasible: 'Vô nghiệm', unbounded:
'Không bị chặn', limit: 'Chưa kết luận' }[
                    r.status
                  ]}
            </div>
            <div class="summary-foot">
              ${ok
                ? r.p.sense.toUpperCase() +
                  ' . ' +
                  (r.multiplicity === 'multiple'
                    ? 'Nhiều nghiệm tối ưu'
                    : r.multiplicity === 'unique'
                      ? 'Nghiệm duy nhất'
                      : 'Đã xác minh khả thi')
                : 'Xem kết luận bên dưới'}
            </div>
          </div>
          <div class="summary-card">
            <div class="summary-label">${ok ? 'NGHIỆM TỐI ƯU'
: 'PHƯƠNG ÁN KHẢ THI'}</div>
            <div class="summary-value math">${r.x ?
vector(r.x) : '—'}</div>
            <div class="summary-foot">${r.p.n} biến của bài
toán gốc</div>
          </div>
          <div class="summary-card">
            <div class="summary-label">
              ${r.method === 'geometry' ? 'MIỀN NGHIỆM' : 'SỐ
LẦN ĐỔI CƠ SỞ'}
            </div>
            <div class="summary-value">
              ${r.method === 'geometry'
                ? r.status === 'infeasible'
                  ? 'Rỗng'
                  : r.bounded
                    ? 'Bị chặn'
                    : 'Không bị chặn'
                : r.pivots +
                  '<small style="font:11px var(--
sans);color:#888;margin-left:7px">lần</small>'}
            </div>
            <div class="summary-foot">
```

```
              ${r.method === 'geometry' ? 'Phương pháp hình
học' : Report.methodName(r)}
            </div>
          </div>`;
        $('#report').innerHTML = renderReport(r);
        if (r.method === 'geometry') disposeGraph =
GeometryView.mount($('#report'), r);
        $('#report').classList.remove('is-empty');
        $('#result-summary').hidden = false;
      }
      function download(name, contents, type) {
        const blob = new Blob([contents], { type }),
          url = URL.createObjectURL(blob),
          a = document.createElement('a');
        a.href = url;
        a.download = name;
        document.body.append(a);
        a.click();
        a.remove();
        setTimeout(() => URL.revokeObjectURL(url), 2000);
      }
      // Sự kiện nhập liệu, mở/lưu bài, xuất và in
      $('#problem-form').addEventListener('submit', (e) => {
        e.preventDefault();
        startSolve();
      });
      $('#problem-form').addEventListener('input', (e) => {
        markDirty();
        if (e.target.hasAttribute('aria-invalid'))
e.target.removeAttribute('aria-invalid');
      });
      $('#problem-form').addEventListener('change', (e) => {
        if (e.target.tagName === 'SELECT') markDirty();
      });
      $('#resize-btn').addEventListener('click', () => {
        try {
          resize();
          clearError();
        } catch (e) {
          error(e);
        }
      });
      $('#add-row').addEventListener('click', () => {
        $('#m').value = Math.min(40, m + 1);
```

```
        try {
          resize();
        } catch (e) {
          error(e);
        }
      });
      $('#remove-row').addEventListener('click', () => {
        $('#m').value = Math.max(0, m - 1);
        try {
          resize();
        } catch (e) {
          error(e);
        }
      });
      $('#solve-method').addEventListener('change', () => {
        activeMethod = $('#solve-method').value;
        lastExampleIndex = -1;
        markDirty();
      });
      $('#example').addEventListener('click', () => {
        if (working) return;
        const geometric = activeMethod === 'geometry';
        const sample = geometric
          ? randomGeometryExample(lastExampleIndex)
          : randomExample(lastExampleIndex);
        lastExampleIndex = sample.index;
        load(sample.problem, geometric ? 'geometry' :
'bigm');
        toast('Đã nạp bài mẫu.');
      });
      $('#cancel-btn').addEventListener('click', () => {
        job++;
        worker?.terminate();
        worker = null;
        setBusy(false);
        toast('Đã dừng tính toán.');
      });
      function exportJSON() {
        if (working) return;
        try {
          const raw = read();
          download(
            ((raw.title || 'BaiToanDonHinh')
              .replace(/[^\p{L}\p{N} _-]/gu, '')
```

```
              .slice(0, 80)
              .trim() || 'BaiToanDonHinh') + '.json',
            JSON.stringify({ format: 'DonHinhStudio-1',
problem: raw, method: activeMethod }, null, 2),
            'application/json;charset=utf-8'
          );
          toast('Đã xuất JSON. Dùng Mở bài để nhập lại.');
        } catch (e) {
          error(e);
        }
      }
      $('#save-btn').addEventListener('click', exportJSON);
      $('#open-btn').addEventListener('click', () =>
$('#file-input').click());
      $('#file-input').addEventListener('change', async (e)
=> {
        try {
          const file = e.target.files[0];
          if (!file || working) return;
          if (file.size > 1000000) throw new Error('Tệp bài
toán không được vượt quá 1 MB.');
          const data = JSON.parse(await file.text());
          if (data.format !== 'DonHinhStudio-1') throw new
Error('Không đúng định dạng tệp bài toán.');
          const parsed = LP.parseProblem(data.problem);
          if (parsed.m > 40) throw new Error('Giao diện hỗ
trợ tối đa 40 ràng buộc.');
          if (!['bigm', 'two',
'geometry'].includes(data.method))
            throw new Error('Phương pháp không hợp lệ.');
          if (working) throw new Error('Hãy đợi tính toán
hoàn tất hoặc nhấn Dừng rồi mở lại tệp.');
          createTab(LP.rawProblem(parsed), data.method);
          toast('Đã mở bài toán.');
        } catch (err) {
          error(err);
        } finally {
          e.target.value = '';
        }
      });
      $('#print-btn').addEventListener('click', () => {
        if (!result || dirty) return;
        $('#print-output').innerHTML = renderReport(result,
false);
```

```
        $$('#print-output details').forEach((e) => (e.open =
true));
        window.print();
      });
      // Also prepare the report for the browser's Ctrl+P
shortcut.
      window.addEventListener('beforeprint', () => {
        if (result && !dirty) {
          $('#print-output').innerHTML = renderReport(result,
false);
          $$('#print-output details').forEach((e) => (e.open
= true));
        } else
          $('#print-output').innerHTML =
            '<section class="solution-section"><h1>Cần giải
bài toán trước khi in.</h1></section>';
      });
      $('#help-btn').addEventListener('click', () =>
$('#help-dialog').showModal());
      $('#close-help').addEventListener('click', () =>
$('#help-dialog').close());
      $('#help-dialog').addEventListener('click', (e) => {
        if (e.target === $('#help-dialog')) {
          const r = e.target.getBoundingClientRect();
          if (e.clientX < r.left || e.clientX > r.right ||
e.clientY < r.top || e.clientY > r.bottom)
            e.target.close();
        }
      });
      document.addEventListener('keydown', (e) => {
        if ((e.ctrlKey || e.metaKey) && e.key === 'Enter') {
          e.preventDefault();
          startSolve();
        }
      });
      // Dán vùng hệ số từ bảng tính
      $('#problem-form').addEventListener('paste', (e) => {
        const target = e.target,
          text = e.clipboardData?.getData('text/plain');
        if (!target.dataset.table || !text ||
!/[\t\n]/.test(text)) return;
        e.preventDefault();
        try {
          const rows = text.replace(/\r/g,
```

```
'').replace(/\n+$/, '').split('\n'),
            r0 = Number(target.dataset.row),
            c0 = Number(target.dataset.col),
            changes = [];
          rows.forEach((row, i) =>
            row.split('\t').forEach((text, j) => {
              const el = $(
                `[data-table="${target.dataset.table}"][data-
row="${r0 + i}"][data-col="${c0 + j}"]`
              );
              if (!el)
                throw new Error('Vùng dán vượt kích thước
bảng. Hãy tăng số biến/ràng buộc trước.');
              let value = text.trim();
              if (el.hasAttribute('data-domain')) {
                value = {
                  '>=0': 'pos',
                  '≥0': 'pos',
                  '<=0': 'neg',
                  '≤0': 'neg',
                  tựdo: 'free',
                  free: 'free',
                  pos: 'pos',
                  neg: 'neg',
                }[value.toLowerCase().replace(/\s/g, '')];
                if (!value) throw new Error('Miền biến phải
là >=0, <=0 hoặc free.');
              } else if (el.hasAttribute('data-rel')) {
                value = { '≤': '<=', '≥': '>=', '<=': '<=',
'>=': '>=', '=': '=' }[value];
                if (!value) throw new Error('Dấu quan hệ phải
là <=, = hoặc >=.');
              } else LP.Q.parse(value);
              changes.push([el, value]);
            })
          );
          changes.forEach(([el, value]) => (el.value =
value));
          markDirty();
          clearError();
          toast('Đã dán ' + changes.length + ' ô.');
        } catch (err) {
          error(err);
        }
```

```
      });

      // Mỗi tab lưu cả dữ liệu đang nhập và kết quả gần
nhất.
      function blankProblem() {
        return {
          title: '',
          sense: 'max',
          constant: '0',
          c: ['0', '0'],
          domains: ['pos', 'pos'],
          a: [
            ['0', '0'],
            ['0', '0'],
          ],
          b: ['0', '0'],
          rel: ['<=', '<='],
        };
      }
      function tabTitle(tab) {
        return tab.raw.title.trim() || `Bài toán ${tab.id}`;
      }
      function updateTabLabel() {
        const tab = tabs.find((item) => item.id ===
activeTabId);
        if (!tab) return;
        tab.raw.title = $('#problem-title').value;
        const button = $(`#tab-${tab.id}`);
        if (button) {
          button.textContent = tabTitle(tab);
          button.title = tabTitle(tab);
          $(`[data-close-
tab="${tab.id}"]`).setAttribute('aria-label', 'Đóng ' +
tabTitle(tab));
        }
      }
      function renderTabs() {
        $('#problem-tabs').innerHTML = tabs
          .map((tab) => {
            const active = tab.id === activeTabId;
            return `<div class="problem-tab${active ? '
active' : ''}" role="presentation">
              <button id="tab-${tab.id}" type="button"
role="tab" class="tab-label"
```

```
              data-tab="${tab.id}" aria-selected="${active}"
aria-controls="workspace-panel"
              tabindex="${active ? 0 : -1}"
title="${esc(tabTitle(tab))}" ${working ? 'disabled' :
''}>${esc(tabTitle(tab))}</button>
            <button type="button" class="tab-close" data-
close-tab="${tab.id}"
              aria-label="Đóng ${esc(tabTitle(tab))}"
title="Đóng tab" ${working ? 'disabled' : ''}>×</button>
          </div>`;
          })
          .join('');
        $('#workspace-panel').setAttribute('aria-labelledby',
'tab-' + activeTabId);
        $('#add-tab').disabled = working;
        $('#reopen-tab').disabled = working ||
!closedTabs.length;
      }
      function captureTab() {
        const tab = tabs.find((item) => item.id ===
activeTabId);
        if (!tab) return;
        // Không dùng read(): chuyển tab vẫn giữ được ô trống
hoặc phân số đang gõ dở.
        Object.assign(tab, {
          raw: collect(),
          method: activeMethod,
          result,
          dirty,
          lastExampleIndex,
          dimensions: [$('#n').value, $('#m').value],
          scroll: [$('.input-scroll').scrollTop,
window.scrollY],
          error: $('#input-error').hidden ? '' : $('#input-
error').textContent,
        });
      }
      function clearResult() {
        disposeGraph?.();
        disposeGraph = null;
        result = null;
        $('#result-summary').innerHTML = '';
        $('#result-summary').hidden = true;
        $('#solution-subtitle').textContent = '';
```

```
        $('#solution-subtitle').hidden = true;
        $('#report').classList.add('is-empty');
        $('#report').innerHTML =
          '<div class="empty-state"><h2>Chưa có lời
giải</h2><p>Nhập dữ liệu hoặc tạo bài mẫu, sau đó nhấn
<b>Giải</b>.</p></div>';
        $('#stale-banner').hidden = true;
        $('#print-btn').disabled = true;
      }
      function restoreTab(tab, focus = false) {
        activeTabId = tab.id;
        clearResult();
        load(tab.raw, tab.method);
        lastExampleIndex = tab.lastExampleIndex;
        if (tab.result) showResult(tab.result);
        dirty = tab.dirty;
        $('#stale-banner').hidden = !result || !dirty;
        $('#print-btn').disabled = !result || dirty;
        if (tab.dimensions) {
          $('#n').value = tab.dimensions[0];
          $('#m').value = tab.dimensions[1];
        }
        $('#input-error').textContent = tab.error || '';
        $('#input-error').hidden = !tab.error;
        $('#print-output').innerHTML = '';
        renderTabs();
        const button = $(`#tab-${tab.id}`);
        if (focus) button.focus({ preventScroll: true });
        // Chỉ cuộn thanh tab; không cuộn cả trang khi đưa
tab đang chọn vào tầm nhìn.
        const list = $('#problem-tabs');
        const item = button.parentElement;
        if (item.offsetLeft < list.scrollLeft)
list.scrollLeft = item.offsetLeft;
        else if (item.offsetLeft + item.offsetWidth >
list.scrollLeft + list.clientWidth)
          list.scrollLeft = item.offsetLeft +
item.offsetWidth - list.clientWidth;
        $('.input-scroll').scrollTop = tab.scroll?.[0] || 0;
        window.scrollTo(0, tab.scroll?.[1] || 0);
        scheduleSave();
      }
      function createTab(raw = blankProblem(), method =
'bigm') {
```

```
        if (working) return;
        captureTab();
        const tab = { id: nextTabId++, raw, method, result:
null, dirty: false, lastExampleIndex: -1 };
        tabs.push(tab);
        restoreTab(tab);
      }
      function selectTab(id, focus = true) {
        if (working) return;
        const tab = tabs.find((item) => item.id === id);
        if (!tab) return;
        captureTab();
        restoreTab(tab, focus);
      }
      function closeTab(id) {
        if (working) return;
        const index = tabs.findIndex((tab) => tab.id === id);
        if (index < 0) return;
        captureTab();
        const [closed] = tabs.splice(index, 1);
        closedTabs.push(closed);
        if (closedTabs.length > 10) closedTabs.shift();
        if (!tabs.length) {
          activeTabId = null;
          createTab();
        } else if (id === activeTabId)
restoreTab(tabs[Math.min(index, tabs.length - 1)], true);
        else {
          renderTabs();
          $(`#tab-${activeTabId}`).focus({ preventScroll:
true });
        }
        scheduleSave();
        toast('Đã đóng tab. Nhấn ↶ để mở lại.');
      }
      $('#problem-tabs').addEventListener('click', (e) => {
        const close = e.target.closest('[data-close-tab]');
        const tab = e.target.closest('[data-tab]');
        if (close) closeTab(Number(close.dataset.closeTab));
        else if (tab) selectTab(Number(tab.dataset.tab));
      });
      $('#problem-tabs').addEventListener('keydown', (e) => {
        if (working || !e.target.matches('[data-tab]'))
return;
```

```
        const index = tabs.findIndex((tab) => tab.id ===
activeTabId);
        let next;
        if (e.key === 'ArrowRight') next = (index + 1) %
tabs.length;
        else if (e.key === 'ArrowLeft') next = (index - 1 +
tabs.length) % tabs.length;
        else if (e.key === 'Home') next = 0;
        else if (e.key === 'End') next = tabs.length - 1;
        else return;
        e.preventDefault();
        selectTab(tabs[next].id);
      });
      $('#reopen-tab').addEventListener('click', () => {
        if (working || !closedTabs.length) return;
        captureTab();
        const tab = closedTabs.pop();
        tabs.push(tab);
        restoreTab(tab, true);
      });
      $('#new-btn').addEventListener('click', () =>
createTab());
      $('#add-tab').addEventListener('click', () =>
createTab());
      // Ghi sau một nhịp nhập; luôn ghi ngay trước khi trang
chuyển sang nền hoặc đóng.
      function storageNotice(message = '') {
        $('#storage-notice').textContent = message;
        $('#storage-notice').hidden = !message;
      }
      function saveSession() {
        clearTimeout(saveTimer);
        if (!autosave || !sessionReady) return;
        try {
          captureTab();
          WorkspaceSession.save(tabs, closedTabs,
activeTabId);
          storageNotice();
          $('#autosave-label').title = 'Đã lưu các tab và dữ
liệu nhập trên trình duyệt này.';
        } catch (e) {
          storageNotice(
            'Chưa lưu được phiên làm việc. Bộ nhớ trình duyệt
có thể đã đầy hoặc bị chặn. Bạn có thể xuất JSON để giữ đề.'
```

```
          );
          $('#autosave-label').title = 'Chưa lưu được thay
đổi mới nhất.';
        }
      }
      function scheduleSave() {
        if (!autosave || !sessionReady) return;
        clearTimeout(saveTimer);
        saveTimer = setTimeout(saveSession, 350);
      }
      $('#autosave-toggle').addEventListener('change', () =>
{
        clearTimeout(saveTimer);
        const enabled = $('#autosave-toggle').checked;
        try {
          if (enabled) {
            captureTab();
            // Chỉ xác nhận bật khi ghi thành công; bản cũ
vẫn nguyên vẹn nếu hết dung lượng.
            WorkspaceSession.save(tabs, closedTabs,
activeTabId);
          } else WorkspaceSession.clear();
          autosave = enabled;
          storageNotice();
          $('#autosave-label').title = enabled
            ? 'Tự động lưu các tab và dữ liệu nhập trên trình
duyệt này.'
            : 'Tự động lưu đang tắt.';
          toast(
            enabled
              ? 'Đã bật tự động lưu phiên làm việc.'
              : 'Đã tắt tự động lưu và xóa bản lưu trên trình
duyệt.'
          );
        } catch (e) {
          $('#autosave-toggle').checked = autosave;
          storageNotice(
            enabled
              ? 'Không bật được tự động lưu. Hãy kiểm tra
quyền lưu trữ hoặc dung lượng trình duyệt.'
              : 'Không xóa được bản lưu tự động. Hãy kiểm tra
quyền lưu trữ của trình duyệt rồi thử lại.'
          );
        }
```

```
      });
      document.addEventListener('visibilitychange', () => {
        if (document.visibilityState === 'hidden')
saveSession();
      });
      window.addEventListener('pagehide', saveSession);
      $('#problem-form').addEventListener('focusout',
saveSession);
      // Khôi phục đề, không khôi phục Worker và không tự
chạy tính toán.
      function startSession() {
        $('#autosave-toggle').checked = false;
        try {
          const saved = WorkspaceSession.load();
          if (saved) {
            const hydrate = (tab) => ({ ...tab, result: null,
dirty: false });
            tabs.push(...saved.tabs.map(hydrate));

closedTabs.push(...saved.closedTabs.map(hydrate));
            nextTabId = saved.nextTabId;
            restoreTab(tabs.find((tab) => tab.id ===
saved.activeTabId));
            autosave = true;
            $('#autosave-toggle').checked = true;
            $('#autosave-label').title =
              'Đã khôi phục phiên làm việc. Thay đổi tiếp
theo sẽ được tự động lưu.';
          }
        } catch (e) {
          tabs.length = 0;
          closedTabs.length = 0;
          activeTabId = null;
          nextTabId = 1;
          storageNotice(
            'Không khôi phục được phiên đã lưu hoặc trình
duyệt đang chặn lưu trữ. Bạn vẫn có thể nhập bài và mở JSON;
bật Tự động lưu để tạo bản lưu mới.'
          );
        }
        if (!tabs.length) createTab();
        sessionReady = true;
      }
      startSession();
```

```
})();
```

js/examples.js

```
/* Examples 1–3 follow Bai2.pdf, slides 20, 29 and 34. */
window.EXAMPLES = (() => {
  const make = (title, c, a, b, rel, sense = 'max',
domains = null) => ({
    title,
    c: c.split(' '),
    a: a.map((r) => r.split(' ')),
    b: b ? b.split(' ') : [],
    rel: rel.split('').map((v) => ({ L: '<=', E: '=', G:
'>=' })[v]),
    sense,
    constant: '0',
    domains: domains || c.split(' ').map(() => 'pos'),
  });
  return [
    make('01 · Kế hoạch sản xuất', '4 5', ['2 1', '1 2',
'0 1'], '8 7 3', 'LLL'),
    make(
      '02 · Chính tắc – phương pháp M',
      '2 1 -1 -1',
      ['1 -1 2 -1', '2 1 -3 1', '1 1 1 1'],
      '2 6 7',
      'EEE'
    ),
    make(
      '03 · Ràng buộc hỗn hợp ≤, =, ≥',
      '1 4 -3',
      ['2 1 3', '4 -3 -2', '1 2 -1'],
      '7 9 2',
      'LGE'
    ),
    make('04 · Tối thiểu hóa với ràng buộc ≥', '2 3', ['1
1', '1 3'], '4 6', 'GG', 'min'),
    make(
      '05 · Biến tự do và biến không dương',
      '1 -1',
      ['1 1', '-1 0', '0 -1'],
      '2 1 2',
      'ELL',
      'max',
      ['free', 'neg']
    ),
```

```
        make('06 · Ràng buộc có vế phải âm', '1', ['-1',
'1'], '-2 5', 'LL'),
        make('07 · Bài toán vô nghiệm', '1', ['1', '1'], '1
2', 'LG'),
        make('08 · Hàm mục tiêu không bị chặn', '1 1', ['1 -
1'], '1', 'G'),
        make('09 · Vô số nghiệm tối ưu', '1 1', ['1 1'], '4',
'L'),
        make('10 · Ràng buộc phụ thuộc', '1 1', ['1 1', '2
2'], '2 4', 'EE'),
        make(
          '11 · Suy biến và chống chu trình',
          '10 -57 -9 -24',
          ['1/2 -11/2 -5/2 9', '1/2 -3/2 -1/2 1', '1 0 0 0'],
          '0 0 1',
          'LLL'
        ),
        make(
          '12 · Đề cương, ví dụ 2.5 — theo đề gốc',
          '3 -1 -2',
          ['-1 3 1', '3 -4 8', '4 -2 0'],
          '7 10 12',
          'LLL'
        ),
      ];
    })();

    // Chọn ngẫu nhiên, không lặp lại bài vừa nạp. Tên nội bộ
không đưa lên giao diện.
    window.randomExample = (previousIndex = -1) => {
      const candidates = EXAMPLES.map((_, index) =>
index).filter(
        (index) => EXAMPLES.length === 1 || index !==
previousIndex
      );
      const index = candidates[Math.floor(Math.random() *
candidates.length)];
      const problem =
JSON.parse(JSON.stringify(EXAMPLES[index]));
      problem.title = 'Bài toán mẫu';
      return { index, problem };
    };

    // Mẫu hai biến dùng riêng cho chế độ hình học.
```

```
    window.GEOMETRY_EXAMPLES = [
      {
        c: ['2', '5'],
        a: [
          ['1', '2'],
          ['2', '1'],
          ['0', '1'],
        ],
        b: ['8', '10', '3'],
        rel: ['<=', '<=', '<='],
        sense: 'max',
      },
      {
        c: ['2', '5'],
        a: [
          ['1', '2'],
          ['2', '1'],
          ['0', '1'],
          ['1', '0'],
          ['0', '1'],
        ],
        b: ['8', '10', '3', '8', '7'],
        rel: ['>=', '>=', '>=', '<=', '<='],
        sense: 'min',
      },
      { c: ['1', '1'], a: [['1', '1']], b: ['4'], rel:
['<='], sense: 'max' },
      { c: ['1', '1'], a: [['1', '-1'], ['1', '1']], b: ['1',
'6'], rel: ['>=', '<='], sense: 'max' },
      {
        c: ['1', '2'],
        a: [
          ['1', '1'],
          ['1', '1'],
        ],
        b: ['6', '3'],
        rel: ['<=', '>='],
        sense: 'min',
      },
      { c: ['3', '1'], a: [['1', '1']], b: ['4'], rel: ['='],
sense: 'min' },
    ];
    window.randomGeometryExample = (previousIndex = -1) => {
      const pool = GEOMETRY_EXAMPLES.map((_, i) =>
```

```
i).filter((i) => i !== previousIndex);
       const index = pool[Math.floor(Math.random() *
pool.length)];
       return {
         index,
         problem: {
           title: 'Bài toán mẫu',
           constant: '0',
           domains: ['pos', 'pos'],

...JSON.parse(JSON.stringify(GEOMETRY_EXAMPLES[index])),
         },
       };
     };
```

js/geometry-view.js

```
// Báo cáo hình học và đồ thị SVG ngoại tuyến.
'use strict';
window.GeometryView = (() => {
  const { esc, num, expr, vector, rel } = Report;
  const { Z, O } = LP;
  const colors = ['#c74440', '#2d70b3', '#388c46',
'#9b59b6', '#bf7834', '#169b9b'];
  const equation = (line) => expr(line.a, ['x1', 'x2']) +
' = ' + num(line.b);
  const objective = (p) =>
    expr(p.c, ['x1', 'x2']) +
    (p.constant.zero()
      ? ''
      : (p.constant.sign() > 0 ? ' + ' : ' − ') +
        num(p.constant.sign() > 0 ? p.constant :
p.constant.neg()));
  function points(line) {
    const [a, b] = line.a;
    if (a.zero() && b.zero()) return [];
    if (a.zero())
      return [
        [Z, line.b.div(b)],
        [O, line.b.div(b)],
      ];
    if (b.zero())
      return [
        [line.b.div(a), Z],
        [line.b.div(a), O],
      ];
    if (line.b.zero())
      return [
        [Z, Z],
        [b, a.neg()],
      ];
    return [
      [line.b.div(a), Z],
      [Z, line.b.div(b)],
    ];
  }
  function construction(r) {
    return `<section class="solution-section"><h2>1. Xác
định miền nghiệm D</h2>
```

```
          <p>D là giao của các nửa mặt phẳng và các đường
thẳng thỏa mãn hệ ràng buộc, kể cả điều kiện dấu của biến.</p>
          <div class="table-wrap"><table class="geometry-
table"><thead><tr><th>Đường biên</th><th>Phương
trình</th><th>Hai điểm để vẽ</th><th>Phía
nhận</th></tr></thead><tbody>${r.lines
            .map((line, i) => {
              const pair = points(line);
              return `<tr><td><span class="geo-swatch"
style="background:${line.domain ? '#888' : colors[i %
colors.length]}"></span>${esc(line.label)}</td><td
class="math">${equation(line)}</td><td
class="math">${pair.length ? pair.map(vector).join(' ; ') :
'Không có đường biên'}</td><td class="math">${expr(line.a,
['x1', 'x2'])} ${rel(line.rel)} ${num(line.b)}</td></tr>`;
            })
            .join('')}</tbody></table></div>
          ${r.p.domains.every((d) => d === 'pos') ? '<p>Vì x₁
≥ 0, x₂ ≥ 0 nên D nằm trong góc phần tư thứ nhất, kể cả hai
trục tọa độ.</p>' : ''}
          <p>${r.status === 'infeasible' ? 'Các điều kiện
không có điểm chung: D = Ø.' : r.bounded ? 'Miền nghiệm D bị
chặn.' : 'Miền nghiệm D không bị chặn. Giá trị tối ưu còn phụ
thuộc hướng của hàm mục tiêu.'}</p></section>`;
      }
      function conclusion(r) {
        if (r.status === 'infeasible')
          return '<section class="solution-section"><h2>3.
Kết luận</h2><div class="notice warning">D = Ø. Bài toán vô
nghiệm.</div></section>';
        const direction = r.p.sense === 'max' ? r.p.c :
r.p.c.map((q) => q.neg());
        const movement = r.constantObjective
          ? `Hàm mục tiêu không đổi: f(x) =
${num(r.p.constant)} trên toàn bộ D.`
          : `Vẽ đường mức f(x) = a: ${objective(r.p)} = a.
Vectơ pháp tuyến n = ${vector(r.p.c)}. Tịnh tiến song song
theo hướng ${vector(direction)} để ${r.p.sense === 'max' ?
'tăng' : 'giảm'} a.`;
        const sampleLevel =
r.p.constant.add(LP.Q.parse('5'));
        const samplePoints = points({ a: r.p.c, b:
sampleLevel.sub(r.p.constant) });
        let result;
```

```
        if (r.status === 'unbounded') {
          const slope = Geometry.dot(r.p.c, r.ray);
          result = `<div class="notice warning"><p>D chứa tia
x(t) = ${vector(r.x)} + t ${vector(r.ray)}, t ≥ 0.</p><p
class="math">f(x(t)) = ${num(r.value)} ${slope.sign() > 0 ?
'+' : '−'} ${num(slope.sign() > 0 ? slope : slope.neg())}t →
${r.p.sense === 'max' ? '+∞' : '−∞'}.</p><p>Hàm mục tiêu không
bị chặn ${r.p.sense === 'max' ? 'trên' : 'dưới'}; không có
nghiệm tối ưu hữu hạn.</p></div>`;
        } else {
          let pair = null;
          for (const i of r.activeLines)
            for (const j of r.activeLines)
              if (j > i && Geometry.intersection(r.lines[i],
r.lines[j])) {
                pair = [i, j];
                break;
              }
          const system = pair
            ? `<p>Điểm tối ưu được chọn là giao của
${esc(r.lines[pair[0]].label)} và
${esc(r.lines[pair[1]].label)}:</p><div class="geo-system
math"><div>${equation(r.lines[pair[0]])}<br>${equation(r.lines
[pair[1]])}</div><span>⇒</span><div>x₁* =
${num(r.x[0])}<br>x₂* = ${num(r.x[1])}</div></div>`
            : '<p>Chọn một điểm trên miền nghiệm tại đường
mức tối ưu:</p>';
          result = `${system}<div class="notice success"><p
class="math">x* = ${vector(r.x)}</p><p
class="math">f${r.p.sense === 'max' ? '<sub>max</sub>' :
'<sub>min</sub>'} = f(x*) =
${num(r.value)}.</p></div><p>${r.multiplicity === 'unique' ?
'Nghiệm tối ưu duy nhất.' : `Có vô số nghiệm tối ưu. Tập
nghiệm tối ưu là S* = {x ∈ D: ${objective(r.p)} =
${num(r.value)}}.`}</p>${r.alternate ? `<p class="math">Một
nghiệm tối ưu khác: ${vector(r.alternate)}.</p>` : ''}`;
        }
        return `<section class="solution-section"><h2>3.
Đường mức và nghiệm tối ưu</h2><p
class="math">${movement}</p>${samplePoints.length ? `<p
class="math">Chọn a = ${num(sampleLevel)}; đường mức đi qua
${samplePoints.map(vector).join(' và ')}.</p>` :
''}${!r.constantObjective && r.status === 'optimal' ?
`<p>Đường mức cuối cùng còn tiếp xúc với D theo hướng
```

```
${r.p.sense === 'max' ? 'tăng' : 'giảm'} là a =
${num(r.value)}.</p>` : ''}${result}</section>`;
      }
      function report(r, interactive = true) {
        const vertices = r.vertices
          .map(
            (v) =>
              `<tr${r.status === 'optimal' &&
v.value.eq(r.value) ? ' class="geo-best"' :
''}><th>${esc(v.label)}</th><td
class="math">${vector(v.x)}</td><td>${v.lines.map((i) =>
esc(r.lines[i].label)).join(' ∩ ')}</td><td
class="math">${num(v.value)}</td></tr>`
          )
          .join('');
        return `<div class="report-
cover"><h1>${esc(r.p.title)}</h1><p>Phương pháp hình học · 2
biến</p></div>${Report.problem(r.p)}${construction(r)}
          <section class="solution-section"><h2>2. Biểu diễn
miền nghiệm</h2>
            <div class="geo-figure">${interactive ? `<div
class="geo-toolbar"><div><button type="button" data-geo-
action="in" aria-label="Phóng to đồ thị">+</button><button
type="button" data-geo-action="out" aria-label="Thu nhỏ đồ
thị">−</button><button type="button" data-geo-
action="home">Vừa khung</button></div>${!r.constantObjective
&& r.status !== 'infeasible' ? '<button type="button" data-
geo-action="play" data-geo-play aria-pressed="true">Tạm
dừng</button>' : ''}</div>` : ''}
              <div class="geo-plot"><div class="geo-canvas"
data-geo-canvas tabindex="0" aria-label="Đồ thị tương tác: kéo
để di chuyển, cuộn chuột để thu phóng, bấm để xem tọa
độ">${svg(r, home(r), initialLevel(r))}</div>${interactive ?
'<span class="geo-probe-dot" data-geo-probe-dot
hidden></span><output class="geo-probe" data-geo-probe
hidden></output>' : ''}</div>
              ${interactive && !r.constantObjective ? `<div
class="geo-level"><label>Đường mức a = <output data-geo-
value></output><input type="range" min="0" max="1000"
value="500" data-geo-slider aria-label="Tịnh tiến đường mức
hàm mục tiêu"></label>${r.status === 'optimal' ? '<button
type="button" data-geo-action="optimum">Đường mức tối
ưu</button>' : ''}</div>` : ''}
              <p class="geo-animation-status" data-geo-
```

```
status></p><p class="geo-caption"><span class="geo-
key"></span> Miền nghiệm D · <span class="geo-key
outside"></span> Ngoài miền nghiệm${r.constantObjective ? ' ·
Hàm mục tiêu không đổi.' : ' · Đường tím: f(x) = a · Mũi tên:
hướng ' + (r.p.sense === 'max' ? 'tăng' : 'giảm') + '
f.'}${interactive ? ' Kéo để di chuyển, cuộn chuột để thu
phóng, bấm điểm để xem tọa độ. Đường mức dừng 5 giây tại tối
ưu rồi chạy lại.' : ''} Hình vẽ xấp xỉ; tọa độ và kết luận
dùng phân số chính xác.</p>
            </div>
            ${r.vertices.length ? `<h3>Các đỉnh của
D</h3><div class="table-wrap"><table class="geometry-
table"><thead><tr><th>Đỉnh</th><th>Tọa độ</th><th>Giao hai
đường
biên</th><th>f(x)</th></tr></thead><tbody>${vertices}</tbody><
/table></div>` : `<p>${r.status === 'infeasible' ? 'Không có
đỉnh khả thi.' : 'D không có đỉnh. Xét đường mức và hướng di
chuyển trên D để kết luận.'}</p>`}
          </section>${conclusion(r)}`;
      }
      function decimal(q) {
        const a = q.n < 0n ? -q.n : q.n;
        const ns = String(a),
          ds = String(q.d);
        return (
          (((q.n < 0n ? -1 : 1) * Number(ns.slice(0, 15))) /
Number(ds.slice(0, 15))) *
          10 ** (Math.max(0, ns.length - 15) - Math.max(0,
ds.length - 15))
        );
      }
      const initialLevel = (r) => (r.value ? decimal(r.value)
: decimal(r.p.constant) + 5);
      function home(r) {
        const candidates = [
          [0, 0],
          ...r.vertices.map((v) => v.x.map(decimal)),
          ...(r.x ? [r.x.map(decimal)] : []),
        ];
        for (const line of r.lines)
candidates.push(...points(line).map((p) => p.map(decimal)));
        const finite = candidates.filter((p) =>
p.every(Number.isFinite));
        let xmin = Math.min(...finite.map((p) => p[0])),
```

```
          xmax = Math.max(...finite.map((p) => p[0]));
        let ymin = Math.min(...finite.map((p) => p[1])),
          ymax = Math.max(...finite.map((p) => p[1]));
        const span = Math.max(xmax - xmin, ymax - ymin, 1);
        if (!Number.isFinite(span) || span > 1e100)
          return { xmin: -5, xmax: 10, ymin: -3, ymax: 7,
limited: true };
        const cx = (xmin + xmax) / 2,
          cy = (ymin + ymax) / 2;
        const sy = Math.max((xmax - xmin) / 1.6, ymax - ymin,
2) * 1.3;
        return { xmin: cx - sy * 0.8, xmax: cx + sy * 0.8,
ymin: cy - sy / 2, ymax: cy + sy / 2 };
      }
      function clip(polygon, a, b, c) {
        const out = [],
          tolerance = 1e-9 * Math.max(1, Math.abs(c));
        if (!polygon.length) return out;
        for (let i = 0; i < polygon.length; i++) {
          const p = polygon[i],
            q = polygon[(i + 1) % polygon.length];
          const fp = a * p[0] + b * p[1] - c,
            fq = a * q[0] + b * q[1] - c;
          if (fp <= tolerance) out.push(p);
          if (fp <= tolerance !== fq <= tolerance) {
            const t = fp / (fp - fq);
            out.push([p[0] + t * (q[0] - p[0]), p[1] + t *
(q[1] - p[1])]);
          }
        }
        return out;
      }
      function numeric(line) {
        const a = line.a.map(decimal),
          b = decimal(line.b),
          scale = Math.max(...a.map(Math.abs));
        return scale ? [a[0] / scale, a[1] / scale, b /
scale] : [0, 0, b];
      }
      function segment(a, b, c, box) {
        const pts = [],
          epsilon = 1e-8 * Math.max(box.xmax - box.xmin,
box.ymax - box.ymin);
        function add(x, y) {
```

```
          if (
            Number.isFinite(x) &&
            Number.isFinite(y) &&
            x >= box.xmin - epsilon &&
            x <= box.xmax + epsilon &&
            y >= box.ymin - epsilon &&
            y <= box.ymax + epsilon &&
            !pts.some((p) => Math.abs(p[0] - x) +
Math.abs(p[1] - y) < epsilon)
          )
            pts.push([x, y]);
        }
        if (b) for (const x of [box.xmin, box.xmax]) add(x,
(c - a * x) / b);
        if (a) for (const y of [box.ymin, box.ymax]) add((c -
b * y) / a, y);
        return pts.slice(0, 2);
      }
      function short(value) {
        return Number.isFinite(value) ?
Number(value.toPrecision(5)).toString() : '—';
      }
      function svg(r, box, level) {
        const W = 900,
          H = 560;
        const sx = (x) => ((x - box.xmin) / (box.xmax -
box.xmin)) * W;
        const sy = (y) => H - ((y - box.ymin) / (box.ymax -
box.ymin)) * H;
        const xy = (p) => sx(p[0]).toFixed(3) + ',' +
sy(p[1]).toFixed(3);
        const rect = [
          [box.xmin, box.ymin],
          [box.xmax, box.ymin],
          [box.xmax, box.ymax],
          [box.xmin, box.ymax],
        ];
        const finite = (row) => row.every(Number.isFinite);
        const stroke = (pair, color, width = 2, dashed =
false) =>
          pair.length === 2
            ? `<line x1="${sx(pair[0][0])}"
y1="${sy(pair[0][1])}" x2="${sx(pair[1][0])}"
y2="${sy(pair[1][1])}" stroke="${color}" stroke-
```

```
width="${width}"${dashed ? ' stroke-dasharray="7 5"' : ''}/>`
            : '';
        const texts = [],
          lines = [],
          polygons = [];
        let feasible = rect;
        for (const h of Geometry.halfplanes(r.lines)) {
          const coefficients = numeric(h);
          if (!finite(coefficients)) {
            feasible = [];
            break;
          }
          feasible = clip(feasible, ...coefficients);
        }
        if (r.status !== 'infeasible' && feasible.length)
          polygons.push(
            `<polygon points="${feasible.map(xy).join(' ')}"
fill="#c9e9d0" stroke="#2e7d43" stroke-width="3" stroke-
linejoin="round"/>`
          );
        const span = Math.max(box.xmax - box.xmin, box.ymax -
box.ymin),
          base = 10 ** Math.floor(Math.log10(span / 12));
        const step = [1, 2, 5, 10].map((v) => v *
base).find((v) => v >= span / 12) || base * 10;
        const grid = [];
        const ax = Math.min(W - 35, Math.max(10, sx(0))),
          ay = Math.min(H - 20, Math.max(18, sy(0)));
        for (
          let i = Math.ceil(box.xmin / step), end =
Math.floor(box.xmax / step);
          i <= end && i < Math.ceil(box.xmin / step) + 100;
          i++
        ) {
          const x = i * step;
          grid.push(`<line x1="${sx(x)}" y1="0" x2="${sx(x)}"
y2="${H}" stroke="#ddd"/>`);
          if (i)
            texts.push(
              `<text x="${sx(x) + 3}" y="${ay + 16}" font-
size="12" fill="#666">${esc(short(x))}</text>`
            );
        }
        for (
```

```
          let i = Math.ceil(box.ymin / step), end =
Math.floor(box.ymax / step);
          i <= end && i < Math.ceil(box.ymin / step) + 100;
          i++
        ) {
          const y = i * step;
          grid.push(`<line x1="0" y1="${sy(y)}" x2="${W}"
y2="${sy(y)}" stroke="#ddd"/>`);
          if (i)
            texts.push(
              `<text x="${ax + 5}" y="${sy(y) - 5}" font-
size="12" fill="#666">${esc(short(y))}</text>`
            );
        }
        r.lines.forEach((line, i) => {
          const coefficients = numeric(line);
          if (!finite(coefficients)) return;
          const pair = segment(...coefficients, box);
          lines.push(
            stroke(pair, line.domain ? '#888' : colors[i %
colors.length], line.domain ? 1 : 2)
          );
          if (pair.length === 2 && !line.domain) {
            const p = pair[0].map((v, j) => v * 0.22 +
pair[1][j] * 0.78);
            texts.push(
              `<text x="${sx(p[0]) + 5}" y="${sy(p[1]) - 8}"
fill="${colors[i % colors.length]}" class="geo-line-
label">${esc(line.label)}</text>`
            );
          }
        });
        lines.push(
          stroke(segment(1, 0, 0, box), '#555', 1.4),
          stroke(segment(0, 1, 0, box), '#555', 1.4)
        );
        if (!r.constantObjective && Number.isFinite(level)) {
          const coefficients = numeric({ a: r.p.c, b: Z });
          const scale = Math.max(...r.p.c.map((q) =>
Math.abs(decimal(q))));
          const c = (level - decimal(r.p.constant)) / scale;
          if (finite([...coefficients, c]))
            lines.push(stroke(segment(coefficients[0],
coefficients[1], c, box), '#753bbd', 3, true));
```

```
          const direction = r.p.c.map(decimal).map((v) => v *
(r.p.sense === 'max' ? 1 : -1));
          const norm = Math.hypot(...direction);
          if (norm > 0 && Number.isFinite(norm)) {
            const origin = [
              Math.min(W - 90, Math.max(80, sx(0))),
              Math.min(H - 80, Math.max(80, sy(0))),
            ];
            const dx = (direction[0] / norm) * 52,
              dy = (-direction[1] / norm) * 52;
            const tip = [origin[0] + dx, origin[1] + dy];
            lines.push(
              `<line x1="${origin[0]}" y1="${origin[1]}"
x2="${tip[0]}" y2="${tip[1]}" stroke="#753bbd" stroke-
width="2"/><path d="M${tip[0] - dx * 0.18 - dy * 0.1},${tip[1]
- dy * 0.18 + dx * 0.1} L${tip.join(',')} L${tip[0] - dx *
0.18 + dy * 0.1},${tip[1] - dy * 0.18 - dx * 0.1}" fill="none"
stroke="#753bbd" stroke-width="2"/>`
            );
            texts.push(
              `<text x="${tip[0] + 8}" y="${tip[1]}"
fill="#753bbd" font-size="14">${r.p.sense === 'max' ? 'n' :
'-n'}</text>`
            );
          }
        }
        function mark(x, label, best) {
          const p = x.map(decimal);
          if (!finite(p) || p[0] < box.xmin || p[0] >
box.xmax || p[1] < box.ymin || p[1] > box.ymax)
            return;
          texts.push(
            `<g><title>${esc(label + ' = (' + x.join('; ') +
')')}</title><circle cx="${sx(p[0])}" cy="${sy(p[1])}"
r="${best ? 6 : 4}" fill="${best ? '#753bbd' : '#24633a'}"
stroke="white" stroke-width="1.5"/><text x="${sx(p[0]) + 9}"
y="${sy(p[1]) - 10}" class="geo-point-label" fill="${best ?
'#753bbd' : '#24633a'}">${esc(label)}</text></g>`
          );
        }
        r.vertices.forEach((v) => {
          const chosen = r.status === 'optimal' &&
v.x.every((q, i) => q.eq(r.x[i]));
          mark(v.x, v.label + (chosen ? ' = x*' : ''),
```

```
r.status === 'optimal' && v.value.eq(r.value));
        });
        if (r.x && !r.vertices.some((v) => v.x.every((q, i)
=> q.eq(r.x[i]))))
          mark(r.x, r.status === 'optimal' ? 'x*' : 'x⁰',
r.status === 'optimal');
        const note = box.limited
          ? 'Tỉ lệ quá lớn; dùng tọa độ chính xác trong lời
giải.'
          : r.status === 'infeasible'
            ? 'D = Ø'
            : r.bounded
              ? 'D: miền màu xanh'
              : 'D: miền màu xanh, tiếp tục ngoài khung
nhìn';
        return `<svg xmlns="http://www.w3.org/2000/svg"
viewBox="0 0 ${W} ${H}" role="img" aria-label="Đồ thị miền
nghiệm và đường mức hàm mục tiêu"><title>Miền nghiệm
D</title><rect width="${W}" height="${H}"
fill="#f7dada"/>${polygons.join('')}${grid.join('')}${lines.jo
in('')}${texts.join('')}<text x="${W - 25}" y="${ay - 8}"
font-size="18">x₁</text><text x="${ax + 10}" y="22" font-
size="18">x₂</text><text x="14" y="${H - 12}" font-size="13"
fill="#444">${esc(note)}</text></svg>`;
      }
      function mount(host, r, clock = window) {
        const figure = host.querySelector('.geo-figure');
        if (!figure) return () => {};
        const $ = (s) => figure.querySelector(s);
        const canvas = $('[data-geo-canvas]'),
          slider = $('[data-geo-slider]'),
          output = $('[data-geo-value]');
        const play = $('[data-geo-play]'),
          status = $('[data-geo-status]');
        const tooltip = $('[data-geo-probe]'),
          probeDot = $('[data-geo-probe-dot]');
        let box = home(r),
          level = initialLevel(r),
          limits,
          drag = null,
          probe = null;
        let frameId = null,
          lastTime = null,
          elapsed = 0,
```

```
          targetBox = null,
          stopped = false;
        const canAnimate =
          !r.constantObjective && r.status !== 'infeasible'
&& Number.isFinite(level) && !box.limited;
        let playing = canAnimate &&
!clock.matchMedia?.('(prefers-reduced-motion:
reduce)').matches;
        function range() {
          const c = r.p.c.map(decimal),
            k = decimal(r.p.constant);
          const values = [box.xmin, box.xmax].flatMap((x) =>
            [box.ymin, box.ymax].map((y) => c[0] * x + c[1] *
y + k)
          );
          limits = [Math.min(...values, level),
Math.max(...values, level)];
          if (!limits.every(Number.isFinite) || limits[0] ===
limits[1]) limits = [0, 10];
        }
        range();
        const sign = r.p.sense === 'max' ? 1 : -1;
        const margin = Math.max((limits[1] - limits[0]) *
0.25, Math.abs(level) * 0.2, 1);
        const target = r.status === 'optimal' ?
initialLevel(r) : level + sign * margin;
        const values = r.vertices.map((v) =>
decimal(v.value)).filter(Number.isFinite);
        const candidate = values.length
          ? sign > 0
            ? Math.min(...values)
            : Math.max(...values)
          : target;
        const from = sign * (target - candidate) > 1e-9 ?
candidate : target - sign * margin;
        if (playing) level = from;
        function positionProbe() {
          if (!probe || !tooltip || !probeDot) return;
          const rect = canvas.getBoundingClientRect();
          const x = ((probe.x - box.xmin) / (box.xmax -
box.xmin)) * rect.width;
          const y = ((box.ymax - probe.y) / (box.ymax -
box.ymin)) * rect.height;
          const outside = x < 0 || x > rect.width || y < 0 ||
```

```
y > rect.height;
          tooltip.hidden = probeDot.hidden = outside;
          if (probe.html) tooltip.innerHTML = probe.html +
'<small>' + esc(probe.text) + '</small>';
          else tooltip.textContent = probe.text;
          tooltip.style.left =
            Math.max(4, Math.min(rect.width -
(tooltip.offsetWidth || 180) - 4, x + 14)) + 'px';
          tooltip.style.top =
            Math.max(
              4,
              Math.min(
                rect.height - (tooltip.offsetHeight || 35) -
4,
                y - (tooltip.offsetHeight || 35) - 10
              )
            ) + 'px';
          probeDot.style.left = x + 'px';
          probeDot.style.top = y + 'px';
        }
        function draw() {
          canvas.innerHTML = svg(r, box, level);
          if (output) output.textContent = short(level);
          if (slider)
            slider.value = Math.round(
              Math.max(0, Math.min(1000, ((level - limits[0])
/ (limits[1] - limits[0])) * 1000))
            );
          if (play) {
            play.disabled = !canAnimate;
            play.textContent = playing ? 'Tạm dừng' : 'Chạy
animation';
            play.setAttribute('aria-pressed',
String(playing));
          }
          if (status)
            status.textContent = !canAnimate
              ? ''
              : playing
                ? elapsed % 12000 >= 7000
                  ? r.status === 'optimal'
                    ? 'Đã đến mức tối ưu · Dừng 5 giây rồi
chạy lại'
                    : 'Hàm mục tiêu không bị chặn · Tiếp tục
```

```
được ngoài khung nhìn'
                  : 'Đang tịnh tiến đường mức theo hướng ' +
(sign > 0 ? 'tăng' : 'giảm') + ' f'
                : 'Đã tạm dừng · Có thể kéo thanh đường mức
hoặc bấm Chạy animation';
          positionProbe();
        }
        function requestFrame() {
          if (!stopped && frameId === null && (playing ||
targetBox))
            frameId = clock.requestAnimationFrame(tick);
        }
        function tick(time) {
          frameId = null;
          if (stopped || canvas.isConnected === false)
return;
          const dt = lastTime === null ? 0 : Math.min(100,
Math.max(0, time - lastTime));
          lastTime = time;
          if (clock.document?.visibilityState !== 'hidden') {
            if (playing) {
              elapsed = (elapsed + dt) % 12000;
              const progress = Math.min(1, elapsed / 7000);
              level = from + (target - from) * progress;
            }
            if (targetBox) {
              const blend = 1 - Math.exp(-Math.max(dt, 1) /
55);
              let difference = 0;
              for (const key of ['xmin', 'xmax', 'ymin',
'ymax']) {
                const delta = targetBox[key] - box[key];
                box[key] += delta * blend;
                difference += Math.abs(delta);
              }
              if (difference < (box.xmax - box.xmin) * 1e-5)
{
                box = targetBox;
                targetBox = null;
              }
              range();
            }
            draw();
          }
```

```
          requestFrame();
        }
        function zoom(factor, rx = 0.5, ry = 0.5) {
          const base = targetBox || box;
          const width = (base.xmax - base.xmin) * factor,
            height = (base.ymax - base.ymin) * factor;
          if (width < 1e-8 || width > 1e100 ||
!Number.isFinite(width)) return;
          const x = base.xmin + rx * (base.xmax - base.xmin),
            y = base.ymax - ry * (base.ymax - base.ymin);
          targetBox = {
            xmin: x - rx * width,
            xmax: x + (1 - rx) * width,
            ymin: y - (1 - ry) * height,
            ymax: y + ry * height,
          };
          requestFrame();
        }
        function pause() {
          playing = false;
          lastTime = null;
        }
        figure.addEventListener('click', (e) => {
          const action = e.target.closest('[data-geo-
action]')?.dataset.geoAction;
          if (!action) return;
          if (action === 'home') {
            box = home(r);
            targetBox = null;
          } else if (action === 'optimum') {
            clearLineProbe();
            pause();
            level = initialLevel(r);
          } else if (action === 'play') {
            if (!canAnimate) return;
            clearLineProbe();
            playing = !playing;
            elapsed = 0;
            lastTime = null;
            if (playing) level = from;
          } else if (action === 'in' || action === 'out')
zoom(action === 'in' ? 0.8 : 1.25);
          range();
          draw();
```

```
          requestFrame();
        });
        slider?.addEventListener('input', () => {
          clearLineProbe();
          pause();
          level = limits[0] + (Number(slider.value) / 1000) *
(limits[1] - limits[0]);
          draw();
        });
        canvas.addEventListener(
          'wheel',
          (e) => {
            e.preventDefault();
            const rect = canvas.getBoundingClientRect();
            if (!rect.width || !rect.height) return;
            const delta = e.deltaY * (e.deltaMode === 1 ? 16
: e.deltaMode === 2 ? rect.height : 1);
            zoom(
              Math.exp(Math.max(-0.4, Math.min(0.4, delta *
0.0015))),
              (e.clientX - rect.left) / rect.width,
              (e.clientY - rect.top) / rect.height
            );
          },
          { passive: false }
        );
        canvas.addEventListener('pointerdown', (e) => {
          if (e.button !== 0 || drag) return;
          const rect = canvas.getBoundingClientRect();
          targetBox = null;
          drag = {
            level,
            id: e.pointerId,
            x: e.clientX,
            y: e.clientY,
            width: rect.width,
            height: rect.height,
            moved: false,
            box: { ...box },
          };
          canvas.setPointerCapture(e.pointerId);
        });
        canvas.addEventListener('pointermove', (e) => {
          if (!drag || drag.id !== e.pointerId || !drag.width
```

```
|| !drag.height) return;
          drag.moved ||= Math.hypot(e.clientX - drag.x,
e.clientY - drag.y) > 4;
          if (!drag.moved) return;
          const dx = ((e.clientX - drag.x) / drag.width) *
(drag.box.xmax - drag.box.xmin);
          const dy = ((e.clientY - drag.y) / drag.height) *
(drag.box.ymax - drag.box.ymin);
          box = {
            xmin: drag.box.xmin - dx,
            xmax: drag.box.xmax - dx,
            ymin: drag.box.ymin + dy,
            ymax: drag.box.ymax + dy,
          };
          range();
          draw();
        });
        function lineAtPoint(x, y, rect, atLevel) {
          const candidates = r.lines.map((line) => ({
            line,
            label: 'Đường ' + line.label,
            html: equation(line),
          }));
          candidates.push(
            { line: { a: [O, Z], b: Z }, label: 'Trục x₂',
html: 'x₁ = 0' },
            { line: { a: [Z, O], b: Z }, label: 'Trục x₁',
html: 'x₂ = 0' }
          );
          if (!r.constantObjective &&
Number.isFinite(atLevel))
            candidates.push({
              line: { a: r.p.c, b:
LP.Q.parse(String(atLevel)).sub(r.p.constant) },
              label: 'Đường mức',
              html:
                objective(r.p) +
                ' = ' +
                (r.value && atLevel === decimal(r.value) ?
num(r.value) : esc(short(atLevel))),
              objective: true,
            });
          const dx = (box.xmax - box.xmin) / rect.width,
            dy = (box.ymax - box.ymin) / rect.height;
```

```
          let found = null,
            closest = 8;
          for (const item of candidates) {
            const [a, b, c] = numeric(item.line);
            const norm2 = (a * dx) ** 2 + (b * dy) ** 2;
            if (!Number.isFinite(norm2) || !norm2) continue;
            const residual = a * x + b * y - c,
              distance = Math.abs(residual) /
Math.sqrt(norm2);
            if (distance <= closest) {
              closest = distance;
              found = {
                ...item,
                x: x - (residual * a * dx * dx) / norm2,
                y: y - (residual * b * dy * dy) / norm2,
              };
            }
          }
          return found;
        }
        function clearLineProbe() {
          if (!probe?.objective) return;
          probe = null;
          if (tooltip) tooltip.hidden = true;
          if (probeDot) probeDot.hidden = true;
        }
        canvas.addEventListener('pointerup', (e) => {
          if (!drag || drag.id !== e.pointerId) return;
          if (!drag.moved) {
            const rect = canvas.getBoundingClientRect();
            const rx = (e.clientX - rect.left) / rect.width,
              ry = (e.clientY - rect.top) / rect.height;
            let x = box.xmin + rx * (box.xmax - box.xmin),
              y = box.ymax - ry * (box.ymax - box.ymin);
            const targets = [...r.vertices.map((v) => v.x),
...(r.x ? [r.x] : [])];
            const exact = targets.find(
              (p) =>
                Math.hypot(
                  ((decimal(p[0]) - x) / (box.xmax -
box.xmin)) * rect.width,
                  ((decimal(p[1]) - y) / (box.ymax -
box.ymin)) * rect.height
                ) <= 10
```

```
            );
            if (exact) {
              x = decimal(exact[0]);
              y = decimal(exact[1]);
            }
            const hit = exact ? null : lineAtPoint(x, y,
rect, drag.level);
            if (hit) {
              x = hit.x;
              y = hit.y;
              if (hit.objective) {
                pause();
                level = drag.level;
              }
            }
            probe = {
              x,
              y,
              html: hit
                ? '<b>' + esc(hit.label) + '</b><span
class="math">' + hit.html + '</span>'
                : null,
              objective: hit?.objective || false,
              text: exact
                ? '(' + exact.map(String).join('; ') + ')'
                : '≈ (' + short(x) + '; ' + short(y) + ')',
            };
            draw();
          }
          drag = null;
        });
        for (const event of ['pointercancel',
'lostpointercapture'])
          canvas.addEventListener(event, () => {
            drag = null;
          });
        canvas.addEventListener('keydown', (e) => {
          if (e.key === '+' || e.key === '=') zoom(0.8);
          else if (e.key === '-') zoom(1.25);
          else if (e.key === 'Escape') {
            probe = null;
            if (tooltip) tooltip.hidden = true;
            if (probeDot) probeDot.hidden = true;
          } else return;
```

```
          e.preventDefault();
        });
        draw();
        requestFrame();
        return () => {
          stopped = true;
          if (frameId !== null)
clock.cancelAnimationFrame(frameId);
        };
      }
      return { report, mount, points, home, svg, decimal };
    })();
```

js/geometry.js

```
    /* Hình học phẳng với phân số chính xác; độc lập với bộ
giải đơn hình. */
    function installGeometry(scope) {
      'use strict';
      const { Q, Z, O } = scope.LP;
      const dot = (a, x) =>
a[0].mul(x[0]).add(a[1].mul(x[1]));
      function boundaries(p) {
        const lines = p.a.map((a, i) => ({ a, b: p.b[i], rel:
p.rel[i], label: '(' + (i + 1) + ')' }));
        p.domains.forEach((d, j) => {
          if (d !== 'free')
            lines.push({
              a: j === 0 ? [O, Z] : [Z, O],
              b: Z,
              rel: d === 'pos' ? '>=' : '<=',
              label: 'x' + (j + 1) + ' = 0',
              domain: true,
            });
        });
        return lines;
      }
      function halfplanes(lines) {
        return lines.flatMap((line) => {
          const positive = { a: line.a, b: line.b };
          const negative = { a: line.a.map((q) => q.neg()),
b: line.b.neg() };
          return line.rel === '=' ? [positive, negative] :
[line.rel === '<=' ? positive : negative];
        });
      }
      // Giao hai đường biên bằng định thức; song song/trùng
nhau không sinh đỉnh.
      function intersection(l, r) {
        const det =
l.a[0].mul(r.a[1]).sub(l.a[1].mul(r.a[0]));
        if (det.zero()) return null;
        return [
          l.b.mul(r.a[1]).sub(l.a[1].mul(r.b)).div(det),
          l.a[0].mul(r.b).sub(l.b.mul(r.a[0])).div(det),
        ];
      }
```

```
      function interval(inequalities) {
        let lo = null,
          hi = null;
        for (const [a, b] of inequalities) {
          if (a.zero()) {
            if (b.sign() < 0) return null;
            else continue;
          }
          const bound = b.div(a);
          if (a.sign() > 0) hi = hi === null || bound.cmp(hi)
< 0 ? bound : hi;
          else lo = lo === null || bound.cmp(lo) > 0 ? bound
: lo;
        }
        return lo !== null && hi !== null && lo.cmp(hi) > 0 ?
null : { lo, hi };
      }
      function choose(range) {
        if (range.lo !== null && range.lo.sign() > 0) return
range.lo;
        if (range.hi !== null && range.hi.sign() < 0) return
range.hi;
        return Z;
      }
      function other(range, value) {
        if (range.lo === null) return value.sub(O);
        if (range.hi === null) return value.add(O);
        if (!range.lo.eq(value)) return range.lo;
        return !range.hi.eq(value) ? range.hi : null;
      }
      // Đổi tọa độ u = c.x, v = một tọa độ gốc rồi chiếu các
nửa mặt phẳng lên u.
      // Giữ cả trường hợp miền là dải, đường thẳng hoặc cả
mặt phẳng, không có đỉnh.
      function project(planes, c) {
        const first = !c[0].zero();
        const rows = planes.map(({ a, b }) =>
          first
            ? { u: a[0].div(c[0]), v:
a[1].sub(a[0].mul(c[1]).div(c[0])), b }
            : { u: a[1].div(c[1]), v: a[0], b }
        );
        const upper = rows.filter((r) => r.v.sign() > 0),
          lower = rows.filter((r) => r.v.sign() < 0);
```

```
        const constraints = rows.filter((r) =>
r.v.zero()).map((r) => [r.u, r.b]);
        for (const p of upper)
          for (const n of lower)
            constraints.push([p.u.div(p.v).sub(n.u.div(n.v)),
p.b.div(p.v).sub(n.b.div(n.v))]);
        const range = interval(constraints);
        const fiber = (u) => interval(rows.map((r) => [r.v,
r.b.sub(r.u.mul(u))]));
        const point = (u, v) => (first ?
[u.sub(c[1].mul(v)).div(c[0]), v] : [v, u.div(c[1])]);
        return { range, fiber, point };
      }
      function solve(raw) {
        const p = scope.LP.parseProblem(raw);
        if (p.n !== 2) throw new Error('Phương pháp hình học
yêu cầu đúng 2 biến.');
        const lines = boundaries(p),
          planes = halfplanes(lines);
        const constant = p.c.every((q) => q.zero()),
          axis = constant ? [O, Z] : p.c;
        const projection = project(planes, axis);
        const r = {
          p,
          method: 'geometry',
          status: 'infeasible',
          x: null,
          value: null,
          lines,
          vertices: [],
          constantObjective: constant,
        };
        if (!projection.range) return r;
        let u = choose(projection.range),
          v = choose(projection.fiber(u));
        const feasible = projection.point(u, v);
        const keySet = new Set();
        for (let i = 0; i < lines.length; i++)
          for (let j = i + 1; j < lines.length; j++) {
            const x = intersection(lines[i], lines[j]);
            if (!x || !planes.every((h) => dot(h.a,
x).cmp(h.b) <= 0)) continue;
            const key = x.map(String).join('|');
            if (!keySet.has(key)) {
```

```
              keySet.add(key);
              r.vertices.push({ x, value: scope.LP.value(p,
x), lines: [i, j] });
            }
          }
        r.vertices.sort((a, b) => a.x[0].cmp(b.x[0]) ||
a.x[1].cmp(b.x[1]));
        let letter = 0;
        r.vertices.forEach((vertex) => {
          vertex.label = vertex.x.every((q) => q.zero()) ?
'O' : 'V' + ++letter;
        });
        r.extents = [
          [O, Z],
          [Z, O],
        ].map((c) => project(planes, c).range);
        r.bounded = r.extents.every((range) => range.lo !==
null && range.hi !== null);
        const bound = p.sense === 'max' ? projection.range.hi
: projection.range.lo;
        if (!constant && bound === null) {
          r.status = 'unbounded';
          r.x = feasible;
          r.value = scope.LP.value(p, feasible);
          const recession = project(
            planes.map((h) => ({ a: h.a, b: Z })),
            p.c
          );
          const directionU = p.sense === 'max' ? O : O.neg();
          r.ray = recession.point(directionU,
choose(recession.fiber(directionU)));
          return r;
        }
        if (!constant) {
          u = bound;
          v = choose(projection.fiber(u));
        }
        r.status = 'optimal';
        r.x = projection.point(u, v);
        r.value = scope.LP.value(p, r.x);
        const nextV = other(projection.fiber(u), v);
        if (nextV !== null) r.alternate = projection.point(u,
nextV);
        else if (constant) {
```

```
          const nextU = other(projection.range, u);
          if (nextU !== null) r.alternate =
projection.point(nextU, choose(projection.fiber(nextU)));
        }
        r.multiplicity = r.alternate ? 'multiple' : 'unique';
        r.activeLines = lines
          .map((line, i) => (dot(line.a, r.x).eq(line.b) ? i
: -1))
          .filter((i) => i >= 0);
        if (!scope.LP.feasible(p, r.x)) throw new
Error('Không xác minh được nghiệm hình học.');
        return r;
      }
      // Phạm vi của chế độ hình học trên giao diện: chỉ nhận
miền nghiệm bị chặn.
      function solveBounded(raw) {
        const p = scope.LP.parseProblem(raw);
        if (p.n !== 2) throw new Error('Phương pháp hình học
yêu cầu đúng 2 biến.');
        const planes = halfplanes(boundaries(p));
        const ranges = [[O, Z], [Z, O]].map((axis) =>
project(planes, axis).range);
        // Miền rỗng được trả về để thông báo vô nghiệm,
không chạy animation.
        return solve(raw);
      }
      scope.Geometry = { solve, solveBounded, boundaries,
halfplanes, intersection, project, dot };
    }
    installGeometry(globalThis);
```

js/layout.js

```
    // Adjustable split view; independent of the problem and
solver state.
    'use strict';
    (() => {
      const workspace = document.querySelector('.workspace');
      const panel = document.querySelector('#input-panel');
      const handle = document.querySelector('#panel-
resizer');
      const desktop = window.matchMedia('(min-width:
851px)');
      let preferredWidth = null,
        drag = null;
      const bounds = () => ({
        min: 300,
        max: Math.max(300, Math.min(800,
workspace.clientWidth - 408)),
      });
      function update() {
        if (!desktop.matches) {
          stopDrag();
          return;
        }
        const { min, max } = bounds();
        if (preferredWidth !== null)
          workspace.style.setProperty(
            '--sidebar-width',
            Math.max(min, Math.min(max, preferredWidth)) +
'px'
          );
        const width =
Math.round(panel.getBoundingClientRect().width);
        handle.setAttribute('aria-valuemin', min);
        handle.setAttribute('aria-valuemax', max);
        handle.setAttribute('aria-valuenow', width);
        handle.setAttribute('aria-valuetext', width + '
pixel');
      }
      function setWidth(width) {
        const { min, max } = bounds();
        preferredWidth = Math.max(min, Math.min(max, width));
        update();
      }
```

```
      function reset() {
        preferredWidth = null;
        workspace.style.removeProperty('--sidebar-width');
        update();
      }
      function stopDrag() {
        const current = drag;
        drag = null;
        document.body.classList.remove('resizing-panels');
        if (current && handle.hasPointerCapture(current.id))
handle.releasePointerCapture(current.id);
      }
      handle.addEventListener('pointerdown', (e) => {
        if (!desktop.matches || e.button !== 0 || drag)
return;
        e.preventDefault();
        handle.focus();
        drag = { id: e.pointerId, x: e.clientX, width:
panel.getBoundingClientRect().width };
        handle.setPointerCapture(e.pointerId);
        document.body.classList.add('resizing-panels');
      });
      handle.addEventListener('pointermove', (e) => {
        if (drag && e.pointerId === drag.id)
setWidth(drag.width + e.clientX - drag.x);
      });
      for (const type of ['pointerup', 'pointercancel',
'lostpointercapture'])
        handle.addEventListener(type, (e) => {
          if (drag && e.pointerId === drag.id) stopDrag();
        });
      window.addEventListener('blur', stopDrag);
      handle.addEventListener('dblclick', reset);
      handle.addEventListener('keydown', (e) => {
        if (!desktop.matches) return;
        const width = panel.getBoundingClientRect().width,
          step = e.shiftKey ? 40 : 10;
        if (e.key === 'ArrowLeft') setWidth(width - step);
        else if (e.key === 'ArrowRight') setWidth(width +
step);
        else if (e.key === 'Home') setWidth(bounds().min);
        else if (e.key === 'End') setWidth(bounds().max);
        else if (e.key === 'Enter') reset();
        else return;
```

```
    e.preventDefault();
  });
  window.addEventListener('resize', update);
  update();
})();
```

js/math-fonts.js

```
    // Dữ liệu font nhúng để chạy ngoại tuyến và xuất HTML
độc lập.
    // Các chuỗi base64 là tài nguyên, không cần chỉnh sửa
thủ công.
    /* KaTeX 0.16.11 Computer Modern fonts; MIT license in
licenses/KaTeX-LICENSE.txt. Embedded for offline use and
standalone export. */
    window.MATH_FONT_CSS =
      "@font-face{font-family:'Computer Modern';font-
style:normal;font-weight:400;font-
display:swap;src:url(data:font/woff2base64,d09GMgABAAAAAGagAA
4AAAAA0lgAAGZHAAEAAAAAAAAAAAAAAAAAAAAAAAAAAAAABmAAhlQIWgmcDBEI
CoLXJIKILwE2AiQDiHQLhD4ABCAFiHgHjkUMgTIbracHZF4Lnrsd2PH+Ms9+JE
LYOIBYG7aPRNiETSqb6P//vKQiY6bZTNMNFRBQ/blSCatQNSrOgpeJiUJlHLKR
hjMn7i7ko8jo8gxzFR801TSw0i0Tat37bi+vKqaOBz+5uGKeanHTsl+X8UYhIV
JBUVEKJwyTI2zsIvkh6xfj98GIibTjsk28o20/ozGZ+q9xI7q5M/7Ff/5jhriY
oIYFajzorm4vUYsO8bviPnAGto38SU7e4WlO/91dcrn4SRwIEQLBQgQSpEDiEA
IkkODFrIZVqFIqUKpUxNdV1nWV/dZ/dZW/Ma1uaydtZ+1+p4XguZ93v3L1vGhi
I98Eh2UyhsYKnv/n/ts+5743Y1N/+vzFAg00DDhrpQkmmNFYJdT/vtM+O2ke2D
NseG8R0IkfAHwukHT2z95l6cqxU44dwJYDbaYdLqAlLcI30KCb/SchWCgh4IWa
UVGuR3s9E3+6v+3vusx/FFvfvH2oaLpEJvR2O98orrkqHEYgDRbjgnAYhyUA/v
qmVr/sX/cuNwgMiYXmtWMZQugA90zbq9tTwlXnZRjdsXTHI8mJAmw5wLChAcMC
8S3a1czbwX6Y8W9+/lr2DDEJ2ZNM0w1i8vX10JR0jVgN0X8xN/JK+oiptYl4qY
hXjh7Jb3D+wXkTU3+sEw6nd/d2e2drW7fQljpUVEAKjLKpHi5AhgvtPSRsszYo
+c9f1F/U6qrRmfsiRG77S4aSFxQHIqJac4m17YPaHJCyO89oyy3p6bkWwB8g8c
fO0LJqBRART934F3hBDEETBBjoCtxN10wShPTvT369Lqtbs/hbWhRUbU6UbZSY
tXWIQepnZ68dSv8v0YyPI8bKx9IRVH6V3/+Zara70AXyInUhh9en6lx0zkXpcv
bPAIuZWYDALCBhF4xL6RlckM8ElroD9xIJQMzSpZggiEqg5KcjlSLlHEKVm9ql
m9JFGVPvunVXhaJxa3v+fp/OzDn3p8oWyJq9yK2mCquyCl2BxAVKYN5797Ys/G
Qos0RRi2OV4mEsxoRJ/rZsdnm/UWqXSHQ3kq5buZPXjT2hzwh195fVd1FlySpt
0ogIxZs5XP96jLlCs/Fj97yZKeEJJxwjsl/2YxnT+mV7+9fudFZR2AkkoP8a6F
U9eQW4bEkxDwDOLj9BB+DPDgGwiN86tGcIzMPh+4CfOuD55Vnjfx2/EFkp9oYo
/qkVM5AzwwwsfeEAgL7bAIDCuFKGoIKFQ9lo2sz+crlJH5RPjZm22mGvg4543w
eu8BkPecobexbXEvy3N/pF78/KCSaaiZma2UmmdoMqCsXAJrYwWbS9/cRXdtXT
WHOd9NZu7deHOq+b+gSgGlWsylSlqs+UJYYjsOPMHQcucsjHTx1NjGEacxhgCe
vYxV6ucJWf+EXNVk/cJxwLJ8OH4WL4KTbECen79FP6M+bvuOxt9R++fQtQqDNG
3rP24VGfyv+Xi57ylV7rZ/1qVry/4qnewIpc0bexzc1m8mvnfTqjc/IduXK6e4
8BiyNPB2+knan0M3/0MNvlS/LLJ8fUY1cdujaOS9+ll4nv6nn7Zfr26z9tjM6c
fP/okZ0rlwxOacIv8932zDTMz//+Z3/7Nz/58Y8a7NmxafU2ue5aXXq42N/Py1
0ficHi/OYXzx588+CUkz5wwnHvOuSgA/bZa481Vlpu2FILLdBrsm5NaoxWKiRf
Hr1aGPPflv8yKJC/vHJyJHnhW4opUEnF6x8Za5PiP3nQvCKl8hsATWjz/wceBk
9CipCUW6Ff3/T/XbP8i/Jd/l7j18zvNJcJTsPYgC9ngJ58oXD2MzCaY+0mzpFP
```

r4ksnUvSLflM9p33BJcEyzLqABRGHRKCuWQ+JIJ48UNyQHvIjV9f0oYmEaMWNN
5oKVeXibi9ABNKxCa1xMeLDlUqjMp8edfXpnZ04sLSXcQpop0ZzBpiHx8chAPr
nIB6lHAK57QZEI1IjrdFS4cW/K7hzOLPrhZUjmdjChvBsxvNNn8FTHX5YpkHx3
d0QQjGpab4xlBYfWpZk94Kqnu6rCsofW+BOfKPaSwRGgss18RIyTOUhrFo1lqC
yJjgjOAoEGwvqc/9Tnoy6Zm9dMZkJlDwjJfMrE1G6MFrgGRzQYeiNiQPjEFsqV
3qRK4HrFOlhIbzvqchGVKwN0LJ7SJmir0wXM3pD5sYUWLryXf84+ITOff6e/DS
rZeJ40aI6y5tWhDzBi6cBWSOFq8uAw0pgjL7HUBulqSWsqqAYDlYIsrmhHmMYs
eaMoA2lXJ2hiVrUVZzITGmx1E+I4JOHhSOtecDR1I6BpKWADGgS7mO2GLWnIsp
WmsLEDgEpehTuWCxGDljaKM++qMeHmZpSAY55xKFAc3sUO2X6sZ68E689TxlaC
iossEXXv2QDNYPMDV+7kCJbFUjq8hHnPlL0lGCrVIIZ1jyXEypc8ifLQqoQZKZ
GviyHQouRvaIaimLgmSO2whfttZh5+0oc68WCjC44zdOF2Gq1N2GZCPTol3JLI
NwUkDOFlbsFyK+M/OPy/3Ws46XPl9Zc80SUjy+SRg1Tg0SSjGLs/Jx4aAFAFNH
hCSY+4QhXI+YjaUC2tgONFQxJlWlqkSJIp560xfAiFwVVSVAsgoTpsXqbvAnn3
tosoWDAmwPgXAF4QkhKQ6+MALxSElAWiIykpCVvNiA0P8lheTLdhBUhA/JEpn7
zb1Nx597tpoVf/m0HJbYupnrhOnxOmEXzGOeORYSjqIQSuJQFkZFPKoSUJOIUB
LqktfaQB2w6OyN9BI0ch1qXJochUPtmBUMdVH1u/V3kjulB2AKb1qSluXEnr8A
c8ZLfbKyCm1sRNzU2OsUPkyMwxMzUhvYJ3+Rhgh4ibx6Ckusjw2VyDLHkrveAs
43eHCtvJXlEgXb1DtlCNSY4o8RLGtkEtOobtafSYrbooAqSnvmCAXJpKFSJkEx
WRKf8o9betgJYd+NupmqXRzgxLWeTB2DwHl3tzC2N24XKwm5FlTYP8htBeocw5
HMRFDBhSgUfMLowh7GlMyFk3CIl6/k3GnHK77pSJskRC5KxGQSda88Xlkc7ci7
UgHM2Ap5plYrl3ByyOJIl1XGSAJ2iyiTtOysSTh5x9IJlZxhxJwJo1Jh0RkGb8
lnbWUSRy08RrnTO5K2U+W3LTihgqIY5SOLSb38tO6sTtLsnwD0Cl4MzjXhwRCi
UeEgJ6FCLd6pawZcAdwiLFjW49NnF+LEnNHu9RbTkA/O3wP5vhsWP+BkHbLPxf
YCBBWJyM/cZVh3mxZQW1W6wepXM/ZeHLzp1p9ekr22xIeXwbH3eH4FnPGcW65j
7Jk5bruHAit2JVangROfI5H4hcTElA09uT0qqFHJMvzBNRRSMz3klkP6X0yjOD
EpGcde1rLsfLzMesOJ9UUwdlLqjSyaZ70UuTGlDbg5Yh/3Ux1I8OpquTXsDGRr
exsFwMskp7Ha4dSjD3b/As6GoMIkFyV1ZE0pCOjoMY73qPgQGEVNhN4B0BCHTB
GyshIGuwOgI+518SMgkugZ4XcBDIxDoZhAkSgWJTKUqiStLMmRdwAsxKFShJyq
hKHuANiIwy1cpKzN1iFlXbYeW59twDbknRFeZBw4PK5MPpjxCfGhqdXTjEQ1ex
gFEcwD2CIAlvPIgnLllmu3QxuW3XGb32EXAPsA9WFrOh7d8eSOZ3e85AzXALgF
qO/Q9Hy459M9X+75zhk+ARDrjwF6yw0/4VP5ZDtTCnMdrlQ3gnjnzXQ5tlSSW1
tfnK1LDYsBEOi/BQCArjZh+CmRvlqnfmTA8Y4eDuMEkD8wDhcGgqZ7h1glzEkW
QMRxRZXFLOctNKqLfATUTSVSUvO8ZVXsWVCOQjQELJdJOFeb1kYI7/iQHWNPBF
fuZU5lWaVyESLDqoJcjQGESMxzUKTWxOmQ594RaBnpUJ9Lvv2mf7AhuOlX71TC
mFHfPyG7GokN3do2mJ9MzTs+psxOF+LjEl5r24OqafccSmnSU1R/QK+tSC57xk
IQbC1frdVXnrpbQ9+tOsM8PbSO5ia944fFjqRJNl5gdXsQsrIy2mLJU5ql5Yfm
Dye2OU9Dc22rPNvZv1/qL5OZzJt8oCc0aujZanChdPuqdnVIDSrRjsqyN25k0L
LNsG4HLb+yMprW0kQ/SIy9hN8Zb24pytMjxik/CHx/aW2GUqqX70e7sXFg/qp/
CwfV4fLTaDSZnxpGSc+1XS9eGMSb+roRCLHtRqZJpU5PyjFtizKaGVg1zfSJbu
Yz9XAUVZokUtR+N5dTVEokkiYeCYLT00q90+5IH6QwZxCdmKYruxVxVdVNECK7
esL1GdOZVrAE2hErhM5CukEv80GuMdGFJDEEU8wEk1Ytx8481SV3omNWhmF1Yx
K7i1OXodSQdIVXsIuhJX0SHsXHiR9FeijB5ND3YNyNBAlmDQAOx71NSSEVxg8x
j0HbB6+6iNzfjBLSMKc4FvqyFIy4F/WgsaxnUKF8ilCCw4KDiDOGFEsV344soB
+aTEQAau1iHOI8LKNcX9dRAQdZcSNmZmsvm+VCbwPcoZUmXU0YH90ahGhQwNwE

2EwaddAthS2V92zEai3jIFjDMKZdhkW0Ob01jjGDiyeQGzGaYfoiYKLsd5NOxz
Qy6gLmEAFwJzLi5VyjaSEVe4DhROn/kS6zYHHp4h8xdcjyQh0zWl1t2OU1Yl45
ArTLjxm90aEWbzMB4p2O0eVLYVwA5oyIsyUbqepKg2umfVG9VsUhHNSmxIUueF
5FzE6ajiKMHmVrxG+VqiarLTzolpFiOoBapu4EzBOI/IQpAzgH8M20IZv4DAZv
ODXPec6rqjt8OrN5CIdYIW6Dkop1CQQHzST9o0YTtk+sL+cD1l+WJOvp0p2E/J
MOHMdtPSKiogQHQIQxarMC20y8mzmVa5sFYzFCHd5CwwpxkB2ZfJ4UMDdyt3yl
QImbFCd1mRFnmaI36ImsS/FsPurFTipnM0lO5yCOlUrCbaUWQnA/3c0RxuKtyJ
EORvoMRf4Ywv8TkvisWjVHZLd1FVP34fN+QT0VrVoSWUgxX1NeI17ZWKcmAQ4M
zh0zzKCm5hEmcqSr4zS1cUZFsDTLrFJiJnEvytBu2FqRhtCUqzO/lPcF87OAai
KV28OgpJK1EuBgjiKUGMSxETn/RLJaeB0NQZswyq4H44TNO5hOTZh8pbYEjV3i
gzugmAyN77ClP5dXRFbcAYYkmM3EpAYd7uB3REyqqX0gmGCguabXUKzKmGAxMo
yO10XPruRd2KYDOFTFpRM1bUxJkhyG7QJJ5DgnTn0hFAszbCnXiN8BOqTMEYnP
ykPvqDd6o1I02ReJt/TsQRTF8EzNHI6DYYJYqroUxQphsNhaGzN6gBQXjxKvjd
aBKtbGmXIxMTE22GdxGZwo0FSWJOpEOVKlbQVLpWqoj2qkxBMcRaiC2y0H4O9Y
nA+KQSYtENllukHCMxGhCYVvNkKeMayKtNBpnydT7C8M9iG7lQ3ACNDhELeMnf
sDhQpBA2VYGjlWxl4ELUEAfWjk8WqHfMa1cGEXkshKEu+e4xR9L4vHrxI/vHsd
oCQ8Fg+7nm2VLE2sx61LgnjSmSS1kYO4GGD5HH5IvYeRmXpXL5w2tH8dXy2J9b
Z8msZzdnph3XciohVykJmn4lthUP+OlqeN1tz7tFmcpS9whf4n/nu6XTMGTX4X
MY/K1lNTGSDmafyjDA/W1XYqG3lQHMNsfhVlS0j7qb30k8iyjmDWQ5Zb1QuYYc
rFWBkOQjNPzrAclBWJiwFAF/oWZmt6/FTYWuxyp9h6T4kkEu1gEDlUqYri/BCl
i9zw6I3+E1wda6v1JpmFrT3Wsv7+gG2aMvqGtgjgkC7uAL2gXeoijB5Muriof8
uoX/8obGysD7zDLrfs1hURSxo8MugUMZkMi9MlHsVOo5fal9qTwh0lKGTZZsJe
cdvLoqBBIs3JUTXAk6ZiWbKMGUYeElJLFX/GHo/xWeYbO8Ifj0qWWuTCmZxgyh
TR5iKrTEVBoY/YGzL2XIYn1aLbkysxYEOhMsKGnhYi0/u3bQ+KllewRt9Pw7CJ
ru/hKCrbS1XLgt7eFob1oOKqWzsZiIRqYvZQ5c5eOgKd/PrKlcs27QsneL39B/
sPr9pq5aF9jVtrNBPSBY9zxFqumPh+4l253SApUALnbplUWNXWRcvAfBqlDDRy
wEGt87abpiO64HfXU0e5Q3dZY/2GGBrf/PKyuWjZ+JPx73REbsNu2+TAE2xF/4
/3QcTxkdsQE5Nxbi+FIGH0YkJUTpdiZ6xLwulcmUyCxrAkDBfpYNeq7vd0yhKo
cuvfOyUAOI4Zmi0PQcuo2ixw2MtKB7uslTYfRiyjt3fQOMSs5vtIhegNcdA2MQ
v0fcse483xKeCkmIC0bNLb4uP+RrZdhfP5JHExQLqY+Vv0KbkqYTpQt3lMUT/c
ewTHfkNJit+jdBoGWse1hLrU6LdPvtVzwZh7rait5bPv6G0cWdhFt2U6ajnin/
JMDVVPIksJpLBKdSKmHa5VU9kDfb35mrUJa7vrceUcCfCJHvs5h2hm2bhydjFM
+GT9HA5cr6NwuYLW2nmeI20dWwKzKiovTCXoYnPEr5Uq3lLcgqwGgllRL5uBcl
HHt4iHenUmq5Q2SgJzajkqkwtUIzWGqLwC0k08UgH42kabCFG21KKxPyZhVbkk
bTa+NaRxVY1SDFfndCJI8KCD4wuPha2CJqNEsMSEmmcczD0jAa9TvwqgOjUb1F
lOo+AuNeYOHWDrabT8tSwnNEhSYsz6QFpWVNPJENU4KBCSwMgGSgoFYwRnPxBp
mQt6UDI2shtmRb3R1BVMDsj60JJes7+/C3kTgK6ILHa+ZrQqA2l2Y0wiLx++5l
aQ0kGlINC/m6ODUPcQ1C3MyUIH/dpIT6rbQwnSS37FpDar0OsUVCg6an3rj7F3
owSDPQmFybsWqm+acXiT24iovXcx1HmvWt6VMHBoa8ZE668BPJygaRH9huUC+u
xA+/P0PdTfhyBxtfCfNWGgpkzR3jdjeHqK7wc4Yz5iELNziyftuLk7pMtdzbMf
sJRYEv3rcMvkYfMEYE31S1VBx0I7JI5dKDz51D4202QU0g1EylkPhoKXLABN/f
1AmhpMMbx4owneY5LAPjb+CyKikfg1odqeoLb30a+aHBVqwPQvfI4UqEG4ZY/Q
UEi/vxf+CBGlFdMVyArHAUOekslyGhzBPOZTOwBULkVJCBjZaJJKV/CJ1zDRi2
gYk/aBH9UmM+G7kKyBJLFvsoqCZbdmW7cQk0h7mTbZ5OH8DxeLEsxRhsJaMUp8

LPNPAkrmPwL9jNnlUowOkeoN4pFBgOVIgieq0khH1yF8gMKDtrGODq1sgxVnxU
5ZVNPBiulHYQYQe1RFi2Xtm0MaVrWQ4S3i12rtBxE9IUY1fxR+hlZNtLsff8Wp
n4AdKbYUOvC+8QMVUiANIh4lmpYu27PuRL+LvHcUp06zUou0fMQTSIVIqG0T02
XJiHg3IxPTBwILtdOtggRVtjaiaUwOkHK6YTwPOTj42yy2ygBQySJ2H1C77ofe
cIm3OOgk8eN+lvTNYB7xkPbkHLb6bAuiEldjQcCxcs3PAPMQyUYBgTqptv1jxh
wSDDzchDlsOBAZqxFjqYFyji7UYAcVdYBMYM8EzuOyjdo+E9IFLBiY7v7gk8fN
l9NSRMk2TwiP4uYxiikEUR1T++pxI16zq+wKjsq6m82Ke4lM2bZntB3c0Vftsg
9//VXJjZNriaJ4Tz3TM1wF4mr8UDaS7GhmQ+xPZflyYz1jaadPN12lztCB9qHb
rm5SUzItZvFEK2KNg6lsCXRj18w3VD0nGKT1pOseXg2mrc1nw3RLyO55nlRs4i
OPA5UJCw7iUf/8Lk9xMj7a9EtfuARZairBoBW6l8Xfyi5iO4QqIdUbsfm4N/Zq
2EEEgxgkdBhYftPwnG0xeX4XgWM7ewm+iWkeyYOxy6Hr/rkCc4RisS/keRbJYu
PlG0jLVRGGk/CVsNA/SOCaQqbNhG9tiAnVzDpkr2AjxZWV3MrwlK7TmztT7x2p
HPXaslnxIAJTEMRDJBwpSXQ3LFXsy4YypTlloItlHqJCsYRUVd9WET+H0A9ODU
Fe3xGSStFNfPGsZxJhV2btTwnRtHrRbe7f0gtp0Q+93NGEjI7L8rv97nPC8CiJ
IAmJ1kg29qMhKx72wl4VV2xPV/obOwZ+buBWd7EoKbyTW+gteR64VM8odek6CY
gK00SxTT7ZSQmjePqyg99YKATFJNGC1cjgSRWDZVmeC6yAcRMgNFEVf1XVqnLe
7kdWKhvKRrhlhgXPICOz+Uho82CZVqWCH0dUwAIDj2RfdzCzMKyY62d5JUVUOV
KRLmEPk982lz2EFSAmV6xox3xG7RTPtugqfMOjg5g5SO9l4zbxTIgtZnlIp0xp
ED3ALJG3aUm1UMvhvTJr4bxLEa/QHG7ilbZpoujQLQ5zSB8zalgWruzEsV+/7E
oh1QrnMThmCd4esVVgWkXEQgY4hE5qaAvBcrPTpMQkPeuF3Oe5K8dLQ1f6uY0V
rxhKEmgkGPakSzxEM8SScrUovQMI+WBkkmCOmmxvESE0zYi9GUTAwdot169Wj9
NIwuARMfhDkZyiPGH2zKCxsYO40xoiatYhqG2L+lCLyfVp8QGmIzEf9zuNI/hE
XeHtET/3q8lD/SwEZyi0SEHNH42uEa+jvUfIXhtyW83fDAgD5psVLI7SET+pMN
GqFShUHnwNn4oPmZO8xa7y012bbxYdYU+X4je+TVeBgdkYbTBNd0tOWQ+JDqVW
qpbIWAm/xiTYr9tey97yJ7/MQMSoa+wxJ/KN++HArub+IZxAX8rda+5d5e2oIG
0h2EGNgGRwJADXxJo99qqiGIf0Avq4XaFcDcPqIhgFg/xwajBMUzp5nnkWi8Zg
k24gJemkr6Pl7KDmoRZceorP4No3WjCIJDYIxk2iRXsp88eWXe26uPcPuuW9j3
SzHnG31kUtkVEoBuLCDAeaE4Rarx7mOU4QxXL6I8Ds9Ua1ZoIzLJsMb8mRItKW
CSFmUxvBoxXioxt1mmgmoErXYjAIRKdNPaXrDcMOWU0tOIZcYvupP+AEdmymE9
QzqYGYFkpfIDCHNQd2P5gp5uKVHHO1vGQVEXwp3CWSpddRWTJq4G0CRei5+Nf0
JS4m68yMl6vfRLupPWuIYwEhtQ/sQuR9toCnhkcz5BzexwFTNpHhrvsLijJ8Hv
AxF6EG4hiHvbj0aXhOScMltkpdEL+iVrElRjVdlYSinMCGJKgqL1NNPtiuUpfa
1ujpe1E5ygXYoPZM3ZKD+mBazKxsKIAcF48i8Uo0oN5hyGoI3Zgqp5MrCNOn1R
glSczK8OevY14vKiF2eY/4yBFgnpgM3KNh/yITa8FZ7gRiaIk3+jb55FqO9EuZ
49SH3KsW0nE1ekZXtwUErisaCblyMXT05ElmoCgAFN9MjN+QET2CXAM8WeFlp8
yj9pJBjpPYYSP6n7CuWmI4CQt/DFCnE5FfO1JeeJk05+4MBqySWw49Aj4TOtvm
E4QW8AOpyZPSiCyGJdRlid70TGKDraQ577RZJMU3Yg3S0QEQxKK9vZkP/jR18K
MngtiHaUmHvAxiwnOpS9lb8cgF/AtdNpICrOXwd0pWIylNNbmj8bRc2kIHwgPz
SnbjPyIsJ3QzsasWGntu6uyx5fJsdf6fJRZVbt/ilIN4buTWukK3sGB3FErOBu
JhqGGxZXtvpfQ2P3L1cuVI4ZxQ7VFk9xDyWUfh0Ewm5BxUp9lp6oxxFvAaZtWB
DFCmnZAPTlHJCRU+EuY0V+76I5CFPoVvo/YVE7pGh9/U2vQffJitMG7XWVCpGz
TqL74Y2Zo+CQxesjTp0dImANeydpCMzfA7wlJxcRCfVQuZPjQKmcb0iFSz/smm
6BR8WtpDTzBwqqiVPotvAilzxhfafadqFWLjjKMuqtd3LasRZRMAv1Qchdihs3
cVTckLniZVCASk00oZNLrQAwzz/i0AgUklfRpA72NC5rriaY0D5D2Pr1oI6GEl

7gxZODlsbLGM5DsEIr44z0Jdfz66Strup32o04u5pRN+JsNuQXLKo3RFVoh3Yx
HRuFyCnT/vNCJLZipbIgCBqFQNbFgsvJb2WMyssEyA3ngrjQyc8lVmrks0wJBz
UfYEbMziTkUHTUcNIjCLpGLmeio3CLNidEOGJEDZDGxUFSWP0rNpEGOXFl9+Op
P+AdyPJWJ6vWQG8oX9veM3loA/jEL6EuEE0C+i37YBuTSLOyDo19PVKuL8KcLw
QB+pKXXpe7HQbwA2hrWY9UfEXT3WLDCIFGAL9vO5B0GnZBfVjLgVeNNo0JIVnd
hrKoVQA4aGGSgVGWjfVC6ODkQc8wPGrlYcHH0ZoGws0zRrWlBTPwo7Pr7ofcPT
dp+wtrodPXzYy8L1Y3oAxIVzeM9LUfNwjE6eXDr0Gt9DDFxw28X3kunF7pa1g6
GVa/4obLpHrPtCUKgB4+RGJv43vdQIpvhAeNcH5UrmRr5XRQ5KxYZs8iFQXKiT
CsVLiHd8K/GRbOi9uAdCpqgnM7dZSBUtQEED4U1Xs8Wpog2pJZtoot1sgyEdiC
KaAeq57dw7hoviYRUjZJFRbdsmG5HDVlkNlWCkc78NIhXMMmRyw+gOJAqZwULe
tbQ3bvBNcFESt6trW7200IlnlWcUsmpV3n+zpQAwo5Fu6VWzXE5IJZI4o70+SA
Sp+5s6Um2+cUR8A6MUAcS0imQXIF+yGkwNYhWlM4eYN0uAkkI4V/0plKbT43Oa
QpaZ8xpi/VuMS82rCR89w+kiM3L9al3Priyb+zMkRtMPJoszVtBNOqmX5aEv5J
fTLU+AsMNEgEWF474/AhynwjK6yY+obsmu1lAmxxCcfcB3Dww1bYB/VkUcxrTj
jOT/DeNUBVLxuYxneUZzHOHsAylZlpLG6dYDJHO5ZC6dE0dTBWMok0H/24egp8
TMqAOZDN0a61I8V0r3NTcpTg96uyREgJcURe/Z3OA9mYcgxkpe9NSTPmKYpmGm
zENKHEVlILFsKtstssm9ny3voZndFIz0bKyRarBkaBC+8+0grxvMdkuYTcjvGb
rsiwwUJL8v3zMWAHGuBqAezBy77nciMxhQJ9lGy7O1v9pjXI+0WIz4QDHdJYNn
8eY028WKfEqvYF5RhnnImAClp71vaMm/JCQNS6SLOaG4nnwRFeIcXSjk55V/i1
Riiw5tpx8FdWAKSiFMpkgdHQHbcjNiqpRP4eWKVkfRcdeNn/N3wXLSyYWpRbYh
AI9jzzq+pyfKbiUm7vpgLwD0O89FYpKz5LYYnWZKya+G/xsplkDXD5UR4LXGpF
m9P2lRH+LgpWiAXI11aFMBx3rMcA4Y3kOj9Gyj6TLxzTxDu4exHiWwnPzJ+pAx
Y0YPt4WkOLvn1IMbvHYNL9qd4F9wZ5z52rn9eaxu9qLRbXaoUWSsfiNkewdvdD
Ld4hIwsv/HK3ckZdenmSsAhEY+ACtkZcuD1h+C6mU8KSo89kgGzao127otwycU
ydWbOqHdYcZtWwqq6iNdUHi49aHYCALZ2ULOsU/NZ+bpZl8B35puZaxjxYtuH8
thXDHMadVSEi9A/ziF67PMbMsEy7DGaZgbGhMLY+1cMk+JMoSDdb7Z6ag+vRei
Bi03V7pJK/BrYVSHYUr0mK0+A2FIispQ+bB16tOphnm9xTzdDKbY1lYHu1XGVL
NIeEt3MovHUyps5Bn02tRq1cH9gXLOz65X7E2GktnxQ3yamSt3ZDn/4hTbM6S2
Xj8p0xur1uqYWBhp53bH9I6Ty9HuPCwE+7iIuhN8ugj/nf96WxNkDhB6CVbwYs
CSxuslIGdIEy0BrQcNaXkluoQc0VIJTKuqGfDKic+mM3jhraQ42bOxjjoaH0K+
Jl9k+VxO78YZRc8ztyyZzegBNmpAhZZUKD1Z+kKrXiM8bWjkvrHLn4eHQPI9RA
tZg/51dy64pXfViA9tuAqvA/tjcwO13ZtyqyOSH7ZrL/lVsTCBrp1wUmkIBGSN
EEPEeqwd8qZJAGy6+xYHDa/JM8ZI5YC1EFzHIRQyEkp0G3ATfe3X79ETWd14JD
Ab6rizrNnKVgyj8QnIKKPfgCbrvFb5WjcK0aowelLDCmcldY6sB0qNLnRXor8W
wf1a7LAbUSYSfE26AGW1tKMCkzjUc9c0NVKPDlVZjjpf9DNMabvN4Jgz+7rreg
lxu79ihSsUdoJitS1d4rFnwC3jaBsteU8gT/x9rThq7K0atxyF7OWH7LQ2YUuM
iC/UU9UbfiuJyEOkDm+9NDTiOSYqF1fiRB72t5K5W3fk61uML/YIBjnWdsa+ek
ObiwlAWH/Hsp/cNsLLyIfuP8mEYhXnfxY+Isu5bVP9AVGTPWCDsiVhf0lq4c5C
DTN9qnP69NTpOzyup7AP9/cnIuJeavdqWOXu9r9jqLgL70SfbZdcMAoGd2pHLE
b16FbxS0ncB7dr2g6mHBYRlGddCkM8s3lXDTWpfEwbD8hSNJOvRWOjdSSRpDEb
ZMuWkwrmn1Tyo8biEl4A+EUiNs5yoOWiklk6h1UbvFNP+jkTqRloCkvfC/eJ2a
rAE7g7jCbTWXyxu3w3pAC0QSwvexAEwyXJRMExvBaKEXoF2nWVRoAR9XDScNRq
3fGjDLxuDXwADucge0PxtMNYTJJuAOcf0z1YnMn97HD689hGMDwAadxB2Zjofd
5vBNF8rYKept1qqhDPEYxC62VlnEMHAZy30lnKzrlC3MlWQmCFxegYgg+nNo/p

flqE7YqLGHAEUBLRAE5IR4yZ89rC1suSlXKqcAKGtei2IC6QZLImrp5rf2TKDm
pP/IlvNa1l3z11Gk+sijuGd1jtM/DgGdoXloS+O0qTuI0QlNxI8tg/2DuHiBNC
Af9e9bZ0/GSK77ozbP2A+4neq/tfcLb7T55dUKt4GGFDv35wbxSMaIc+CsCJAf
uoVHIWt3Ct+KRTkDDSMO+hcSzreIInjUKfgEHFvNPDZ6Rr/C7FzquBAYf+6RGE
TduTlkLx3qd1+VGxMibMbTdAvKbOOJOXw8m+bTLGaw/0ykU+rXTRh9AYsbn6F3
iju86xT3zhMBQPhwADU9Jfo4kR4iKPshoDmTTW8jtuk0z21k+AAwjyqp0cuHIo
rgh12/gE+Hl8Lrdoye2xYndK/1W/704t5AjxskXDJGx247xseLf5TiiP7pw4ZM
VT2Ty2SZJn1TzbZpXPpfzhKwSD7xvuEjhwVd4XiAXizj0zLxVk5xaG4TQxmuOe
RFwx08kn3Lm+spkXPpX8erJJK21ULmR/E+qUjgvn0JZ7D1BJdt2crcWMHpM0uz
1btOHjUEF7r2yl1UwjyyWOBHyYLQDkUBQRqp59VPBwiAg58F+JCl8zdqzwnoGM
NTsuk1jSSXKvZ+0Wd0FpzhQYBz+ImKsV1PlpUjm6OldgcrE4+gg65q0WdRhOHp
l8fUTBGQ4jWUiqP7m8VITY/zfvbeMd9n4lguQer62dCgcceTYJX1TR8gYmeWvv
23Ic7v2epPe0Lvv7wG+TSdFgHV8f3UV6giVXJKMH08nu/6LduMWcC8I1lIccpj
Z4w4nGo5TpasfbeAqqCuX9CFYsqu8EGHBvztu3ZBDHOSfg/8dM/pMgRq5HYUqX
MCcIvX6TQ5x0irC5Nz0G/8XUBlD4DdtZrYrgmFpTgNd+WGbGQO2khmqD3i5rZO
7ZyCWX5bJJoKrhYzlxGFANyotgS/pshl2Mtt/+Cfw772O0LQxx7+qcVT64PCTc
jUkbXkI+/eFT65+kP1uwzJKHLTh/b1a1XygAe7JQCG4PRoWNQA7jQSWVdB/H+q
o1FCvooEXmIDozehTZLUVZGsS4HNrEsQr+ix4dwhCjYlMHuIBEVobGR9u2Cd+u
Y6LjNpVKtXCkE3VZlcMTVQFdCnf71AhRIXS8rJQsGtXaO/iGtlbeqmYiqmyMlG
MlYJMLC1Lc0IyDNg6l1zF7mNMiUbomgQPcsjISU9GiCyAM61Wv5tq8idc+mNnS
R+xtCAN8QocPOk0mnrvON2Mx/I84pK7EVOPWW20IivyEXtUce87drTzqs0wz+F
cCZgD8xPcFa8dQ/ICHLEdx1YLm8RsZfItllqXFxfF2SVpKCh+A561ve9BnylYU
vMYeqQmXgc4bFPqzAL4zX4BzdEO23+Ab9hPCbQom4UpPOc6V1Ih/Kd6afdjwqc
yo8hw1rCYi2X8x86G2Q7oFATdAOvkvrUv5sLpfcbOruS0VhAyBnw7apbjBWG2g
lWt8lLYQkzvdSMn+XRr/FjXoN2maqyiT4ltkzFn95p0nTXR70QfP2+BJOsemrH
V+LDmj8lQXi2/RHHbt4cdOjH5SztMj53MOFlyl1jdRV05UH3302URPjfqaPesW
oehJtmVp7nLG3L7lajVfK6+0ECKQv5eLGJtBl0Wz4jhCpvsN3NYPZLT9H4Eo5V
jgc6zkQ5byX2eemFXwa3gFbpOjmB9oKfrxavT/egbswTe/Za5emUaYL3xKqcQq
YsScgcUEKYZjzmQJZv8bFnXoKhXLEPLCk7CcqTQ1etpqQ8o4EIuSqmRSWBbV9D
Yx83jqzZfXmoaHWSq4I7/FbtSuT0nlkATq19KgD7dXtIyFXmWwC6MR6OhTLmdl
UXUNlSfwQT1dRavY3Ke/iTiuMbiFUbBMh6rj6AiX/+rMTavurldf0KHD8evfNN
i2U6KHYhS5cveiXqiHbLMVpVRscVp8Vmbl7xWlsE6nOxfgXD+lJWjGy5T4pVj0
QKyHwx2lLP2fT4aGFybVpHhy2+buYyFJOh06mih/kUTS7Ffi7lrzJsG06IQvtD
G6DYoADqf39iPQ3LmnRuXUGp2srL7z8WWtHnOkptA8vTB3QusResxLd1aI40+v
reKl5hIeQq2wIswPmQoTvcfEwA5gKAl/6Wnb1LEHas102CIURXheskkopjRnJ3
FsJkJAo1Qv6+pv/1aYO771CGpqbi8t70Gc4Yi5zWutDs/uwgC7+E9BfR33QkV0
Y1WxaTCF34/Fek1yeJcr52Z0bfOs271W58QkjKiX0nVMsr3oAqrj59NWgOnflX
umfd4KCLzpk7joHjWmZXoLvNYi2Dfqg70F2WW+tkpbwqp2f+HmTgVDXaXLE7mz
phSVrIYxhKdYdOf/2RZfgvV+D/yysP0/L4bBzR7WxECRjDK59Yuk7B2ZqO9nJs
ltsh27LN3U8MbEja01TXIVlxNCaPvX7+hYv2M/mvFcT6dzBHadFl6fuCmTw2nA
wIb1W3veL7ANAEMb1vLdfN5ebbd5bpZqDS6q+GSXoaMIcFexDFyens1dVfNiRL
HrkwoRvmbAVqYd2cujaq5DqHdqZil4/KTb/69bhFqey0biV4NVG5nc8XFxzyTP
4uLGycbFxaP5KiMii04yUJHcsnfv05C3atqBB/QKDmS/zJOhrKO5rFx0oUwtEb
2bfPF/pm0CmRFd/XUut+M0ShceO51bzsJOYSzhxQ/RmhyLvy63bl9sVNHwhrjA

yfKV8d3JrXGBDaMlU0DW7eDDEr4N8qPAJqoVIq6/Q+/Exb4T+htxCWtFwIZCfr
4N2wovWQRzdLJ0uONPOlx0t4/SUcVwl/QmKWiKB/EvjUqZDh13MPw3Wc7YZl9J
RxXHS/DTVSeDOVl/78kDm1bqi9PXIlok7H2kmxKQUFHV2ClP7RU+u7MsK91Ema
NguvRriRRGZ34U9jgsrdy3m5/92qbgxBIZOVUNNiveea64KiM5OTTQ5bI3WeQr
U+2ycFOYeDhYV+W2vAecgpq33x4sU5JcTggO//h4YkmmTZMvM1YY1qrVCr1hm0
gC0I/udU+5x9RFbl3RaYGSjYhu9dO1z3CCIkM3q3RVpliucxKI0B25V/CFbyzA
2j+RAKk4UsiI+unzOM+PUZUtO5ZFF8Q4zPVsQkgdkIMAOxUlYKFV6vRYw7rpGM
n1106PnDd8cPEoKmjHgxMLK2fOGZ19NP7j8Znhb9GeslCX+0eddQEyfaFbpwpl
lHaFRZJixynqoPc3CfWkkgi+N29s3jZHQig73Z1plt7S35Amws+OJWiawbjyI5
v+UJp2fdNrwfG0Xsx8DeK389thgYDGuu9F5vTGOKq+KcogTX7mQCpw8zhMpv4Z
KTkXwU/jzc8fyM4uezD60DxDQ3E6DiFa9EABP9mVFv93VdBscTa4T9BJkZhny0
vCGyB8IuHNKA9M6iqfllY+xyQW8/L8KcUz/ns0ftrkWlmKab4iPtW5cxWYGapg
aDJCvrRYu5LFjIhXTPbqP6PN6zn+XXov62CNMA/WQ5/p21RU3V9QyxDDo3V4/R
bnRYkjOV2eGzJ5UAMh7PQTicG4koWLMmpj6rPM2b70jhTxjcbXw2T0ftdNr9Pj
4kVHvDvpoKqg0zcOG0oJ9l7E21n3P4+DVDIss3WGm7j+mkE0kkIO/nczXR0Jqf
Th2HESXcLTLFjiWOpsZLF7RhE8xhIUawKBaMXgjnzxv03oOWPRZjzlsj/NBFmv
jvFfTQeNqmga9+mMEawY8BahjOGKpY4lhQWNhHIwcgAlSLomOezl/xMnJA9Yv5
cko3Mg3XNxztG2SzBOp82LKXguz24bbacqPdHhYQYFG4+CEtg/pZbfJjdHcRN3
6FPzVAr9UKw4c/dZ743sbpUecwsdv6ldwwAB4PTYWOWN3blj8wsciTkluJ8qMx
dpJFJcj3F7lvq6SP7q8vm1qtyk6vL4M5mjzvg01atbTLdERdOnhPgmZebSSLQ+
Y12wNVc2JX5mvkOZp2kaZtcylcuPXJnJ7iB8NX+z1giTt2znsxRfpLQ5ipVUEv
toJqP8ZjkDe0vWlR9l6Pge2Ay3JDcbQQzO1KeAs2lit8a0Ty/XMZ90zH7C0nHd
JCz2iN3fp6ejdHVVrYiuW1MRXlKTblhlXA01NWP949ap+5S6a1fB+TJqWf2pcF
MgaOMvdzEa82/QSbH4el2xk3n+EzCiU/ap1027mJTsWANJa6NqZfC5aPWMGcXF
YFhRFxhncEoFaRmFPzSCxgULIYoMVhvihO8Yc1krQIhPKx+zICi8EqVZNUG4dg
8xDVMpmXjvur0TRK8p+DAiGc7hBG+M7DRBfoQRkoRUIRYWyJ/cFR3Z6nIVFOzp
1bh+6HJ4cse0st05FCOEztZ6uPtBR7BOMHcXdxylo7jjF28dQ5y/Cq7p6sKyYD
GQA5Gew5TIYMlwZ6nNS43Ktx5tip8+8Qw2qx3i5WYUjdvN7aZpaTDBHm13BmjL
0hs7+2Khqb1mK/jmRKpUoJVoBdLUE99YgXn+AISogjpTxZ3V1gTuxwtTIUSLwN
E0EslVGUA7XSfxUWcFPvyunR9cWaOPD6oQaPas5GXFyE5CiPMqJHYixcuS+8Cs
ya8Ob9WE6TjLF81SsWG+dD+Hdw2yrCz2zz/DmI7HihpDtTSHENrn1eBG0l9v05
orNM7joY3efI9vQpI7djjWmPxnx68ZMh06a22x4X9h1bfP0QZ6p0a8qqs2tDmq
BN/18SMGwED2kUZzfsV+ZetEMduPekixiPQc3R4j0rGLDUXiHxrQssNhRrnktk
NBvFMs4fVfbcuBIo1sn0qaomOrZBijXEVmhus4RctYHSKAw5gOZ8fOgH5gbJ4Q
8KCzkTJP5jQaK9Kk/Q8gkcSMQxvB+uHFw0soUix6H66FaDoW3IUdZmD+wxCq43
mRY84jB+Avcso2fypdJtRh8KNHAj1zqp2tw3QyVPNu/uSSfsf9tTe+qqlWT906
PFXVM4zsW0lydGGarYdfTQZiCYyogm7Pqq2WfFYOXauGBQI6KQYknZBaa85Hob
3N6LyjgUKPC+5FFN51Gtvaad3s1ze/frXI8721FPxdmzlFkfJ9KwlVuHMA91gO
QuiYF5pbL2A6woG80KBbhaNoy7PCdLdV+P4/g9OPup1qO8zeC0Y2NF9gosFBj4
IYrdLE46tCS/M8NAB39sqB3g2uufi/IVlDG752K9mREzcQaPu6xptZg/UR48o9
/nu2AK+b+Fc6O1pry3W2M715af9rDmDZIr+Dm20r3ZOWJ1iLpeDH2ulztT76pH
KOXSwSij+oKXoebATVUyaT93WiVL6qfZSM00dCd6LVixd78v12utmWm3QBmyMc
Ov+/NUqMO8EewvBKrSe4E13Fy5Zdhq55zidGi5QcJr0R8iTX7gtQsYMnv/iRP7
xhdrnGpiFa7XsrT/S6gtuXJhenBMs5tVxbFVdoK910qT3USYNhODqYLTxECTcI

lfv2Iqzt2awBvLK8exKoFMNXVdVJ3CIaOzVYvBy34riqyxOD2xBqGtJgoxu4dO
5+Uwy7mcvpYUXFDb2WraSL2mRgWnBo/pphG7DPAf2ejM6cVTyfW+5vKNGU8/3Q
uWj13LnpeR7OfrBPq8DKnoQY5V2NNTHXntrjE9zxq6LAirxGx6jM0SPd2kLoe1
WB+91k9RH8fH8RItkpCnxGsbYwOtluT1FJflpZTLlJ27ycVk9ReUcOb1jWbktz
4snVEgjHGKGJkyEBiapPQcRDGG1AX78/aVGEQmsX0SqcqWHi7VpIKJgKG81yHd
futnN1crMRnioQQtrt4rCkZbvn/slopxBUOyoMfuNRKmaNum4lfj2V73/XfPNM
oNUwL9yZPy+uyJPAbAME4YXTOkK2/0xIE07HwD20c8x+gTVCx7w3fkoRWye8s8
+IwMd+hPbfZlArv39sBdeXbP/dyPCWQY9qgKYThxFCA6GG/gNka/5RVMf9NTRZ
E11nqIknnAQZTWz4DtTk/5Z6jDILQAq4NlldvvV/0YSOORya9Jit4+r+vxyl5f
3gfZ51wXoTFuaE1t8J12H32nvtLB3XQ67+bPERAUz91c/WobwLPfVHMR1RBv+e
svk3CD4iVMnNPIGO+WTC9CK2jk6sFyCTTsFQy1ao4uajJ5iOeUBGACZVrlZbs9
7nzWzVlOO1kWXT6iV0SRiIpzMnR0Vselr01NYAAwbvdQDToeivfTPjYiZbp0a1
tjcfbHlHdV8vFIyJXzv6+mVA/qyDg0tDS/Jm5fdDJDPif0wdc1u9y8DQ8YuQg6
B6S+W61jYq5t6TJJEOdZ5ism6EfRLi8gHZHSDC08pAnxVzWK5WoDlMLQbxaQj2
y89AGcXlUWcSTsZAy87l0bwulscigAXoxD++W6CtW1unRS1fUBS73Cy8+CGDeR
JLZHUSFV7WLmjxVx364fS9k2dPg0tabhlq8sdf39M8kbh9a50JzdL8pkGH9LqH
Il7n9fyIVHttxk/5cF09yBnN446ujI/SxjBEC18ppfWtNJXAhNmSA/H/66KSmp
vlDnkbnSdS1SwTahxYdYRoxbZ0CMEHMY1nh1T1j1q7tF+ECffTVrBgZM1aSKka
V0Q+cXOrp5RRkRx/1/hQfnvzMDk2B8rRC+ISEdVYiBSLzZIuhntd6S/uh88vkl
bX1IlHrd73CrAuQ8J9kfilE+pbITbdfjm+6OUilzR25RtxMZcXQqgBg6fSeKXe
SE2iJ6PM/NYJPib0Mzpb6NDyxGi8DsutN8LqsRCpg1cy9QzLpKW57uoRbhkjrG
7a7BLvsubWxcuiN/iR01C8AHdlTVj7KEn9WpGCbX7FaJbxvSnq3LL426Z340Jc
AYumVRfZR1Fzh8oDscc0VWNmFjD1rMrq9oANG2sV1d9J5meXMT7CCHJc/b13Oy
nuH6Pt2zw6jx3Ypb8HHFcgQFf+oRKf1p0Wk0La7PPdrZuKgdc+QTd263bI0rli
a6QOzVl7Ok36QIabcEV+hBuCIL21O7sHJkhMH7tb56zVx+8g1cvmgUfFoKcDp6
rFFCm8NyO9oOij+3eE/NjnLxm7slJ0V05b4nYX5GmSTCffnWYW6xQd3iSn4ow9
Ylle0nejq1tMEY7LqU1CtnbSJDelVFCe+c0Crool209fwYIQT6a3aBq7A53bB8
2IY+M67MKVC5gOx1JwaBDMZ0x4rkgkVEFITZfwlUwhA21FBj07jBkeDUwZYZpi
YLMoIBufF+3f+El8PjwvaEtZMDI99+PDYh22fXQlSViLpavTuk3sjmPAthQqeE
Ffx4JoMPiSF69gYFRqokMXK76veymWdn+Mcvo3M3LCNtJN63GBWByAL9Rn4Nvj
x1PKMNJO1zEvtDNqOYplEF3HddKe5mbRoJkftN5LjGTr0KN59RcwHaOTYJijRi
+ATn61OjXd8Nrtl7YPan26YawdnZ84bD99OQtGMjKFBVnVUX7XW8sO0ZZ9cGqj
IZDia6iFhTmCGaok5wxbl8Pm9WS0zvfHA4bjBZHALRZr0tCFsMdq0PXQz5ewTy
q7+/3MX5ilb129BoH3P2VVPzVFyJu85OgpFHpAKj8wjeUndDUxf+dO3aGwuW9i
wkPe+lyUMSfdyBcNu7ije0upSKbVN/rK5wpnxqRQKdTrwAwplJCYIlF500EEtP
4aGx7uMVl/0no8cYLSBldm2x491qM/nZ9anrHw/0OuMbkrouW1ds78Uuj33zVy
BeASQZVwMKLMwUYOw0e09nNuu6X9QIDg+518RS4+ecmyu5L+Fx2e3qARfRV3Q0
RMpSDC9f9JE+mlMWg4N4JOovQEDjCTH1unnK2u9ojR+UtNdIau21Bl41ID7322
IO8F4aSTnqDHN+rYpMFQcB/GCBVaaQacx3XET5SrsuoKIohf1yBPp6bZ5O8g7O
5FXINB++DZ/OvzmCM7NxQeQheV8rUfDOgGloHpoHpEncB0M6LVWrXOsccWb/7T
ji5eUKhh2jc+dLi8eqGOgxN0LxEVrt3A9EN8Q4IB56tOj9Gj742V8vCvD9fytm
iV1aVbgYLZYCzTs8XIFovRwCFbfFRaJ13H7hJQ/QFBjoDKWdhKS3t2pEOiQ5eX
1jzBdLzaRJpxjnGOvCBjC4avwGk69GKfHsfK3wq0k2J7UeKCAdMxLhQ5i+g6FN
qKPP1zot9CUmaQNJA4L/hstZukLLO72l9tFenQ7YEKwJfOAB+KnxU/AzlELFQ1

+ReoB3GBDl3urbrH0KHXngS921Edz00DphdfpuWn+4lBEr0brmMYHAkpAUyXzE
aySifP3oaYmfj7uEFP1zEulPj60YZZ/vw0NLPtMvkqM2gtJV5mEPufcSVVJFOH
sCtQyD8ejAqozRj7JxYsFqNJtrgzQt4fOwyGBi7/ky8tKpbnfe+p8YlMvo6xPV
QNYzpmEtH4FkrsjO9k4l0kokNn+yzDdB27R0UUfzb6h2/HVFrxWGuMGI1FYeZ6
GAuGWBzAQO2Vf/Yf/Tmra09aHloYCQkEKMQ/DdB3DwFlFJMLCFoTmW4HtHkPJq
GFTC165Ck3RPrGGi+LwAKhApSlA+g/joRYdkgWYsc6Ev6TwQQTwaBoVs/pMhSu
RQUhDldF8rp8WUW+gX46M3+carwG89L7B3wbHOrZPFKFrnkjG/cUfDtdOqzrqP
ULbgqL6W9LuacFPMYKlykBeAoxzvh3fCzQAxa8FqW4k425uewUzOCO3UIDg2CA
KGoKY2G156Qx0itI3+LWGrAUdHHSaLHw/DaJemPzl19AFlQ7Qoi3HVn+N/8u+n
ceeUyNCwJkQX7i5OYolig1I29UmzWalDwO+BlNuXGZjofRie/BUEJCCEaWANVo
qsi6Jis4odLihXpWnfpxhODr559dlu7gSzBYLBldnCnlHzo/9r6VM5tKep6HId
I5XqORDOJ6GMJlbYKgNWR+JZISuPCJbi/mVGXkl08Wa62x/1yXUaUUMZmUnzAo
zez+dUprnC7J0xV9yHlHnqTzNBx3J1Q6SGtJlU3z04teTetd7fJHP2S/huUkZj
hWroXi7cbAD6yOJ7E2Q1luHK7AIay6QQRLJbBIDEvn/XxeKdiqhjpgsVIsUYpF
SZjFLhvh1ff+cpPBWgTax7xGV8yfbo7yra4iqarVnpjQDITHb87WLJfHPrNK6O
cgaUlYxL5TfVJ26fhVq6hyDnMdbdKKuKeyp3HEtHd4xsiDn4AlMX9UNW92BQMi
DqsTRQufcdi7DSqVbZxsnM29GflbN2GIEf5XITmunOfSox+5DUEhLeXKJwnhUM
oAoj+noSvUJWBgLRF1jcZHAVl/wWoJsNx5lvd5gVydgQaFfQvRtAwbYb9t52Tm
XnRxDA5ovW+TpkeGkvTWG+ydBsX2deZl5/D4YueWtIst5aRGezEh8Fyb2rsmqm
0wuiBf+b2UrlszZrHYStMvTBqAYIcTmxiOnWbSI32+lpbW1rYWpQHBbmCwUNPc
3OLQGdJQN9IZI6BsqyNo6/NF0pmnMZplQYsiXigWNGkR5E78taeHCaVo3pTfAs
UFAUFXtTYuh+xYu7UNd4SXFxktpdAr6MyZwcZKfBlmDyaxz/Zyz1jrqe4/1I76
bJ/Nvy8kHfN809+b/0GCh1MNee1QxhsI+f3KZymmKv17rH648ijie1Ut0BXkgv
x0SPBRZXp6WokxdV9EstV3d+7SnCmFvrpWN2IPVvMECyhygArzKiFGGEPRmDe+
PHbmPtDFDLG5rsO/mh2pxU3f8DTX61GgViOIYisKAPu/ox/kyJMjDV7Ozum+mI
TCijEhSGkK4yyo0v4Yrsh8X2pSyVRB//sLMtZIKrPdLStq4jJC56dHpe+91o51
yyGhmlE480qdOMxP/9XvB0AF/SsWHUSEroxo4tcxYRDc0oLKxfuwuXRWBG/r13
W6T9N5kWdVyZ5RrMy7kaXyZBDDU4QUhsgY+ohsZPozybPp/7/495/ssCr3Zo+D
Yfj376il7HHfqBpNma6qmll9oy1zHd165dx5SiVk64QdzvHxVabctp9rUsy6fk
URlIJM2O6XFqjcllCjtHzjxP7DKQQ077LVHN4maZhgqRyysCIPgcbPIJo5zCOj
aTBy4ybo2ybyylpC2VOryB2YnZo4MR73vWhb32VwWXWXggaJ2Airjjf5y2yJV7
8sFprWO8oOQY2f4azho8av9kB2aNTAIBS7rNMd8HJ/tFOzYEz4Guk/JznMtHNw
kNV1bFpP7elc6RqpoaGfkas4/p6SkYM9f401h3O2TZN2krmS4jT1wvyUjhUutG
aENRWS8fYya3Fjlr3IjLcxJHIp19fcWsDgFCxqOOuGTCnZBZVFZRN2xNiTtEcu
bXXaydxr3Hi96fa48VEmdOzH+sfkx6/xghAPbm1hpRyCxvnFhM/5zno/ND4Lbu
x7AyV/q7HrnfpPKlBnpK5Nh1adwk8S+H3gzXSnvbgPQUZ7ijtFgLYtIU4R/P1Q
zWorBCCT8794Ri1fC8B9dPb2Gf/oVCt+vi+ma9ipGaUPzWHh5ocZpalsDR1ioo
tw/VlHcTt3FhYvDiYfix+dpj4gmBaqYiaEQ7BuIG3Ph3/wkcXrEO8/SMxGOM0F
mR2BzmEJLAZCDregyUaHi06ozRKxW63L8MEP022rK5DXry0djqGpf70G2CW4NZ
sUf1REOwy7WY0T+UWbXbZJHF9++/y/FKbK1Lv6onl1sjxvefDN4Bat6i+V9mbJ
WNkXPSQC8+o0RH1Wgy0l2dhpYPtxWpgOe9I+BTNX3BwKGf4EErmeiDtFnQhVVS
eh13aC6wPb0UHttGfgbDOq+xtGF6NQAvQ6PC1MTan/+OjH9VMoWFv5/5NaU7JJ
mw+9D0HQtQyBAE4cEx9adnTo6LJQ/JhEWCDYHWBZjZ+eQOpNN118fZkRciCvW2
F03e+k+DlJi1SpIUoT9esv2Z5zUxfO2D/1HS7lUCoik24uuTVZ/iDWMKetZUlH

St6Fu2b1K/E/1pA2uZRViTB+VXuRNKqxFVzeXBIdm5ufJkjujNC8pwMRks7ml7
bIjSApX8duwKkoiAbJXerkxxR7JrRjmwurVHGqy7sL8PGelYeyieNNhJ9Uu1Ae
GU3Wz6wcBo89jLuhJDyXTpH0EyX+u/nxHwO4PGg/9uhRN2r7mbyaXH7Xx5zq6v
tRofqhL6A0erdc1ffrCSG3I9fJpJsQVsqMZNerz5wsu8fjyHDfOI/TTSzn30yX
CFafV8fXaU+uYcIzQ/u+64c+/bSXZwUY+RSiV1Qk0LYz15zU1iUkmJTSiWj9nq
tw3H8TZq8Qn0/kGNjekNfxik/D0Cfw1T2oD+KF+EEC1tLomWNnJisv+ROQodkj
N7ObX69UyKcKvYvHVjP8TP7ict9swjZVrlj5Ort57L+0QiTBf8kQNX56Jp2mhQ
lekB+Cq7y1AooVMXuyvDqydnVb2OJm04q45pTgowJV8erIWnn15D45t/kU+mYy
CX1Jz+bv8gkpJY12/5tIRbDzrm4KFO9fR0P+OHAT/hTICPSfBPX0O0Z1/2N/ni
RDbzyLkr1U7e+q/7Ossgfn2H+0ViBRLM8WDh0e2CAi50dNoiTTJXSEYc+7ZTlN
3IoNhYdiP21OW27l2RkIvYY4L1ySmEyJNuz9hcbZwvJECT5E3ztXlBS++wwBQf
ByZ+dNkppIsWfNX0VIvZ3O5TCAOf/ZHSEqiy4TRew+XTR6JcSHMhshb95I3JOE
rytKijjVktaPqktVWsupw0mjqkX7616w/y/0d0/ln3dZl5NkxWK6FD34LmGatJ
bGGH8+9yFaP3WiELvBczoKVn69m80M7dr1BrhwNaykwdhJJvXswH5wlcW4CcqN
zD8CM5gswWFhCFHYMWgJC+KwmOIKuKrqGYf1x3mEKiTzoJs+iy0lWazrT5yBi7
i97HDOvbhiroCXekrv/lkrjZFr8A26KRyHTpxhIDvtXGNgrHJ+5Oy865GcCRx2
Nj1yumI6rIrCvwHmP4umdAcLvKZguvfXz9mRkADr4sAQE7VqUM7YpbUJ3Ssszo
yDuZJw88Y0+zRUmLNEamW62Rw3yzeljO/UN+zkNrAljxRcw79WaXSrn7WvRsi6
HuWu4vdljhoOxmGL5vf+ygqxCRvGqGXx/NDFmXu4ezdea13yrp/kW3oCR/d08D
CXW4HHmHJN6exGFihrwX5l8YMFmWWCeTroOo2+oRRbN5+PQiwkXVSm2eTrSsr9
Hq6Fk86IpbKI8N0HTk2JIuQZFE/otAqjR+bLJWEqMaqfIrfJXfGHCBcN+im1tb
1fNugCbWOFlcLnorjWElHZvwpNhFKEWnlTGhPzW+DSkrwNu3P3ytJzxjD0WF59
QVSL/1JqxehDpztpHIxbSTHr1/oiGmOiGTV7B83JpojSMlVHe6j8umIwQvtclB
EtAW9TLDHL5mUruHmQvYQgRfFDUjCzne7saqBf4RUw/O7/Ho4sjxd5EuRGpfjn
d86MZsiQYV/4SwWzR9LNiiPZL/0MVC1uzxUtEhMeVacvB2tRPbhhykmbhiEKGp
fAF7FghHAO5Zhh1jUhGknX/jbpZxWCzlQJKCjkUwKe+XX1lUg6jiK0Bno9Y1w6
CM7DqxBCG//0O8jQuW8KDaPTaCYalnkBVr68fIkD5cYH/OBVqYaBjOoa+fTIi5
i1oPg4i6vPmMqMNMXI3wes+cqXv4myF9p76AnTPOWDqq8cSeseCMiUhKc2houF
bN0Go1hY2F6cgrC3iz+7Gp6ETkExrj5DNWHq/OUbnMi18ThS8vUTMR5xlGa1xD
2iIC5bReLElxRZN0E1TS0Y/WFS8Tbo7f9/H0/jRWStBBkQKnn2t6q5IHJqCGEL
OY1SkZTIaex0h30bxaXcYY2dRA4chsQYO1EhGwlFTGotUv0NZJtYFI2OkLANrs
N91IQpHD+r6IoOootgPH9Ml5fE62AbTCJ0moCF9ndHWmvbHoyTjXvQlujK14jA
/5tG5h93crR36UwYgdgCTqNE3SO0N7U6RP14v0PU1Cq0NxqljZw+71U+fUzhP+
Cz0PzjwkuO6cLzYREd0l3xczj+CVN81MwxYQJVwT28PnmB5nfA+jm2mGOCvbm9
MHK6wROnehI2W+eydCD7UbcBxndz85UmFY0ptcfjGNudS2j/oupbbNTbUcqkqU
wqnm9cN5GBf0lc46Y8xIQuTiqodu1Ng+mT9G9zAXawuq7SEBPCYDnjilgGPKoR
XrT4n38WDtEg2rfM54xhaMjn+/rJVze9crzFUrYVuD5ATOhn3U7mE2Xm56tQTI
Kk73VVJxUsRk0IY6u5pE0mMbPRABr6TA0NGQIwdgajQYKhoX//XbgIatTghiKW
M44Far5Ee521phFauPjff4YWUhCCvYOhytdCEMPLCgqKix0QkDPlbeaSrQztZ5
eOSDB01eeZyidOZjcI7znTUtYioOQAcoygMTQ6nhhYUN/7NJCFC//5Z9FiuDFq
FmIganDa1S4HC1bYFhhWQPpJdDjNhHdlIHVvHI6tQzLzvom2T0G/yctE1pHr6E
MHgP3x4sZie9oRMr/vbXksUu1ZPkwKZGnArG0M2tOkAnJ4dyeG/mpKA5nfhhhY
yuoBf8ZyCM7Q99Fxodqf1N9SqOI+kzwD5reMUEO05rnOH5rz8tW1frUQ5xTnZ6
2YW7g6BRz9jS3gBH0ZiybmZYCeKLy5YUQ2wtMEOsf50rBoQzODt2MomDHsC3Jg

E+MDHRre0NFbcaanLetvhDPAOu7ThnUalTOPhgnY9am5w0tcK79cHmQFlzdJN/
YnVlbYuDgTyVM5Nesa/k5V3ANy1qCOCGOgxbJm2X+qXv7A10dBispoQ7CDwGA4
FXj5XxRu5nJSdpXG9W+V1UNChML1MGaV1fdvLY3blcLhNsPof9/KKaA3LuLwrI
sW+iNslGBt4ULgAZYK7og+s86lsYWMetJduHmn85/3aTgNDzsnsSK1iCPAaPDn
3crFN9Nf/AosO85ZZAT6U/q3OmSWxfdtDGuy4tYvFKGLr/eXu5eBUwZgUDc3vI
pW5wD3ImuElRKs9S9cPB7E/e0r5hIchiXMmmyDXr+C1KZPw+x5NJaoNpCS6lg1
1wWn7MB3pMBDeZlLU1MCtSIWLU/0Mb56XbECX1FwQV5C/IBFdXHVqkJ5MAbkhf
SCAspfyRjSjmC1EYG58wKK9wkuWWdFP7emgFUJYNXXf6HWOpJLvH913NzH4WPr
8Lqx4RuvLTnRLkPkjEhoLGtMWDTR1dM7saQqNMnq/kNNfQtP2T0l/2UHtmFnMJ
ZbFd8T362P61rffhvK4REkDdmncgvcanj3RgjCoYH6qVZw7S9qLska9W1OTsJc
BkfpL+LVD7VkCAOm98oDfm99iitqb029JExRKmr2OhyZNo5aLDfbRLgjjioydx
eo9odH/xY/XxnmvH6tzc+3svU5SSkGYQnhUdY0cvw+js6nSJAAy0rZyLY10WsF
e2+DuyPWmB85ZVtzUyLnA++JUTOzphfhNOwavm336QVoTMmuzZpQ3H2/K0p+DB
pVuKvYiDbFJp/KKD/vNy+LKJK6l0ms8b9MQ5YfJY2Fndy+DiGXgczvL4pIil+7
fWb2unit/IywAq5ncIX6waj4dhguDgeJO2j2L+y079jCrZu/Yg3lQ28gwZt/oT
/eBoo/Jm/d1MWGfk6WNDzlsq62gPFgpZdsqyWcpVz4zDPJszMwt5RwtDfk8HeN
B61X2dyndeO6K3lDM4gSNyhibHu4S+Rxu8mqdbMbGLXNdU+57Kut48HK///jne
BMEQY623RENrKNs60+d6Yt4Coofi4bcT2TPHMBy71EnqHHEROsA66YRHOGME4F
tWpV/DRr3+Cvwn2CH+hXL/gmgjMPp6ZAHfC5JA7BmYk6Tm3fUYRyPbJ1xKxRgU
/K4wd5fAEfMyi7i5khri6L6Lg3G+zYflJgdSpv2eSfTMtmBU6/CXR3FAQXlvoD
HRMCoUC7vi0lsdyWVOj3oV49InsG/kKh11OQrFZg61H0uyc/fGvmM1Xnd1l1dB
ylWU1OIyAQRu7BXujWvo/ejRoHLJ+WNwwWNRU3PJk4Ji5njCcDi4qhdeLkcBE3
s34Nk55e+vD4ojAhgZhWkR6bKTbeDgDkGRNaNlPFY2R1RrlzHWMzhJ0hfkat8N
f0jOCNuzy2/vP0WlDonFNqzSwpGFWdda/NWR9QvBuj/GJ13kRfQWHPN3sjS3x7
1kja5QfW/Eabt7DNbuHvMpc9e9ObX3M9x5bXYB+VVn+rVJSsX7X6anEVnzwC1l
Hcp87fubCwzTJUhNOZ1zE0GbgKBzdZc7nJTMC+Cn4FWQow92E+zqqgrlMVHCwE
DGDahMeXGOUcprPc8DSFMDl/FV4HmQXNTKwE9Kv6tShBoj1RlvTSC11QhCNrcf
1XMeUHOy1RPShJoM8Pbr8KmB02Pmf2Mu4WuLv6TgGlAtm425EGMC8TguZ/XT7F
kFM0JwPem0q/dW08l7XIBVGuzY8et0Wnz3LnCkbTblwFBSH2EokV8U7XuwvmjS
JW5aM3wdVFLPai2+CWlGuf7wvNvnrHA3OH6JT2tIAshwPwAsW1Z5+seYwf4szD
O8TbBNVhc3N4VmbC2JsRrZ73byT7Z/NXi//W/0KUi/YbY06Di+N5StLUmVxgex
COhZc+zLNXz1gbk2g3vR5MknN4SjbKyM2v2zvMpo6uGv3HF3sXO0GyXeWM4vQB
s4Ufbs+pz5XoN8f5tL7BNG5gncm/bagu293sLu5rU1y+1rtg8axHwMXbPIXVVm
U1UyzV82pn8s8RP8pp5oIcTURHIqe43oGt2y26XWn5dCbfk8LunreG/2rq71zO
sxkj4KJP09s3v5Y7MuMZ58/LBHTun/Ai0zL9SXvDlPk/uBKjPr86pZYytvLU4Q
bBrFMWLj1oIeDDtHgxPt33waD/6YPwqxCri/F9uALPPlGSa5Xa4OTAzIDt8fF/
eusH8CNS8d35eDIXJ/EDGEJTE814uWe03CPKyvZoQWmxvYK+WxanMD3HJVx/FJ
4zsQP4yd76+fhdsfTIEDGp7sB+gj9mnzZ/3eWCOLCcJqGJ+/vUV+dOhueawX2H
3mOv7Q0gzIP4n769/9tOaOgkgYZSykGikcM63h2PZr9gf+c4RHK9PYpa0UUMLG
owYhD88fta4n+bwSei34zMjMFPYfQ5cwWrocWUmqeOUsfZ9ArkYx4WUgEJLANm
egtEzzCVgERXjuCKZA0dTabw1QM583MaMIrCQpAtarb7LoAABACEv64N8tmp3t
4f/PI4IIKb9Axw/yr1szGQy57D4DguEeAe/tzVGVP1j5EYw2rRakOMHfjQn8TB
5vHT5Qs5pJBR8Mxb+Z/oKnGg2T85I3JsC9jGxU6x2LuZb9FKuIZ6YxLu9+ACyW
ol1sonpyvGdfJFJK1oJTl9fLD5Z7EXdQCW/9ybRj2KMUo2LJdoM9gDYFDrp8aN

5RQywmSMzXm1Y3iXk/ULwJabw67bVNvIXDuLrCLb0kyVbdKxcR3f0g1aC4zx1m
2KJKnITV5YT3ecbT7nnkLlat+hqZOyEcMzyTMDKGrRoB6CINira7VQl3ltV7QD
Z166Bx4p7+yT8VzuT33DZv65PVBEeXlYKJlmC2x/I3qTS4E6Li+dCkXB6DEZx/
KfSZ6lvEpOiCIFtiFAMm/xX5VUSpSPgOwK2UhBfW9cb33BJ/ETmV6lcK+xvytt
LNXIG7eVsyUChLU20NZEp26lyO+A/GMZd3DOeyWLvBM5XjY+VbAXALD6m+a6/s
WNdTXNc0UR7JYZK0eH/2sMeNSeqNr5rDI0HeV1a+fg9NlH8iLq5zL8WA6D1bZ0
ST3nLoTguMWCE1i1Dk5nYDqGQHdTJCa620jtNMkv7r/Iwxsryf4hViOWq8dyMV
b78JImTq0srLaJMww0Aj8WeVPHhjcGcXDuXE1tbY1yrWGgpobQtKZM8PhPOatr
HFwk1zzA7gKal+MLHcbZs7Ktx9U1a5bbreJR11GD6yeU63MTaaOrOjs/uqVc+x
572v690wy6kTuuMkh3xsGJ+gVmz7lFYC/KLmjNHjg7odzFfS9rZ7YUUXPI3DJH
zleUvHIFdJiGLFCqDqiMvnGg2rUlVm/hbB2X+uFT2dMPgdsdTOKMZRdbu9dvAU
83jNkivukq0QjuRUpW+43oTVxju9M0xX8ACnacEeohCNro1fvCHeCXb+c0GO7f
p2JiqLWAoWGO7csvNRaL4DSuf3fG1VXV1FTR+YChD88dq+WuFI44D9FouzMiLk
JnI+kQk0N/abDP6TaUJVQeQVfXAbdL/GB3g7rVKFLxTg+jPIECEIcJgWTcuAwu
vQKXXMYHZnLgAL6j+mr3OXB63J3JGAwIKOB94A1ABICxO5M3VT55fKlzVw0egD
kzb/7EfrepZdQbaBmEl1wGQxGITke8bvuISwF/p0yjnxecP10907vj81LgnfsZ
x8aONeyS7DLU1h5xhQk4P7z6D4uSyuNc9nHl+HFrosGHfxzLuOoY/5F73LpuLP
jzRVbgumDvWFC2dR3G5aGH38mHt4C8KdhNlLkxzuYxz63fSddPpXVnt47KrtnI
RG9izCPzznC4884cAeYmbAOTs3lU4d1y1p3yUYVntxIkHUpf1NAUWhwea7VJl4
Q3hRoWpUPg5Yy2cIg9oSdHY0vM82mYbILNUQJFeSbeGQGWxnDkELNnDo3mK+7+
rV6gYnK7SzPI7gjw8m/AhUG+VhIvo7MxGszvq8rq5jJVgtKEhLJZSnpTDOiEGd
MlcftfjLOqsusxBIZgFg2CaTAzX5Pg1bidPcB0TsAd+foNm6aJbl1PCqlGwweh
E0Ef5G2AchsgIP3O2zO4MamZYGDGNVe6Q+G0s1/6fCQyHjO/+3qEi54InQwpjT
yj4R3b+8Hpluvc1VBuA5zbAD5jhlRx6PrEa93GuaK59qVP7aEzJuZEyP1vIBjz
nk2YGTw+LXbXHXyJMCxPnKvp2r11QtiPqfHvlfDW1hu9tlredFZ0gcitalu5rm
/tntsv1pJ9sUlDPRG7Tgim2WPTDovrZq2r0LgjCkS/7CTa5/pqeP0UuHejxdrr
THZp68Q0mhHLE+XZ6MzdBrthN4tuFeXlYUaafL7NzsKF3LByeIw6l+MIjfPaAn
mG2aN7Wew9htmBvFJfWYINddDB/Rtpwcw+MiqmwPKtnpOa57UsQWGY2TFuaq/H
ZSrLrG8oorH3ZZaZPK7ecVOZHTCMFujHXHkkbSZ3+mKdhR3jVjOGEbXg5TjG6k
LfKldV5YezzHsnD1r47KHEPmve534pU08DnddoV1F6Q1vKTLjr4cGtX5Ffvbe3
97dUWerE5zs3nro5XzD/GjV3Ue2p/4ZvWrW8+1O1YNyt1StwvPyjNsjKMrsTk8
yfaPnFkYQ7o+mBSGwgCBMOvJWrwDTSJpN8WFBy1Y/HPQrEkE0yvYyTO965ajfA
TQSRLJ8xR19jTa7M81lMent8DoO2Xwog7wV7onne1FST07aNLpFhwnU3l/zJ5Y
ZA7zTIfPhAjVn6us/DnLPD69VWbAC5NUgoniB3vJ4S1hLbrEWgeV/+zaO7b8C3
isLGHnEDaM9uaHQ0nAkc1XBjUEm++tbpTNlxud4ATNbiyM1DSmjHJasNT86GbI
FvJzNUYfb6/ksjHSp6c8vdaKygfk6toPkmH4dt4sJ6CImmQSJQLnVPC66He8SZ
KVjzR/mLiucMA8ksj37/PmvKjI5qUCAQSTCKpw9+3GuVswjD6jSNP6dbP7oStd
WSudoVAVp6nj+wfJGALTSMcCSyo1PPQtP/fYzpKYzTXTYK74oCg3MSuC3YyQ3i
4Jnk2dl5F3nx1MrFFlSAyVyl/eYDO7k1yw5C0JmaVRMHfznfTY5LE7HBS++Wp8
JjL8beC7Cs5b/yHwYXRN8VAAgaBbJe8bkjj6dUb8/jEUAcDNzpopuzj4sh5eTl
jibD5YG0m/C/2IJ+sFTa290uzp0VKKdHIIwfgqUyeCheKBGSPLIYLy7CiyN0VS
qbG4VikfBFIoTbaqSlQMC18OQ2COUKtULe4g8EPRwUYwY5eBDYgwDnBJngeKFY
wO6N/KpySz+JfagdfBOKtdRENd9fyAu3jf9K9G9L6VO9NWm8lshetkBcRHm/QH
xv+QUh1TCfRY9A5hmMDIrAzNfCjlWDB2EFnShARlGzkXr7feYL+MDbR+FV8C1g

ecuBmuDveQn4+fHjIpwFw8b447EyPuC1FSEjY8+JutEVBdkyRD5kZbVs+puzva
VRmeL7dh/vj3cm/PhfGk6kMg0jG3UBlQAhTCyVXfdPyVEhyoxX52X4zAuEhrpy
yQLepJDXxZ1isjSF9/23NZlqBCK31E76YWbhH5uKgKPHlDqqHVAlVGHqxGiQlE
UmXm+O13nteJqCTf9tLEaNjAnGUlzXyFi8WUZjpjdYT5laGS3gxyXm8CxkGY8B
iMFKoNtxIgpTvIoNN+eH680E4sGb4z44buYxj7NRlpRXLLnBgLOQNGKMY8xAdf
Vbi1XRb1gGYzNbN6hraLod5m2SyWdtNyc7XThoIEhZoZDk/l9VX5nEDUsZW6wu
52h4cHk+B1o3bKVMU7/FGd93IR0O8ikATFVlnLiYwVhCyFOYds1Z+OfT2zoBnz
g7aHA8cHUfMnEhMAhCahwxZgpKbvDhmqL3AmRi3ObSzCCNvbJdXf/19eoopFwS
l/UsJqO25cl44Ph/EejaD7ce2f7f4db8VNevT9Js1A1CmIQMoZb3CqBLGhI4Wi
xinh/UNSHa4oZT9UBHng2RLVZLSY8gNbr/0cHeQG6Kuz2tpDOmRQE+Mua26ZAr
2SYGpXx0E9E8dpVaB5Q/14KeC08RYqWFIj0CD5e/JZ9pO+qKaeGemZn+uOnLv/
qb/vvzVgm3AMDAgITMZXFMxdZRLXn7H6o58LABuuBPjKwUejZc/nNK5xTklWps
UoPH0jAs4uqNeao9tQC1lkalVudjy4A4jQFa7GWjHNc9vCxbPPGQQ2YNi2IY4F
Y+YylPoS8pekl476VT/zus+0i7mHSIplRp2ggL55RVljs3WSUCy8b3CePSI1cT
sPSeqCbFN1ILDVJQMbjZnbNExCTMipBFtANkQFVjcUHOZLKqMTxGZIJG5FQKKN
2n8X+tOAJrbKT1YCbIKSKmzGAoz5AxD1mSpQbVe+kmOGyAVPi2LSImc6ALaK9H
QllNdsVruoFKvsIUwJp6xvQZoMhxnSKaBb1HU+bhWIzf3TmLwOc1R+4COHpD0H
agVbzmwr4M7iM+lb64jU6N832KbhBHl3GPGSAPpHKaIM4FkJ84iYI9z1IxbpJ/
q8+xxoLcB/QxSx5eVpamTCEH/p3C4eh7T6Zuw+iSI2VaE/X5Lj4oEdZOXdvCIn
LyTFhkaWbt63kqIzlQ1j2Se1QHaQIAPiGOxCmYXd8Ngh5iUAtSqWayNWCJfzkH
Q2PcxG907lgPy7PItvhVgkfUMZP1a6d6ilWsRG2vxRzgvGXZyjKfQT4rA9qAqa
p4A7YAywc2MiuyMtDfZoWZl6FDPijiPvrKshEJJTV6ZYxRWfCMApzZTGHh6a/4
gI3L40He8054kyuSzYJbR1lOjadu7dy29KVwkyzFFo1C+4Cu2I+Ox6oKicVNQL
7j8JBg6oKs5Usubyfl3SEC23eH8by/O8Lp0u40UcF2p7MnoTtavzu3i4xfXJ7k
ucfhKqmab/AsZ0GQDAUjnTzipdgtTWkndGlTlCpK3iAh05Q8abTAR9lFiyPAW6
aycpFOOBwHlVsGTumE7E/S8u3OR3RGeUnX+c6sLTUafbXELU6NMqcV8DheWvY5
AXqJLSdehO6BG8aPUo1U3NNKQq2FMhxqC1TjnDQxlztCtOgvzkyI7MkVXQJMvO
o+NsZlXBicEKq9OC0/J/WQX7IpDIhCPfBFR8VGFGFyHhcHYrlJJ3XOWgq8uE+Y
43Jur0BsQToByT1NObpJZIa73gJ5IkClwEnSAqYqagOnlHPAw2lpYzQV5kIT7Q
jccwogR/OIBS2OLLdaG3DHBBxNowmpOxDCcYXx1qpsW2XEO4qkmqCJI26SXqOa
QpLHlWvLSxTf/UQ4KuTmk/Fh255dvb0ZRdAV12MC8PGvg2FiYePg4uHDEUgUAS
ERMQkpmTDhIshFUlBSUYuiES2GVqw48RIk0kmabP87jUySpTCzSJUmXYZRMmXJ
ZmVj5+Ti5pEjl1cen3wFCvkFFCkWFFKiVJlyFSpVGa1ajVp1gZlnviV2mmOFQQ
9Y5meGLbDVB7b5lMOOJMtnPJZmbMKXfMV9PuehDDNzC0sraxtbOxKSUgoUKlKs
hKOOed8J7zmu3wUO5FZlhhluhHIjVahUpdooNUYbo9ZY44yPHqJi4hKSUtIysn
LyCopKeRAVVTX1/HlNOAKJyleQkxCLwxOIpHxEFCotPxmDmUt+JZvD5fEFQpFY
IpXJFUqVWqOlraOrp29gaGRsYmpmbmFpZW1ja2fv4OjkrNZodXqD0WS2WG12h9
Pl9nh9o7ptEYpUnAhRCdw8KFvr6hy/gezLdqfHMalWHx3yGmlNzS4N99nvbnPf
sLabl7GOYZge0Vu6AoAOBg4ZcKWH1HZ/hvqsyr3x63Ljea/bPYrDnS1d91Ocvm
9h+D5K85OOKvN/7T8Ao+GWwfdQe8XFqDViY9QaoTVaGS6IW+ZhukDAxVPQE07O
cJTKpygBDcoU7vlZuZyyyT1nr+qRjESPpI0eOpvRaTvLZF5vTkjP01IjoCYX7/
SI1nKhum8t54nMCDBJwphUKUiyELB+b/hOiJbEg+m66kZ306EGXblE81vm8Ehn
v5tJwhQWtKh8DBuZD9mR46xogIs9maaTPC0D4lInC5P1EpmY6y2QmVhm7ZF1au
dS9OwZVtmGQkOzTeV2YbEZe0l7m7q28yDHs9FXrnnO+n3aKUFaT6g6eZDqEpFm

ACOYwmTW6jsQraXrVmbmJRaJZXvdbarG04TNCKYjk7kA+O7ZTXSBI5nROKGnJ0
KHWXiqcLm4O+Y2VJr0qZAGNm4XRiM3CbueGSpT7CZI407r8N7qctO7O+oaB2HY
bDZonpEGvOnarFMFF3NXWlypkAYu7hbuFDCDBVIhDWzMUWEzucGm6/QNLr1RD/
I+AFeLyo5A0JLywxAY2mMdRwDDfkwLYBBn2Xo8DOPTYNahXoMK4fVM+2kGoIGm
NJtS7KweWIH105MAAcB5JAoaDJQdtg6WHQoaHe3ABxjYUYiJ8rAjMYc1p7A+AN
F4UUPQh57vH4/ACBrMi/sAejtvB23xEXjaeIzeiAKnEw==)
format('woff2');}@font-face{font-family:'Computer
Modern';font-style:normal;font-weight:700;font-
display:swap;src:url(data:font/woff2base64,d09GMgABAAAAAGLsAA
4AAAAAybQAAGKQAAEAAAAAAAAAAAAAAAAAAAAAAAAAAABmAAhjwIWgmcDBEI
CoLGeIH6TQE2AiQDiFwLhDIABCAFiGYHjhkMgTIbXaAnxF175HI78Kb8Z553wo
1x9jiQOhKMomZRVlmy//8/JekYww39AE2j6j8ky0WRjpFwiFyY86CAwykDjjUC
hwYd8q3z9AHJlLpPmSGzVKKJ2XoZM+GVtChwW0pH7YMyyekx8ooFtWZfsbo8Qx
DfEGQ4icXiQsDt6HHDedmvlS1hvd1v+yAZai/1/rqsL/8Q8CYJsv9QS6jxmu/W
SU9GimNPFEdGXEbulZTI9mlKtZ2DhqHPqYY3Uc7PWsM5Pu4IjX2SOzw/t97/2/
6i/18SSzaithH9WSQ5xqgc9ERoA6VEsUHCSE4sTu+MM+70Qk7PuDLqPL02L70q
Bt38PgnBQkIgQKCG1mn9mVVOXMcvNow6zXe+Hg89R+3PzB64dyVWwPvSE2/LAw
4DTuS4OIJeFoUb7gzltJyRkwPJTgqCwpMQLSvwcAFBs/ubuf4GZCu0iSVj4oCz
tgNAO9L0Vf5d+4Rn5/2mltTdL2l9KDAkPPY4zMWUSBqlnb11Oe+el2F8x/YdGw
ILGCxSmjIMNJsDBN9vLX+SmQUS5vS9syVhq8QCkHUFdqr/p1f5V1XTHl8fL4gJ
gPXBfq+uDBYVkAJfmVS3zw7QgUPjhYR9G9IEpVvfFvUWZZ666svHyQ/eD9tgcX
/CwzFvnjKiESah9ZspUzkp8ESkO1UO6JftCwR24PZ2GbtNmN/g9fFP/LsthGJI
zVBMie3z5HA4ZhSFDW0A5ewSVatVIKcB9SuFMr9C2Yyzp/keZc44Vh303+918y
LvpbjuERojS/MgJEJt4s4B9wu16VZsG58Ei2fQXfnenAD/ac2082YKyRyCKh2r
kgd0snJ3NvRnLjnAzRwmpdyVkhLmNkdgi7qywkYVXGWFqTMVwlS6/p+qZTvkal
OnvWrdrV2FXHR37tzZTefS5eAPQA7+gAEDUjcYUDII8s4SuLtPBKRnCtQGUSk4
xaSlNlIXtU65u+emibFrXXRuW1/Tt/b7ls7u5ObTF7qKlxiz18pc6l5oRZg0Za
AbxUPIWhTKYZRk/tdqdSG5JfFMSKYt//U58zkX76jOmY9gMreIiPdg1sgQWuKx
rFU1gcTojUXbeIAQAeu112//30/9dzGj3Xbycow1iIQScsHL6Nqf2nHst77SfV
H3F7XrGYWBID0wGGUBHE/zJBIAP7hGANblQz+0BANTSXF6P/Dj1Hd/xJ9xZhWJ
0EP2j5J1KJbTkZUBG28UAKCv3A0AGe5TQgTEGw1kdcQK+tX39EOcRgWWY4+9Dj
rsmDPS//uezz32J9RGbsblpbyWH+cPK11BhTVVMzVXcSlxSIbIaDKeTCzzl8ll
aplOdlfOlnPlmFwk95PHyLfK98tn5W/Kz8s/kN9WAAVBQVJQFHQFpvBRSBQyhV
KhUoQqkhV6hUlhUdgVmQqnokpRq2hWLFUMKVYpNii2KvYpDireU1xW/Kh4oaQr
laoTqtdVb6reVf2o9qgXBj4L/Dvw35DUkKNfyObnATJQ5YM4Ku7iD7iu/h58Bb
4HPyCM/619hIMyWEaVYTKRzE8m+4fRw4fk5+Tv4BpgkDtQBV/hf+7A42sUTYol
ikHFyroxxUu4gB/MfnGKUvGslWqIBfrTzvn2/wua+a9/dmb67JmTr760ad3Kbj
veJn3kw+95t+c+22t5TFrZinpbXE9dtVVn7vO11lht2KAmHBgDeeG3LZfTTjnp
hCMOm3XIAfvtM2mTURutt9oqPbq08ShTIk+udA4RwlGQx4uRHLHFnNTw+kzfXs
oSlbDdbnEo+AOQ0hFNu/ggAK9Yf/Y/d8FGRaLhlncWfo8Nb16Z+uiNTcRh/923
O6P+JEMDJbgVJga8Pwb25Xtq1z8DN9bYfYxL5FsPRJ5ODDHkxUJ2nPcE9whWMh
4AlFXUihBMy7IignjxFRWg31PmL53Slh4RoxY032ppNVdE/LoCqRGxTT3xxWpA
jQajMhve8ILuwV44vbyBs0t08JIn08p+cDXs2iAT2hjBXUowYEBaT6TVOtuhB3
90eLv8c+gmxsliQm03+MoZt/mvgMrlu1n36sXdPtGSO4vzO4+GAs3Us6utGs3l

JzhRIj9mYYn8dz0JsWOEqRjJIr9FtlQ0KxtB5KRcEFwPBBfmtg+50/k4GR+nY0
cW0vBYlee1s/pkhOEs9MQeAyVFraIKTEDsqR9DnV3A0JQG+s77cl4ehdbXFEnE
Re8pWsBVx207XCii0jWpdv3DD07k9OtDijIZbRHn7RA7l+YWxLyBJ9cB0VFibc
f1KuqgydEAUIiJMds0FRCML5CIekvCvqLe7bQJIO22uTjG8vzZmPRJMfDiuLgV
wWDOyWqiaz5piLU0iRxqwIchUkjsMbiQi4q+pyWoBYNe9Ldy0luKimed8fXvhK
3rsvlFDiw7ITY4c7s9NiPKbhe8E29rnno4Cmrs8mVNs6KDjgM057u1jV2ZmgC1
3HXhxIq49GA9clscY3mRi0qXEfmpOmUq+tsXeP81FDwY212yoawqeni/Hd5/We
E63x3znt5HAYLfuNgyhFXphrkTI9fC93JOD+GigIqPWPquSnzX+W/dZNPpenlT
qBwnI8iw7GDayDlrqTKEAYFaSfgOLQBcsDEhCY5+YUr7v7QApwq0+zVQmGJC25
TMiIoiHkXLp4ErF0RVvSf+/oS1+BMl/4m1O6Z7JQoIomLo8pBEeUQqeYVU85jU
8iqp5zXSyOskzhskyZtTJYT6odTK+69BUBO+3lJou3HQclKOfdx7eLI99vl5S3
GKk/CxTVgPdgEUFPmGDClyKAoomlC0oGhD0YGiC0UPij40OwKaZGXj/MiooBVf
O2zDCTdAHOVrdYIMj453j8/LFMAMkYwkrcud+sql6DwnM3MnKvbYbrYdxGm3qG
lhEu6bh868b1Yub20AT1BPbuIUbLNrLyAMHMuKIiD/QZkZv5D1spECPz8vW0CD
GXZFsOI/oIrbqK80FXe6UUdFo7R3FwWJdNSNJihuKdNv89/7tsYJYcfd4/aaDj
sJLlxc0917IJDf4exi/7AuqlDx97U42s0jA5q8Ay9kv4baVJVaVFCkHj69P+k9
uQx7WM+nbovz+arpvWyHELlQVjKNvCVvqhjv6cxHpH0wYy9UmXo2PDc5xHgvN1
DdT4LKWVmmad11mnD6aOURWnidERMUsEXEGAhf9WQHdtDEeQvybiubdS96lR1u
c/mABhtCXc2gpFlPnzz0SGR8CrBBiGD5SMwcMUmm0fPdDDUa8a64i8BUwq2u5s
Gmt89bzAAum+lWpNDSvXx8DMQXDjA+uRnobiG86gRSAwRSe/uQoXNzC2isLpPk
jzL3Xbzk3HXbL/VWrPxxr8C92Fh2De5HTrXJr9cz73TrosaaE01OeUDmppNJV4
m0N92Gw/mCBrWsmw5Po5A61O5rnc7/FbMMBn/gPgz1l42sO58rY94wsjMzYC8I
obOBM4x5mTovlnAf5/qDm/zwAG7diQtSaKt7IedvXUAB0BviNiYu5mn3V18i8P
VdQY1pLipRw5eRzM6De+ZdwQDAtMFVjgc2kBCGnB7Jsc/HCthEQphy+TPsESck
zAVsUQFhKeiVCk987YHegAfYCnpnD7t8r/YYlhAETotyHHBJAZMlBIUzKibscM
3b1zp4AEMwJiiAJdA2eABHuOeiEB8849WGyLf7fBR00OMeQzFOEiYF5CoCODFA
JT1iBinlUcbLcgiO6BAeqSNAnQDpeZ6ULixdWbqx9BqCegOod0D6AZPRJ0dfHH
1z9BOC+gWoP8Dx/5jwaz2r7aO2NuDG8XMQ77yZjmgMqWx3qwfqe6oEAQjsxmYg
1UF7huADxR4fjnkC9DcApwQAEL7BDDz1FZg3hrAWcSCtKMZqcT0PAg+VJDYrTR
q7GMCLeFG6XgrM4+iEYhQOFOjJABLvVyKsuFwa23oFT/A80NKSGkFEAMWNB0mU
sgKPB1ETa1iVcraxYhZKTBSoF78oDZePhaD9q4vCvlWvP3euEDJxzP/2y7bdqO
fLIsPClVH1+GyqXhz7kzbnw8AWdpYw2tig19hYjyk5CpzN8sYkKNe1eVmVDWcz
sU74rXbZvrYcbhiuwfGG3XDKgeiQ4QXV8Xat0GoehDe2pdeyE16e6R4Obyx2Nq
5b1/KXBrUC1+nWL+rToWtfWZiCMprZNDwl+o7Z5BwLp+q4Ts555hMZ0F5Io/hk
p8+saw8wjht1c/bamaV4TDJ7vL6ypNf4f+Sv9mQyPHv+YI+eYX9zkefZE784Xl
y9bJ8XDWNLNve5L8zdfVa67llhiM5m0JNBy2Iirg77B73BsddjdDBu54wKh3l5
UAli+LmXndFw5A9pP4m9OpVepBNdi4gRWYSTiPGI0qqT+vXA5NtGSoQhnIiFob
B5UfOZV/DEekZNHJTNfuVc1Wlb72lUHc0vXThzNVme7rIO8y3Vws5cCkZGL6Jb
2FMBBtwnlFAkxj/+XPx4HqeAk8lTP06cw3aAPClo+u8Uwq+VGEaQCpI3/U4kno
CYCK3FEN1whOfxpTM5J9xHYI+Di/Bx8n3Ur21FgJbXTd+2ybLMS1ymObOG8f+m
AjP9pGCSKWCnhgnOZGi9KcKG5cWpsQfxlndxQgPg1XAiS8BJNBfvhuzRHTHBGs
06cBzJANFrBA3qHA1dyLQpATq/yIJoJnG3GRTRqxW/OEAybMvPgCcgKkjdd5D1
29htiYywVehBhNB/4HxcEF5cRooGQKCOXdGGbC97tXm7622lZK6ZvvjPMOctJq
P4KrzqANEdYVqJtJxH6jU6NBg2NwB65c7J252mX3aJfS0kYDGARSZsJOPFpkRy

SzikJlTXCu1HIp08CtQ5qs93sJfbqcGzzGyFFMJNgaggx+E0NJWn6FqUEUp2ai
X4kYdCkb47ZAoSM0guhTNx5OSut9HqZW+zQj3QMxqMr+l+fByi4GFgFdmtdYWX
HPc0lpIwhUzOkCfAgnfvAjSJVhF3kvBwBuiAsXiIHWAgw4VfDCwSJIMcFu55ei
cHkULTdtWXk7VyBfFZwbcbxHAO6WnmrII5UODqgmG5ba2X5Y1WDx8n9P79ygBp
gov/xzGh40BBu3AZ4hm9WEiW/KA1WwD5pRvYNaFCHqRg1nf9DU2HBkToBzlNlD
OhX/qcjgFM/yRSNYeE2h720tOqXWBNTdTEJzHhohAND8MEaZuAAUScqkL7DZHE
JbQLHGndRF9Ip5gDM1bStQn45VDsrQVWdLLV9MMRdvHW/vK4V85G5XSSy+FnD7
IbqfX0IaVTn7xGGlw2Cz3FHYekEsqnQGxkE0ZGClc1Fy6htI+HVYh33oGwqSHW
QJpLqejC8tW4CCenldCI5NUwgkGEIMfXeQZ47FR9eb/peIxkR4OiCZBWTb3Krp
wn2boxcJI2PUVysVMa2OOFiAbSh/4gLwBdYAm7MITDt2USeO/xhsrRJaE2aFDM
JnACi7JF5NBbYhhSlkgRiBkyE6PuwTTpmi/2/TMrTKvvzJU1mBGhFeXYKWTzNc
TgS9gDlDE5mvJCv527RDZu66gIu6PK2wpiUtMqgt1e7qlrTsg9IBcg/wLP/MJx
YF8T4ceKf+1g+XgEa5QEs4l7q2prscoG5KEtapJGVHM7KcxA4CSiru2asFCgLv
2SP4biy39XoBfEz3+OWy2S2fTFj376U8rMt5rwCUPlu39QNP6PhYdOh9qaGjXn
TOL6r5Q9i1mqJuY87LX1Ec2rbOXRvEDBj87Ck5BSTcjQEONIWmcNBkCf2jdZ5T
1B+DO2xQTAC85LFAJqdEZjkKaOMDXnPvZCitTNs0x/PN7zFihqolHniAAtbdxD
qm0B8+rYC3IciUhQEvJjLrgoMP+HAE6AVnCWwP3g6/XXLpIb/cozXNQHB9ibU8
yV5xk97LrVYLYoPsrgl9DZZo4CYMWVREYBaT99U3l6FgszvC5ePsViAYAmC47A
Zt32NSQhvyATbK5Um/ZsoEJMkJQfiEEj4Yn7g2gnRksGqroqJ9NHr0/7j6dLoa
HpcGp3JG7WNYN4PW01J/Dkvj3q70qHwyDjUNAgZ6lrxCiAb4pLsmNUMHu3OSnj
tvzZU72BBk/cCBOQwtXCFX62GeyVEbWmcjw4b8O2nK7ENHCdkxH2pdSRUbrSMj
Lw7qhxWc+Qh3r2q/ngHxsGFkFlshpYim/zYG6k/KillSaE18f9R3CK75qi8fkN
+L5upW6AvtqaUF7KR1fMVfgN8pLgzC5Huk088NtrPka30XeCAs+MgFH3jGFzlW
PkeJK0fOOcxfbJUU+pA9zJTUF8IwnG6G84X/8ZSWFfVVwZJMP6lzRjX1nS0O4R
TdhfzjeOPWiAmOK5MWlLsZQyYmJzKh5hIJGrKFuUjlgy0hpxXI64UoqKWEgvxr
nS17uhqhHvRxqvkAz42NeRjLJlfjcbkU7jS+QrKaQw62EmLir3yNawsLVXKwFn
pXVTPLw28bUkc9xM/cZpX3kCyU9pCXsYvAmdTbGhFCfFLP+BDm5MhsSKDx+3Gy
n583mQZHBWjLKaAWJrKHTA6QiVn95K7qlV6TIXKSpc8ZSiaq20XtGsLgZ5C8QK
aJK9DUyFmUNPSbccjcjgaGGd4pCYyZz+dBedY3pwGDggDabVC71xL8OKQ158oB
1UtgRov1gYGDoYl3ffK+GQW1vAoYnFslUi94sc5yNVL4OYus1dY2y264ZCr8HT
+d8wvmMPng0UI89STzDd+CcREGpEciSvidzKnxmDvaf2F/wTk1kYkM45oj7iqe
JNhd4mVQKBN8bjnwYsSk6Ve+FvKcW861sB5ntjWpVbhbxsMdOx6AApQAFTqdwf
Vf20/xswIaYfJ/E6qqF9aDw79wkYPGVP1NOYaq5xloKsHzcAemwF/gomsQPzMz
OUjSEF0f9i0HhMh8bjRS/jWKeRVn16xXWks6A3DHz48OGJoHljEjP3OpQyIDo7
RMuq0y+ZgekLsLdAjk3E9ugt/pTusYdpWi7pxbxsIzUFHLB1nf9wtegL9cx9Vk
YBv7aMtMIEk13sW4UhpB/ztF4UYUY6OcuhyKKNT7cqhaxXzC8ZAQnpPE6Uh7V7
tDTV66BL+zwSLOaFI0cKUXGmB1EX3uidS0dRv/feMQkKFjjj5h4dUrtQAMuRgr
wV6s5oJmeWXNiwm2xOkY76gyJQBgu0xC3hC5w5T5FR/FlWJ6aFb+dRoEU0W0V+
jWNvP0nPI+w2axub9oExpIlgmQuCSYWwb0OAzhukdgoItxgQFLfKbxhrUv2DTO
vfQAyD1WyyiIII17FLBWTDizGiu61YbafYPU1IJI4RDyeQzr4SziRKMqzhGv8B
oBDe3c6jIsOa/w1Ffw8FNZdWea18JBexZn/zwC8hkvYbzP8JYOFFrcUWqzm+qN
ujT9IVydsY/1hOoRB9KSGwoFsP3MLxaWW4wq4rAd9XOLrVZxSa25t8ShWqlL/h
M0zZorMiuYL8An8X/fcC8xh/AHnfpIAHUoRKa83q3cZuKf5SA6mT9zEFPRZv7V
1GWncHUfrvMc9q19AhWyAK2lM3kd6CJvZreNcSUESwB7ebMjwD5CsQulgRSDPv

TSPBYWcfu+0l7lw9Y8HBg+D+TSRgKXPoW4zUXtFuNoYSw87ZXDp5IwYNoGnRty
zlUu9T3240rRfTxgtJRdMzPYXBc5ShDEvUPEeUIghJXtXLZgLryU5AsagSu5DL
8fI3/NwOtSvlTljzaXYbCy57tf6M3ASDKLx5wJ9wV2vRrqXfeJJEUo0/w27XWL
T3Gt6bL7v1lZPungaPD7GrggZvHR18D0csCn6a17R73pMaueG7NrGjq+q3PitJ
AlPGuEIua06990WXS38cfy8QNa+pmeMvBYP1SoNH8lou6ZlDJkCXHguANo48WL
Rdhoo8Pta759QhYt/RA2EQfC1QHjogByLo2/EdMlo6kdZkLX1TbZ0SvUPAhLT1
p/LZF2jJWRQ1tgBp1VqBXUBm9FACFs6m3OY4m8jpXa8w8HcTw+IQuK5kkAxm7v
MQHIvKs9nPd33zeActkCOddJqijJXWOnYeJYA1oCOwMic0uhmU/vJVjXgHwqen
O9ax4QRuT0v4F1nZ7Y2z8MAUNCrrY/c4L2MZaknSQ3JYdaf5csdnuOJNiWGZ9h
YTasAY1l7mHS5wOsjzN/rIlbLNYr+1ggUNndYdMcRmkWoDgQMxGuCbqtzw069k
i1axpz9OUcbCk+e7ONL8YvOinl/40MLtj4PSznKBr2SgbdwOvj5lCt9oRNkXSI
CiDqezXx4WW8GuY8D1RkouVVmJRpCySGNhajvFhcQmKpkYxVZeFMPRfhzFhQY7
hBNQKwFEZ6FuQgfqIoQ1eq3Le/uImlXhAmKX8UxXr7KwoIl642b6MqTluGZbhd
zTqEEdRnhm5odcxfwC3hPM49PrgjLpKuqciMOkdGYspVHGjUKMDRwVrfK74N1R
YRPFOMPQa96GTXiyFATB29CY4cpXfmcNhRQtVfTvqSLKy45sQG71P4kf7mOraU
/ZygCFpl1oSIZUtlpNFk6kbwfKAeSEtxtEjW5MpNreYLbGp3UFOXU1BZ3TieJs
csREpDzIr7Et9qs7DkBgJLXtC9ERKFwNUuQUi4EV3kY+gQUoiKoXtvlT2B3r2E
cGkiWylvaH6/wOfsRtpLubgu2NX7x5w37S9IEPHANq32slur+/2X7ve49DGj5g
A5REFxFrQ4FkCa+rvghD5c9D0Eb23HGjir9hWNDwQm8Tp+Y7p97sECfb692NgL
DBu+fuwcu9p1fX0Asxb/TjqZvJ1hMbkLcOFG42y1jDOG7lVzxXSm5MEbSIrFoy
3lOlSM3P7k0bvxQo7h97g6Dux5arXJ3l23/iGMyBeI0RUSry5Q0u1u9PchFrRo
XdBEdFX1pIWSNOvEJoAMsP76qRI4I/ZjIZ8HSqUI8eHWPvUHU5Ln67i2mWhBm3
I4/HIj+LUyD6WffPZcLxAKIs9XO9IwYMEBc4GWW/pYzA6n24+D0XGxZcVqALLv
aKnpdYmhQQ3BYgfALl60gbyI8RCx0xwE021IY8BXXgM1REFMTgjTQwUIVCmn7U
JgOYXuLboMv3UUDEZln8Da0C0rCEBvC1E2eBSJldPeABZWH2TV7lwjq/uiSTes
1ngtBKZgeCavATNIbFDLWVl3IVRZsL3+gcOFj/6IcoVIUkdCzT+Bf/9lTGDsJZ
xJiyjrRgGFKqCqS6fKJzCJYPQEkhcpphqdvLkUBu0dfhNuGVrG5ygjmKQ+WltU
IKExT4WSJCde3TwJmmCG+iu73Yml+je662ni3nUS0Wv0muSpbvd9VR042wvetq
DuHrnprmqkUYTE5WVSuSs9vY4WEpoLjLoAEy8wf4ow+pjkAPyvWWEZaXYgDZHd
EC8DcX9C1xdNe0f0/lSqnC1DeA/ElZGCQvC+EwgPDZ4F9aZCRub6LJI6xff5oG
Y59XsZH5t+z3r1c7Z2k1O8+m7et7ZnP2umzioGkZcY8A9rdFuA3yIbasBez8ob
eWDpCAEn3dxynZnNMeo2qfEznj/oEHS1Ze+/5wImcyw3yFYa1n1uQZqH5E6d0Z
pLqpTgvgIrEBQCoZV0nQbraAQxraEIItUnqUUzOJ6VRbRFkCNsHl2lMB8VMDWP
JKFjgGlsBDI+Qo3UZbA7vdeoUxI6BJcB8U4FU6pWf5dIt4pLGpEQUm26LVJHud
Ht4pSI07jkCrCA3HelGrUQNdaTmq1ITri1lLwJphOL3mjJjwUwDfVDQLOzPVlS
1b2cnIWThLFYQK6V4ltUlwdDtMejGM/SEyeCD7K/vUehHRqzyak7AGSE3g8CSK
y7dYKCOLqGIIHMn4sYjSRkpseKS1ZF3plomAQFmX9FBHS/tea7SitrCjmzPGVS
SO8CwMB/jqQ/LKeFs/3MeGNFmKhhhvPzPm1bggiNASP+Xgpc+ZUebeddzwUOYZ
jGiJUHMxun3NuEEV4xOMc1TR5K+wfryBs+sSIVW2RH5fQG4l5kPSSXfCV3NQPE
j0OltgXQf8M3SJ2JmBYmUtZwES8LMpSNLZwhNOvHTSyS/tEmTvQvrIAQjTb4mq
aAJQjPOpP/jTS8Yp06z5RVGPH8S6bKgOnkLaCWTK6oE6+MY6zFgZXmNtmRDOlq
v3IGa23WxRCogLaWxDm7zQp7ukk2WvaKaOAenWT+qYHbkl3IUlhqXW1I7yA92N
oOnUj4YFwa0CwpqstJYkKLWIiSPwLib2IJRHSNrYTzGlCsLWCiAKhZNypCibQh
leRe/G/cHYdHaI2eC4nk2spNgC/HQsGqfa+c9Y+ieCrrllmwFmkQnlAmOBPaoY

```
Ut1gGyi5ObB4nRIbolfATyKbyFJaK1dJ4/7A+suvMn4pNsiXxqGa8FiLOdUDNl
BE6lUhNX+h7RENSKuxjFmd6Sik574Sd8c3TTo6gYIU2Tf6TCkO0ug6uz+Rrsh+
OVYoh1wKyyehRYssbdRRqovjTKgVrSFF1V/OeHxIgZvzOqDDat63HZraVSMCJ3
7wVN7OvOb2X4bE4uBYMQ5sjT3U3boT04/b9a56xpCU8V8EMgBOO0QC4gTfh9DE
G6UwrfMKCuFnt1IZnab71a8rvEsvi292ADqelWDczqXOzGMWoWOETiKIoWE+jP
+DGYKCgeGSuOP0We8ATrz8kMG+ooV5ZKPQcF0jdbvfzZWf9wvF96ButE3+Ipp/
EVka3aA6o1DJhqgh8zdfjb8GzDCaHs+eAcbGJkYB1oX8zeFGHpnYXXDbeK5765
vtZvNDiOnfaOrpvI7dgrInwh+h0di5vrrFU+wyOZd1w8LFMJgr10zW+u9Yt3I3
sBHu42xwYd9MgMTWctK+M/aCnGtsA1zJjLDm6vxDG7EporRe2iIHEqB4iocL1H
rHt9yVWX13Lw1A/BMMTjTvhFrOSkOzt+SFiWfKZOhpl8XqN3bnlS6EnT/rjRnn
8YVfSgYH7a35lPS+JskzvZ3lD/SQ7q3ocRSIglflytWhMwwZi9qvuvGcLgnIf9
pWfZmLoriAZLTFzVw63SC0v+ACF+1sL5/tGefFZ+12XI+LbxAyN+d0cZsT624M
q15r+eM6MNqAlauFDBP/a7WdriRagRmVA0Hf2pwn+EqP8x3h9SeFmbMb5QuzSp
s6fa4IC8IXmqaTE+c1kHCyVr1sfqCmBBdmtbAeDm8/ddVtD5kbwurL4Ucd3xEQ
BkJZSyCCa7dcIVxsdpzrpHWKmDxxYar26+Irp8waNGutJxq+AjoqudVddONWpz
lg1ecJvl3hDErt3EHp+jwVOrHg/AahO7NxbDPtD3dVf+pGlfiAT8NzF6mTLjuA
v40T27Jsw4iyQ/FEXkha9flIsSxcCQ/VMhx5Na9GAiHNCZZyBywuFzgjOhBoWT
yD9lEcJx8ZklSamD699HzJiYzg7d6Q0IG0Yd0mQs+3efnWxj827NJjE/zRQjSi
4nxGZeQD/Ku8h8ejgUcMVZhgU0GUTcAJKYpZ/kxG8vIIkqtzDA1v0DExI7PcO0
RQT106CWS0oiLPE8DVzTAQWHKWRuFIXmHkrIqMCUBlQW+TBiYu0zIUr2qTOuQK
sJdMcNldFkdi436T8v5F8UcRvymeSOGIKoV7kfkx+HjJ4D2Wi/0iNX3r4EsKX9
ZemNCXc8FHuj8dXsaSExUPKnzy+umZFLmmvyRlL+eSfdT1qXyr+0XYYS8u7O9A
rc8k3KRtOalM8YoKD/z/EHVKcRLN95xTVNIxqyqRT6u2F/QTacx0GJyC1aqoEZ
EQqewiIOMKat1GucQ294NyIrS30lm9QnGwWQOF2HmC0Fgwu7omn2saSytpD0hD
e637eSZibjqdwf5LyBPkjYkWpbe0o45XGD7OcfcWIPuhclawO5Y+4ZMhebH5Ks
W/qdfKgDEQMWJ/+hHpYyBH3FK4yJ3pLLxGwNApjGnCHX21/fsONIb8PLvRZHyH
NPEAoTFkWdMp+i016/EqAWn9BoXucpJBiGACoUtpUCh9yqwGyQkliCWhaxfxAm
TvS/5nAwBfYkqbANmmNoAFkdZgwGalf10q7pj4JxJc4yoP9YFSao7PeS1d7DP5
DLWC8EbC/SgNvivO8d7ojcC7n5A1qfAQ4I2W9tfexj7SHjzAHgTHdLqWra27K1
9rInCJobCc+Rjji/6g5YQZ1oMRkjElrlIVAkDY9cCRBmF5Iajo+pSsjY2v9kcI
zKlMty1a0evZ7fx4diI7ooFlyh9IDzN2QjeIso90Eo/KyxtlaJzEHYmzLvI3pB
iEH0Ha/JA/nIElY67304cP5aXTxJI965DhDcQIrrak71UISr9Cxi1CWzmTNIs1
YMW0O3syYzaZ5rN4IsFFxwhkG4Z2uk+ZE8fLqzEG7K+fqJ3T2frbYWShGGt/6Z
fhbfMv2pV/c0IIikz8nb+i/S2jvfFLf6PsyTisHFTH9bDwmbnY+pW/MX44aFtR
UvwbknqFtIuT3NZq3//NP6V/ywFxYr3wLTkRz12Z9sT/0u3cl4MvtMECRf1Ctj
TCDL10FaTz0y1HS8EJT3gL2qXTP64X5T+o2gUW4UdAIzb+KiYsT1WpNf0Svixd
lRomA8Z19UdOIxMADkvMThzL5tLSIGrHXu9dsc74MDeUWYa9M5t9zQ0xG9ntte
uq9BJXQLJG2qT1VaQbmbmrespkFS8fXfFgZnzp358vZgeGeY979/cyPJj98kgY
QHjzoVgOTng+Ht6Hf8KXBx5fnP1FmAoUa3A2pprlQG5xrj5TYx4NMcDfmtG9pP
kNznQthwrqhzPZEp+nb/CsZvyjhnhC5RqTGv7nN7OnnKEVlcJnkiEV8LfWVZuM
MX/EnMKwr2TRx5q8lKkG5XvFfQXbCqc7dlqgHlTseGpE/rOk7ynVv35BPrwcMs
qhmNixncBOU1xsMXBwcJ/IUgQeRe6FfCJIq9m4ga0kis5XwueaXpBblsB9wSXB
p2eV/baHlJvL0cZxezipVX05EnxSIamh7whtorngxKPrBv+ZHncLJ9fkHamUlz
inrfDsziRvcY7vgGXXC3nnYhscnapXBcQxK9Tsky4LycJOnVPwgftfQeiP+D9N
```

rVW86mdGb4LZYONNU+I4S9XPzWvmlxhI/RwitgP15G4fuBo00J1v/dz6laq73f
4I8ygfFzTH0cvxLH+JIUXY3P5LKvBYTlOPg48r97HgYNJdy4xr40CtVviArnfV
76MPkYLoulSjYytM1+MVgcS3c6RcR+VDHOQ1lCqvzc76p/jb14fTxvbc6ldzQv
tGpM/71LzW2C81K/lFxbxvjGPFKcVgAn/3eNjwSyKJRqz5Gelas+p7THJ+pXLF
rxemNLKAmZj6prx0ZAlH81ZIMGjCwLZqvpSKA8VSIgf8sXQAdRBRRK0fEZ/O7Y
P6hL8lMNvqIjzcV9cVljlm9FVrZTIvOUrqxR457t1Wu+Iae2kMzmgiuiE2TldS
enrhbHfaPTMXEpU8HYVOOs6drssqy2mvVj0K4jYDkWxRkAxOub083nRqSSaynq
GjFrK2ifqy4eWVE0G/l5MH8FjYhqbZP2tB0UDjIO1PgyqK5km8Rf4u3PJmC3Gn
Pp65O6KCxtoCb9oLN7UuxVsssLvGz496g6oCMF7/Mt+OC/cowml4pZn3sUNKiZ
g4khV/Abee5+XCoUShpWECVfaBDNoeIrYj9LIkL8Cv8mRyJM4w8iwyxczfDSj0
9nl0cIJ7CHSNV4+Gs5ssVH11U/tdBzqqwrIER0k+g1cAnvqoidBl60e1hphfGP
T5Mwb72kFuFKC34h3goUoN4Fi06Q+UpAtW0h/Y6Ddw9pXozhSxsCNkBWJ7NY7x
rLLxPXpmS4NQlQJJWw9FVlgmJCvDVwrjGIR7lIZurQl2snQwa1NjhSRt+TViBo
zLRPQ+eWURfMOAySGcjuvNL6IcfsQ2GYYQDOQG+cqYXsQOO4+EIGUnF0VBTZws
OAggEmbg1Gz1HuJaannf9KIbvQQq8lUc/1CTbmtfwMn4NXymHjYtLm+5sV9e4B
+ADf2J0j170fGn+B6t7pgOFHbK6HLFtSUwhR3TjsEGLSaOxWLbAKZiq+U/BKh+
TPufokqyX1fpYrLipUOi8vSkA7TZD8jIwX/5eaIoCBoSCQPua1pnO+XBeQaXQc
shwGWloJHH9X6UDHgOvpUL9/eBJpTTwgtZG7JG1oSu5TA5LF3mH3BlVUhtQE/g
YJSXy0SZjRzLynUCHlZcD+qTIX4a3/Id4oBOyEOL14A1ycGrVsuggs/gMig5pI
F5RbiWXjG60g7oG1zUKMMkaZ1E/FotmQvZ1pjNoZcoUEc7oQTT2LnJZlvNihQR
IRqPIg0L6mVYpQaTF8m4tRfEovH8AaA1//oYuZrNqPOS04mssfZYp6cMgqcNLo
teWeOUW7d2lWMO3oKW72Tl5bv+dS4euQuTC7bxaAOV/jpXunQGgkhwW+BYUchh
qXk+r8aTN28JbnYnV7K0wiX2vK84TMAk8nn65+ZuhkRC4a2ZdIYI4u/W1t0li6
V9B7L00QnzVsu6mPxN2kF9B7W/9JjvZ6oKC339Hhbk3IM5yCiFHWyKUGXz8m26
tFBnY8vslq1pbf9bcerPTfnHEbzi/3UMJjyPvymIklAGSqgEnEgUtxSl4tUVkc
2T+TEep9fyny8zZOyoITr+xmPl777MofJXhD6Z2/8rcn7mSAqxhZmzDxRYkhti
mlDrTZojUYgISuyJ3yKkLQ9Zm/UR/FxSOJE0n85Is5szuPhOGc7myE8WY2e/Pp
658X78/v0lJ+UcdlnK4rucQWt0FGL/DoY1CKHxzCwq2ir+oHf45Ua2D4v02eeh
9RMQt2vz0pfPYT6ITMOnoXxeA9SSq22nCG7wJY1LQqGloc3+/C/51GptSy7Ea+
Cjf/H4T7EuaXM/RaY9nTx+B29mb1q+WZkB+/bO8IaCGDgMggYeX2B0wN9fiMOt
/lEd0v+zMSnHx7DhDAkZechapX04lSjIj4QIOiFf/SXC2NeqbUUcbumCuxpq9K
J3xX6MMwNsWfUTZO6TIYyppFMObrv0tf8AypQz/USEfQh1PUvdetSH93Fa3AaF
Jb5r6nj4XnGDA10YXuKKOanvaFyVGvhBNMUEenVbllTQAmJ3/gV5KcAhXsgNpH
zpStMx8aGff4Zb7Vycuyo97gBLkIjJ5cM4cuc/1axSU7/cR/V1vidCb3We/uYl
o2lkVh9+PC2xWb/hwuA2OFYJ4UTM9zOIeETsA6NV1W0dP//O2yFJsqclY87HRI
XxVyT7d3N7bra7M+lPMdo6pP06m5dN0wjyUdtwZl2bXl8XF/B5xucqTXcn8fT9
IENQd0K8+hqZE5jjxtCpL8kv9sBJAY6YsN9DA08rYwLOyBWzCAYPT6JMvLVMjB
++d/iezKUhMhGx1qjCbt0qwovKivBP7vHHT9tLFgDK4DfyrfJYmczUkhD98Nwh
XpCKJlh2NlfunGoYYgnT0NkSkKvjGFVWH3d2pu9vudV8ztqMsm6Owt+3GkpI4A
AX5zle29da4Zlo7TDV6rlrhLJZZFP6A0GoOYT9eWmwSKFMOLxO2ZtQ9n9mrK+Z
rXX6py5OKo5zb50Ye3WYDv1E8kpye4Rym8x/UhYKbpgDxEeu/9a/LAWkd3TG6f
+u7QF2E2WeyBqUbAptl3fsfGaR3DMK8mo5BhQnwnICLJUSqCumCBQuCZqAVohE
KG/PyXXhNkpvODAT+T8hlSHeshh96J4c6sOvlWL9xddW7P8kpSQWs7Mcy9wVBT
3JsZuVstUtDnXnlC3GHU5rXxOfkY+1ylO3AoHvWJdVzVyPO0IjVA4wXQIjeZ8m

J2ak0I4lzsrDhBG7hoiLMMjq2LE20DBbmzDcU5XsyQxt3SBU7/rWeX+D+7dsW4
kpVVokk3TLQgCrLTJl7wvUuZ5dLQ/NymlYeD6w/K+y/RSH2QwrdZkcdVD4udR2
un12fm7/QsUafKY0R6XqOl6/StvykAwRIMhcr8hjvG6E4VpGAEV7LZ4BmBTzNs
t2xQiDORJl3m7YArm+1AMxPGKqtQFrxco12KB5BMnPaMT4cYb6+hun4Y8/b+Dz
CmXDeiJUF1OWu5kcJieJu+lCmWt6UcK4Vb9VWc9gclj9K2OiRpiMEYVlu2kbsv
fn5DoZl/9uY6xjDf/HcL0hsPXxnmGYRPcxajGe1ugrZ1q4zhTToggRNnfhG7nC
LEAA54tgqezCA+PvQJX6B8o7r+IzG1/eN9m1s9G/7Me49cd3T60aDF8VEKDdX8
w7+7Lplf1XZ9jj/j5iKXvNqekxqjDmndWsI7tZK8kBhYndsxp22jGgA1BeyWu6
8s4appvsl/t6snOZYCCPHWbzY3FtaP8QXLZK2Y/rDmWYqt+ZkiaMUoLmempvUP
HoaBj6o9kUkGMQ+i8y/dEMwdHROB1dvx1l4uFh0NY3C+bz8wEcFo7nflcLopZr
l8E1/0IU9UL8y8f9bmJH+udCAZfEqxiWguvPf7pr8n8ppHGKo/kf/IV7dbtbIk
eiq8pLf3a5wEAF5nOrM5/fABqZ1dVgoJhHMwyim269NJttUF7qM8PpuBbVAgNb
vsaQA7QR2hpqXlJFT338OoszRq+I0Vuc6/1aS9mRts+0dI88RO5hFZhMRUVrD9
2ZQccwoQhRL8SffAs0P/rH3j2uHbQV3bjkSGhl7XkGnuBeXSRoTbfqsovAxTRO
RjkxD1MMXAoromT3nT8pxvAgHBM5zhArZUSbpxzlf83JmIJxAhQi5BCW494wL7
6AiPr+DBFxIuRPzU52mign8qzPkUO6YzuMjNY6ZUTY7/ZJXbot4vkfVfo//Ken
Anl4EM4LnJr2/2MVjqwK7+nbXedoyxFsxYQNUbkpnh61wyeTcI2Y+9uuEa3r2b
mUI3jJqerwQYsN9aub+UDoXuzDQd8XwzG6UZw9d5gB/Pr1BDrurHULjXLStMHv
KTKq27ZDwcC9Oi/OUOzYpmPJ75TfMVvEZjBFSYGCkPUKvY6Jt+vacaZuz2640i
JH2lQPp1/jmSNTgP7ir+tx9kuHGD8xWf16Ag3P+e0CHQ/pfmvoX4eaIvcx+sgt
znWJdu+G6XUWnI6upP6Ejl6DyXhTh6epZCtk26czRxqGjl2LxGno0slNTDxACU
HnEUjjGp/z4sign0RETLkmNMYdR40sfJVOE+PMpwQZHtA8pEftxBs/3KBOEGnG
zkOFHy4Euo7kdU7lUKeLso4CHAIz9CEQAh4gFpt7+IKeP3PoMJPj1bO4YrhDgS
54Haw6eakrctRqMPU4xtPKCMpnvg/ppFKyX6zPCa7UHl3J2ruHM0wNUFJZvRlZ
dZ7ZwD0r2TPb2YOYOMfHGNb+dJzurqpz51qITkiG2FTOy91qqq8PlcBhMzXWQB
nVbduuo+YRceeUFbq9xlfZvNs35juNrQW7VQMqvvTiEQtaM0G6OM0Nau8/uJwc
MxGZQ9FqUPoCLhrg3h5kUZivoFZCGAEGX/Aj1JvukbJ4R3MbVYBw2XD52yObvL
qZkuInKocP5dATx6/p2oP/jssMQ56gXkUNhDAXHkhR4JIj4MHWRNuUNone1/7d
1b/9gk+VlMwEhdPWKLZuCeUH7917hAGmKS0kBiNhhcvDgItrVPyYnL3ciWl463
kMa0xYjIUpsNVp+DM/7qrW6/Gcnz6/ZU4EXrypQJirTp43ZwKzi1kg0DhXUQp1
cSej02xi7W6FzMAtbnLI9yZWWyxESiup+v+azQc5WlRrFB8XGeV8ntbDpKZ58l
y6oxFS73PDdrnsRV45zeDPC/ohpHk5r1/RYQ2rgZ3KFkRFC1BeG4dPc4VLSO/J
RIIO8n5B9iK8nyEER6i1Bo0fsTUscCvOaX/n7XYOvjUwrJXopzFQaxEcSTgS95
364BqZ4hlvb3/b2Vhdinz8fhKT9yl4QKXo0+PTj4o2UMCN614cyT5iXOg5TsE9
K6Kb//Q3dBqicjr9DSfwXxs8OF2xagfKxAvLH1R88sZZbYe+/H4hjiToXlF0Uz
jo0S9gMl7SEeljTdopyYIdnQaBcKgXdYYtWqhYqAhb5ERt2ASVJdCucMCSrKSd
Ptb2yFKcjg6ODzLx8HBo1+6U6ZTpvPeaYwPt3A3V4oiterDzKxpu+Kl6uHpVuC
Euu2Z0QU2ldXtQC46E3MScRTNQLUac/tXqzp+p+ICrNacV+uZmSuG+jv1eKLkW
j0pjqc5cmNGQ0QiFR+BImWLL1iwwcNs7veSN5rmWYt20zkr6q3vCLSkFQyc4IJ
Uap6EdNXMUvLl5b+/rtxZQswaQGkmTNJp2QsgBUF8CJIPZDCbHmU4TLP+jqQaS
SikwRwBRUFQ7I+JivYlEyJQJJpeKGGMkCoz0/fV0X0pDS0MKkvohU8bEWv0vfX
1wG4WBMnlDkAg7gIiOpuxr/8FAliXsWcrZtRjmQIRTjx6dIkAcJCUKj0JeyBSL
4xSHckvT6s2JStp0lqyfhPISfo1V6EiPBVKrXGryS6iOKB1407okE67seajhjQ
hEM/Kli6SH8TW3sHZoAaEMYS2jSI8UE4oJEOJvCSqoXdTbYIyr+nZTToEpMb3g

C2l4OE+vZeK78uuykmjdv+SvMIR9WF9sd1LU640RLneGyiVR+1GT/PTpHTy+vJ
oKJygrE+3FlyuS+urvG83dx2vkJr1bJj5PeoGUH4GsSCFarbnxl0qhn0rhLymQ
QDD1q30irrsSF1SzLdV56bR+37zXKio2+iOLZwNb6KcehkEfqvlJvN+JWW83UL
onvy8OaXXRCxuLuZParVuoWSlpZpKTrtr3dtoEV8ohQb/ixxzelQ2sYinWVW6g
LSv1fbr3LyXzY0r03NIFN6i4RW/CTUk08V8L8EwS3Ips6itUSLsSO6W+1wjBWi
sxIbYd9+rzcaSGFRoZ0d7YXZeQOFXjdfxx8kYcdlsXzD8nwMY1hi6ODNav5KLr
o73qDF8JJWWis8N4p5nBKwvIKagErbnUEmtmWIUpLsIUVx7GItZE1a0eBjICNJ
UwuWYYUdNjpcYgy6S/TApXfCETRsr423GEll4xqeQLm9YxA49Ar5EhZBrsWdSW
yzkmstoiD/iAkRELTp1rbXEhhxUjSOQC3yhbyi4RWLBAqnKjZaEIi0VcBMRW9d
6EgnigiLBaRlbgOOV4/YK5ik0cnySEDosZOKV+OqPeJCP/X2GnmH5LSutKzI2K
kDRI/Yel2D+EMXoCK537we0GU7RAgCfNzXC6PjdRilO/aiKPsvR7IATPf/XM7j
jrwo12nHK8pAml4uwv4qzxtkp9iwWnzDU33aCg4wtJCO5FDkVc4J3bcsKBrx8l
Q0geF3ctrH+6B4G3xVqbrI7Ize7T3XQGXNkvgSYmDHezUGp5NUYqa925Q3mXFV
S5kxu24ooqr6Qlgxevqx72JPYV4QnmZpqz2z1tDL7gfJ5mOnLtWPexa0dM+i0U
6etUHYRguNOodNd0Lbs636le88Dw/usu+9tvF9Fyx6zSerX7Rf7QcqUp3eShbA
seORVR/d/fjXVtnZVUtvSFlf7FiLMn3or2ol56LEfjfCXtq41qPKEqwxJWulZD
GQ/Arf7chQqYMq7HqMvIpQUFOG2ud4EXU5dwmEwKICitE89+8fVDHoXX4OrPLx
Ya6eesijeJ9JCcZUujrokCqTS5UVosfcjCzorQhtfG+Cj1NVxDjVBcJOZNnMlI
f6WUVrBkclsho47H1+oqHpxBpr0s/O21+NqNYCIIwu77BXYTA2S4QiVwBxTPiZ
cK3Ybx1WlRRPnvj1UFSODrZYRyAnLsJAKax28HsAOKd0NJbALJh7NgK01V7kco
p0/fbWhgm0kY5Bav2ZecGa6gd33AUL9wQSQTR0bPzuXY2hHcKoxLXJGsaw/oNE
RoVwTo2pMTV8QJrTjluKd1joprort07QSo7hsIEolwZOHZuTykP3E5Q8L1y6p/
Xw+iuAqOlmAgkYm8C8VcetSAr7lBt3wpKL4QNLNYqmXAyccrUxXWhHrXmU6WKQ
v5Z/mpOHmwsBql4GIxTCh7Brl9DJGLEnDk67ky9yAZrzz2TnCCwWkpyMUpx6sa
UCruFyKG4KQgVSh8o7TIqiI52aZIdbpr+4On+KHsahCzdFqG4khuLf+QoDpqaT
Ixvq/jP09K1yzMUCSEG77p+CiqQ4OTXV8U6x81fwsrjec4ARlifRlfovAXZwdR
zEgkCaYfJFDeeUe7l8eAsMGXBw+HtualaRamo5BUigDOFRi5dEk+w2eiAQOH9X
kGd4j+17foZqsIflr1Dn0HX6P4aj0YWgeqia1p3Pp+0mG9nMOAIMDgcBNcJMAr
MwXkgDoU82hFg3cLzojBSrbURwyxiEQgPGYOvg4KFAXgujn4mBAQiSxI7CNjg5
WCOuTNL7aEjHz6KdbJ429BMr+ViweerNn1svVlJaR5ic/Yngz4xyhZaD2nitSH
qLorgVtJLz9cil96ag2cehRGPsk/dl35YpVQoTgr021RcFMsdRKZT36xf35Fri
CnyDk6JN+pGZT9TAwJ3sqC4rUeMmESigmQEUjqiUCBdWlZDiFUnq8Mz16cKRQ/
F/pkTTqBBXsRh3972a8VcwE4JAwMUElDWHRUOJbBcwqD8175IVhyZ0nIuNA/Oi
b064qMJ06m37YD655PfhOZI6bEI/R3KP+PmL+XKR4oE0+0sV54MxXfGx8qAr4N
+PIDSm6rISiQoeYReR4B38NrbeXzQCOPj+xfubKgdr+XIw2xTPsl8GLVnLfZfr
0ByE+wKEfoc3mfa99lSr4iDiegjMj1bB+YmcR4Q7gdr/LnHEMeJkWJK7+gBRHh
l3IF+qytrHFZ4hdRRI50WyR7FjUiwXOL6o9T8GiZunRLt6pUj/cSub4/wz7jIk
/Lj093UCAHZCQHXCPVXrwUwcCDcEYeg/9OwPcpRZSMaYTbgGEEnACFCgFBh+d8
fzCyK7ILCgvDX20gK6MHYwYhG+WCr7+Ewotpa+to7+jo7IZUZEGd6xttR0eYbn
tnp+zkRQGfIOR1fFPo5AkJDF88KRNpZde4ttYVHAIyK9dmQRD8+x6JQWrJb4zM
Dc8+lN9TafXlaCBo7vHWh0Rd+umebP62eNmAo+zXv+uydrlsAiKXsPWQ7cQeI6
eDHuk2M0pc+dyK1+Dtv+78hfNxt6QIbZxLeAQRDu8zb9UuukG20bbAta4EZLGf
JNzunG+Iy4jaswb1KRWh4e2uOxOd4noLcUNpW01p9MfNcjFbLJoXCwnkr7Hu2C
zf/IxvFLRiFmrWcvORxsjeCDdiQWR+RKj/J5oeSa3yvNOi13Dd35dVN3hruZPa

LVuYxKy0jCyH0Jw+weVbjFROeXR6e4MtJiqkjHZTslZeHsrSyRy6wyS+gpyZlq
H+GQSSKsvHsprJvfSPiIkHW+9IZDPKGFWdjBp3W0zg9E9Sl/NpBT+EnA0N2xAY
9qxPYyd41sjjxxrq69ZC9h/eCVofHnw5POhIaOAzmvyCtiyD4M0Q/Dsy7EntM3
tGtPYTISEHblw9eY3LinBrzkUDEsQq2HE9J3u3m1Aw5l4/bHwzMHW+peSHeEjn
zqCFvw5Dph5igyTcYCGH8Nl3NS6SeOrKetGy1SGl6kz3gc84wohMovubO0TI92
d0reVUoppd9KTK+H1Uq7NbrqG19xg0jbzb2RpFzsoFlelUSwY7cz8JD0A3Qq37
Pjeurjrqdi3OS9C1tLz1JkVJ6tqwaEF8ecqylxdaE7u7TYmN8yy01GxGE0Szqh
bmiRw7TcXYs/RatNxGSPp1GW3jgpb5uvPkrFtvcbAkJwbohX/8YQ2ZHRC8LR/f
lSLZvqZdJmTleb/1Pv5jU/aA9MDAeGD2v5d3awxpW+y33ZcfGW+KM0MgPjA90B
4wNYpIDPH25ExqFMn/SEiFNYlPeDO/1kKKosIkIbzPokMcVfLmOzKqGdXzWap5
VUxLP5KgQ0L9yWuK83B6/f8EEqTs5ZEDiddXhvgA7caQfzxS4kkCKYjTEiWgE2
iF6A/WIWNM2CocOX9a6+Oju6Rs26S2n965S96lgjs2K+2jR/NyNm0MSe++23im
FXzZCigLOWEZB4radHv6mPvXiNq3aAYdxcIsBZWel3uUKe7Y9lHxL+X8nMl+8z
q/seXSKaViagNHWuvMXQN3P8AVFxX4/bjS/NG2iuNkvAT+vFyVeQuIBa4V6YGf
l8IlOB1dsTNSiP9a+vvpV7mOq+lXroXjSMw1yjWoH0L2iyGk2iLFfjz6ajSWRv
CsNX78Ni43yvFHzDPz5Bs0nECjBB9UHhs9vmdaeTCYQiPgAj8NVaKE8cQ88DP+
4VLy1ED+Y0loRHumEgJQLhXRtnrGBiBnhT5902EI5Ypm5A67JHBKKoVICCMv25
WLmLqK21cuGKyLt4GNvasWE6HhjRChnFx7lpHvb+AXm9TQrKJMQ0ZXm7haDTcf
1PRW+B+SmwpcCZwDEfupQidPP2xqJ2aSt06swlmu6moWvmpiK5mYaWof1vOcQu
r+iANIle6VI5hVTk3XVtLSkzPaM9KTV9KuNRW76OYkie7/CTN63uRGR0E779GQ
pXjuFehPiHZvJ0Qb+DpZo9/6UW+QoLrR6cUsdEa+pOp9ioWENcxzkWIidOlSy+
+zDRzCxM+hBZADQyiW96uWaCdQ1uLp0ToIIY72JI93T5GgmJhVOFIAA/mHXAJO
ANJNxTzXRRAIkQftxtI3TTXZrQ3mxW8YeO78Tp+W0iRaDZEliKAZYjLbQg2DZC
gQXHTxijdJAQEncOUfAoLXK9/LY1GI+oA+39L6ZaqzLWdVy+p9SwP6iHoKi7dX
7vUif0XhUT+UZSP463n7TwH8oA/lxpBul6vn5IoPHpApU1t75UhEV2//GQ/aqZ
Iq+aI2jhG5ueanQz++wWBeCLWtCqGDHQL1VzRFnrtvSK9efcskXD9+K7+K7TP+
OQl44bXCVLkU9g9/JuDKU4VrYS8gfT7u64/Gov6+4x8jEA0zIFKFl9LGOj9ho7
LZVNsE6zylzcuAmP7kLEEtP5uHfDyOpD7w1flSrn/THmRYmqOFEbpOJoVaRObI
dHQE1uYsvSD007YAZBH03VxevEs+6Sv8LopAfuBioix24pyGA8G+HxXi1YuWFn
7kC0MczbkZH19ZotzP54FPQja/5C25CR8/yqZjqJ8E/fuHN+1HXaBWTgutereP
P0ohHtApYMbQ0dgvt2mQiZFElMFl8Kd+qUxEWWd/v2bWMZgQg/VGFPfDwy+DBh
7qBdm+pgacCLW+Q2O0QsdZpF402qWH7QQWGoFwDKdNl+WrIMKqJRkqZTWvCJUe
Cu/6W04k05lO3cMfsms5CF/uoXC5anOC5RhV9ZlDh32dtCPOJ04cOxOhDUj5nT
/cGPY3j2H1070P/joHvUMifkGwkzPD35gXdY7LUvPNIWKjNkLr4Z1xb3cSSBra
kr8Wvjx/wJXrLPYxXzbTd9JjNRG+yOdiuQpG6Zoqpl3omwRF2MvYmrm4vKbIUQ
/t+gUBgvHGeMIVvqyzs3bUwEkUzUuwG89Wh+PqS/4N1QVmerne9P63c1/Uo3C1
PCMNHHsv+E/fzHI1HeLGPtTxLvxz+WiXYiogFFSe09BS/vURyRo4lqjsUIM7f6
tsRQCJQYEdaYLEHDOok7uc8qi15wV9t37RR4a8zU3/DTJUcRIwQQA89ex7FyWc
ZWSngdiZPLOPOSuAJwpO86lQrJn1tPrULgdrd6mUvGoOv11lD7C9YOvv++nt3d
7yitRppLwpPABruZVkXf0SF2u7Rs8hkBC9f1ZFd2St57iFfozAYZFCki0DMTHD
USTfpdkNLbHukCJ9UfT/DTJlgbB4XVdD5GSE6QAufOpHV1udqAw0tDnFNpC4FP
mRQsqHhyAiIFjg0vwAdSDs6dwsCQ4SuRcdHH7sL37mz9ql+Og+S0ucKvBPlrDc
2hCtm4PJtet9okNZZ+duhMempSfmx5l6WuySlSTSNUSgNcZUF+wikmty2/GJeQ
jxktuI7vn9o6xoGunTJkgiHaAZTKMExmUysUEbT0aWfdb9TUDaUE2ihrLlIZk+

Z4BIEOy/NWUhCb3jZjxLSwLpoVX14DjQvFQBHweOVZmVNEoiC9bjUeHPIPglrJ
gK6ysnfSPXOjOopPxiRYzMJF/c0teooK5zllDJLdFKFNa6hUOilVnJd0IkurXZ
5TRSPsqG9bhb/Q0EZ9VHikIH/mlfyvfBPjOT9Zlp8mqYVyNnx+xAqfzNEYmDAp
b1pCZjN5fbpUhPuBjqrwPGNnE6KrTuBLuaivitrp1OVOsyYLndPQWCmyzmzQJB
dw+Wq8+VY86dLtSVYIn9aycAsrX0pWy+JRrL5vZ28LIgIazUynKHcrNF2YvbnC
gh2sJnL6WzvFWKFPt2UFZ+EhV0gbLy7an2SoUX6P9GS3buBGGEWAPfrYXoM5wA
ucuA5nQudvP4ZAgholyjzMBx8zoXozmGGn6v1g+v2LUDFABmRrOVKhWpFstL+e
0C+UJlBStsT7FXIS9rSAed4EQXt4myVRxaYxIvq7cjm4tFW0ASYQJpHQTuQyCM
kGxnZ4/Y9Z37+AB95IDsFkW2MLqkTZjdTaMuTp4NwVcrP84NlVW60ZK2LBRjTJ
HvHar9M3H4FvUNCvWTEnMXLFpQPjncceGQNTndML1ufLCewQJMxKX5cq5HChMI
BJ0qX2E/4xpyaXJRjLaAlVGps4+RqEH7rDKTJzP+b5Bc9U1EPvRO5bZhTTWZ+R
OTvrguglCSFUW5vKRRbP777Ik4PkwqzLn4XWZCvcUgcyEoF9mlSyvPYC2gYWiu
xjXkOqOwA/0U8wWDXC37eq5bCsNEgk6dr7D1uYUMNdkJZ8iU7++jgz+IQ5vqwS
mOoOKiZ2zZlJbChBjElG/oH3LU+99TyAdzEmpSiq3fPzTM59we61ld52NewIEh
0ug+TNAFjOf/+jo/HRflXM5OjEG2Scf+f9dggUpdPID6SUBvo7K5bQcho7YFkC
KyNL+ckQlzruYkxMF+vuhAsbROh+fge867GBHhQUa5S77B4RhvNWtRvqU+sFEQ
av968ywolf0seHnzbyE2QWNgvYVPYnW1ESil1ls3QjyEyJ2Rfy+NBmdnweavQ+
0VdcJPyVASbGObMuRDXB2RROK9y/nuOSgAuQLDt8+1ezESiajjDslzYFRp62KR
Otz2Q2y/bX75EKjnC4B+rq04wCsnhob+tKSDK7DFQcjo4w3b7CBNJ+CjxU2g6X
5Nqus5HK1LtvbYs0bQfNhidXUAx6I99FFHoYDE7NqYvmqNfd0rtwPVgUGmJuGJ
kehS8NE72P6l25upyoCb8dufgO1RnEYOv13oJ4MUuoprHWPvlkEBmPIH+6X9VY
bXyhjMzLT33y75QYkFQGXvjnUcfQTJ/ITtILWR0+Kwg20bHu8RijxabkeWoktP
jDQJTUGE4vZaRUMDF9/YxSQJCh2j9D2LCunnDCe0qQl8MreG49JyOOsdG+43NY
ImcHQXUHED5DqDk0BEczrTcZ/JYyjRgB82WXPWW7bBGSqri+ayqjLgbZYbSEA/
yLTB1Dk2WUq22yE8vyiKyF35ZFKcBc0yuUL7XvKLKOMrn79SwtIvSn5Pqxju+7
Ro+hU8loeRaO/JQIV8ckVQUctwobSyd0WmLf58cVrkruxUoyWocHL48ZEGXDId
SiQQeZhLNsh5BS+aLm1XTJ88mGOaNKpPmfUxOWNnQMG/WgLRwy0C5Ufftmp1uQ
Iur4BuqNJtugxWccCqy7e0b8npBTyuIFenPW+rPNWINlaeOm+7j3RPdq/Ex/tt
S/mxUvdxkyQq5rRsduXsG/vzCkO7Z1cCsJn5E5PU3nEvIDe8rjCoXJm5ZQZjKO
kixUZzD5PVE7XJMeb46MXYKCRzqAtt0/aXwIIPY043ghPU9SHh1lR96shZEuXs
90K5RCr8V0BN4po9+RkIO7i1jtWxbIGOYwp8yVnsKumjOXkLJ402+X5RQ4EhO8
lUYSnR/9D9pLfG5F/4OzRgygN9/2Mungs4i0lCLI87AbbO35YgomPt6RNR6goD
6ToG+PGcBR9+qBWCnf8xcCayS5BFxGBPedDmxSOJ+9MUSGLQM0IOBw5JxjyqJe
NkuoaB7NrF9ShelYy0eHhdX++ojNYHPkjOyMhYGLHMExo0bJ1cU0GosMLq9vA1
ufN9llOqgLTd2wh50FO2byxMJECQui18HYjf4TOZNOlTcqD0wDm31TweO+62nn
OXHNgU9TyyoPaMsQeGvtyqBpV/j7xxdgUNCuj3bYlCwW2gK5F2x0vpaEsdZu1l
wL/+9lTgr+BXmNHrX1L5x33jUCdol77Zt8RTz9o+5Ft5EHTYDd5uTyvLnZ3nZn
ZIeQJ/Zqc7t7RokNK/sBE08HkCY3snqOw4noa21GOWVnl/efD4671UB2ijnjTo
fkBDEZutY5rOCVxfj5+yz0DW4mFTgJaK8nnIK72yC84y+gBsAl/TgJxXeIPQ+Z
cvD5L5czz/Yujx8+c9wa5iP95xPtLve341zyvg3FXf5VB5i4uQwr7Mx34H1Hvy
qpquxTXl1QvzWS/41Vf/bhxrj3E9Vm3Jy9uS+6G4XfT5NQFJN2NYMJMEQ/Lx1x
Q9gGsGjVcaLkP2IhI5j0zNrdtS+xfNBJSmfs+5GMnIr6bYrYAzN1u2Mq26VEwN
UpIS2oU+A2VscxlCQmLMKW47c47HdUGBgWNKNrurIP278IKEzBzpdU1GSxXXXm
T0cgylhSaahly+CNHThEZnUmpigWRAIBxrYPrMgAenvCEH6hf9XJ9f4xWqJfb/

7LGccrJQImK5+7wldTnh93dnl057OPsqEKHzweY48A6eWVL4ghbS4vTXu+qhzA
+tGRbmV5yojOg/hr/cO4DFuwy7BPyqzHBVBFdXz3E3PVxyjmywF2M+zvqdSYB4
PF5gY9waF4dEvUqB4H00JBpYskd26h3/yn4A9Mt0eDPY0Fj8wQdfTPGwTcRP7x
M7BJx9M5AVM+vaiNhe8e074k0ofwqUs1mAiUBeFpM7wj5fNyKJUiflCmmbzch/
sY2e89G1Qc5/uT1AHw2RIBbwGaFB0bXruJaVUZtpwtwkdZ353uPXMwLaOEFbrk
g0YDAFgsTl3WEmXxva+6pb6IIgHDST2VlXtgQN8c5fIJVn6Zt99EtsEZ11pAvn
eWBtSZoFzM86XPjdywWC7PJahBmU3/Fki2Iet9YsckQmV4j3gxML/UnQACva39
beTinB3or6/CT/ljVnniYO6eqL4MvqpZJF+WkxjRvne+xw+ERbeMIZkOelXsVK
q1ZYKC+okenYN3PXn4cb3R5HD4N6kEkva1ibH+WbUep+xH/qahx1IFVDPcy1pl
dlcYjGyKJvd++UBkvTSlwU/dkooiCsJKjkTlgA3/hPWEEWkRFj7V9JZW8DeX9R
r6KiBUx9TYGNdA0atoUbj+huTU5fXEesy2HEZCdescE9DMppJi01ufqxKdSaE1
29Uxc49dXtc+a9zJjshCsEtUS/yPeXkHoEEUqDr8XFG2T1FSy05BrI8ww9ZLRd
xzLHHuiqGcLw7CaP22e4uTg6IySjRB7lDL+O+RDqmcSoq5L0FSpbMllSGj1n9A
moZFofM31Zh2l0vsZ/G7HcaPIbTiyJTgteVqqoUS24InYMwrNVrD46ehlov+Kz
YNHI7LI2d5foUnjEpQ0cKJfNjjAZEJOo15i0q+QDzQPYLT//WwBApuq16KI5rK
1bx3TStNNhI52C9jzWEsEsWlWxuh1KRYdPweHToM8xIymgtN65Rp4gxjCYUuQL
SWhHU7WsusE9vkNzSOtJMKCTsKZQ8cn2UM3ldsY1yhp0SxN7k+oi51mMfnyzpX
4fpYFPfWcTCeH5q0Z10Un+thIUKwbLRSw2kzES7dgJlth4dSM+R2CDZ39eJZeH
DhHEpLqmXJtzdbO553CJDlflaGREJpfBI59kTvK3ireLt5nryEWgzezEfYcMuU
VQ3sHkMBk95guEIAZSqicpGANiIuPGfMdBMqCDLrbG6oJ8DoJ/HUw5g1bCr883
ZyQtlQRcm/skTXm4hEZHGb56iu1idYdE6G6AKnuKo+ipXANd8TmW8eqDRj2okI
ogJsCND169gcRBXQtpwjiSygtwm5iSIHxd78tA6U1qvBhz+314gVALVPw9ctZU
G+Mrj8b7vvI0PsfACdBqwizcpiq+4WK4tITrGukuZPSfAMcNdGo++Y9yOEPVcr
vUfGfxxDHwZRNj5yqfBY6QkPoW89yuzHZOx5lDpo6dAK1GXkPtRUh4jprUcKto
6DvP6CcFYfkkBPMNaNVoE3zj68Yj6DhFkDTU0bNy+/n4+HSMb1OtxXGZy2kZtV
l5HqEhh2ezjqZhfLsN2EcvcgVdwGa320DDON6/eBJAfOtNEiLwD9wTHZsicZWi
/AI90Af2l3EEtosiI8iXyFoxsbgk/98iXwWGWLS5L1CfCgrC8hFSpVxFqQsNpA
MF/CcFAr5Nk58vXxwCg6LLKDYF+AIs612DQi5KDTmjngIE4p9giXFLyEiIgWT5
reJ/9J17IDxrT/3r4LnLuxn8qD2j6MGDnI00tYrG2XD48Bj7hX+dO+iAS3SdX5
Egg3tpk5toS0mdfl9ySDIZiTFWn/HRbLtcjlCXbBrtZVaLEZmcBMnaOqefS6UP
HZha+3JeW5SZ4iH1XangHdpCG0D6EFp/ZbKs0qzExc2ZkRvhH7c21F+/XjfR2I
DcV1iKlxgKrZa1ayOvWqH/VzcYqJ4NkFsVxrJygvcdTPe29WvXiLuzIhoWZSdO
dSIUHVqpUzeWF+Bo2PrTJ0+eZumTPvqoCcXaiXxFsq8fV9j23bS2NUXe8h07cm
R8cvbw5OjhV8anZl+2wNH/RGkfVKfoI4IXrATl2NTqnf7sNo09B/fQNDJPqP3t
t02bIimwTLSIFjaNP1sctObp+K4zqT4DHix0Khrtvr2gnk//ok3blumPYINcGM
yerOKtdHU39Vohn9JrHHZogdbZkcl8By8rNTczp+tJmwNCqusb45+/ByYlBv7x
W+6jfYY//1R5m3l//altO+ip/qC22jM0ZDSsYDMHHH/Lf1GYBeTAL8GTCILqBK
lYUEwaXu8Jtx6VU4nkNQh9ApouheDA/JBgiEC+TCFyNgx2GZM/yeMQKZfJBAhK
P6B6qWyCjqwhE0D8nn8+hKrfB1D1B/DVVXyqUZN3ISP8/s3FeRq3hoMIugUnUZ
IMOlh5bZFfZkTBxSoZSbAodjQJ+vAjuOoDiHG0rjAMOGINltfvXrX8daenEoon
ivekAQXjthDqt+AvAuC/z8+op95YfptIYHKEIglruaYz5gVekPz5sqpcvupt9g
VL9S03Dt7/Xn1m7JMChuaJ+jcALVQ+aSDNh4cz+95VcAWUNzPZbMbbsbbX9y29
m/zHYb+B5rveDTYClc1msr/q2PK1X63RVNvKfLU0Nu0rYCwJRJgyny/G9mg1dh
m4xmYoGKTA4m2nTytdesZl9fKhraAYFPIpKuhy8kfWmF6U2RO9cy2BT8SKf3J5

yyddLhCzdh/pmPA4qDkGwGOmKGkuEM2Wqufmt6EhjuiHz1yLf4avgnt7dUIekU
/YuTY61Ean2DpsV5Hp17QgCSx7UotteV1pXbhaIxwTFq4D/kzmi7wNm3elvLeg
QUZLJJNXB3pwJ2d15xtrO5dnjNKzkwqT6M70jRvWd376UDB0JzGzgrusAT9gR5
kYxTkVtPjTNXsPPBgJWco3r+Obgxc/8OvNPFwdsuTbAExxlhMoaRQdGzaUe8io
UBsUrBWiZE85bGDr5D/95IbF/UVMSfeRCC5CSUuwXMhNRXY9llgqzrH4h/wM5o
K1dc6RKFuu29p7uR/0TujNvtoKts7UsOm39+wJqc7EAbmY7kxOD5wfWa0fvRS6
AuN3hlwayE7dOh+YnE53iuXe09rk1CIrG3EEOu0Zr/3W+/76nd81Z+J1Xm9dQo
izXC1JLTD0Z0RnrLGmBLWCEx+4EM57f8bY69cPN1YloJgqx4G5Voz9z9e4XpPe
DjWsW+nlXlbmC2nCfOVl7vltvbdPLFK2BezEFHAAgBafm1m7Ndq9f13PWSt05M
WCHRNLze49K4GhMyehku1LMy+syHEtLKdZiIJKZwK0BpIrVtUT5CRztdelr5ky
cmoUiwhkan3yKXa9clrp/jyv3mpJq1vwCy9+Ciy7+HpYckbqSZGwKCEbLxjJXQ
F8E7CvsMBNHFEatq6DFJ8ppyfDKjC6AYbeyWuie/wBfpMrgRkJGBECXbG9iYZj
JVhfPyqIe0JiJKJvQIqvG4XHSrB9MxhEOz8FPVpSpwGTU1Cdpl4DctaQqmMFTD
qBHI/yn1wM60AYCWh8kpl76uJknOhPhG7nGQwdQ5w7emlaEiGK+2fwUI4KMxo8
FHo8VsZj2KOSycdOrlANU6vgCJ0ZNeLbXSo0smm6MDIFfaSZD7s9c0D59/RUOv
Xtj+C3r9CYHAZvKKjyUcVTTsWTOu2iHdAQmP87l6efDCjwT9tKYEYysNa3Cq8z
xHkGvwbfGkAZcib1VbcDxtaHO+PFOhAEXUuIdrDRD//4Zo/EbXv2+6ROnI7Kyn
J+yvdAnMFmpcFPBf6I7z6Oz4rme4MXr69HSnmveTk8R3HXrjItcHKIfjle11WD
jZrpuU2mU6Nn7Un0JyQAQUelLJXrh2RbE9NHvYG/mj/QIWmERFyvAJvURkbqgn
WRkfK9fIGXy6uFsmohXia1QdB6gtmcII923fqsMZRMRvfKX6lelODfMyGMvSdO
8ZsEitzfP+Fe8x6BIWm9RFbf7EfogqjY4M8cLpUSTpjmLuzrwEsfNFaK+B0CLn
tRZVoSGcKSG7B47joDZlB4qEyxa19GvSgDMKkBXulNt2IElGme9BOwYFgXejJQ
QucbsK3ZEwmeOl1Wmp9tlhK4nJ5WlOMwJGhDBMRUj/ryImf7PzDnP1sMN+kpwh
1JQYNDSVlAyVSYUMHXPCSAlTmo6FoaCRuylA+v+rX2/UkehQPwzPeDzbItkTK2
HUxTOfW1JrZ1pOrwkc+O/oxgqmGlVr8V6e33rxeRnY25WwXjYSuvQPxmS6O9Ih
04XDQOBQGLVdLEnoQ1xgwWRS5U39XXbdiQJcTBzGIr0dBmTi1GxLBcC+khaUFQ
4a3ljVRGW5xgqdCESqLs18Ot3C/bim3fZSwzu0NHEbSPuwn0zH7rhLx/du/apQ
sLB/e0dZnRdw0+NXh95LMeQhlC5ipXZGhlvw7bgWp7K5FCZo6cclHd00cuQMYq
2acyxv09C5zOLBvevYprPdtU/cnJNiRxACKmVH/pIkt/7oQp0eP0Ymc0CNOfU7
uef+fiepFC5eiV1VxqoIKdSVlILDea121Xzgij8Pvh3rzUgGit2bN7qNXva43e
l0RNWApTHE1DwLVhApapRQQa7jUo85Ajw5L3UKuIQj2ePCIayVFiZ/fKvo9Cto
E8rF8BUclYGX+nF7PbavrzbiDI/H5rw7PZa9pNJMPcdKyMjkD5ffczVjFJ3lhs
wlPmRu915w/+wl9tf7VYhNPj6xGIQU7oCHMotrIyod6uGkVlAbTjb6CDMHSEv3
90o4hLqOUtek6ld+UNspyGUqXR2+ot4RI5YqFP92EWLlCqpnAFJxKgWSurknrw
zKizNlmuUutZlqUM1XO1gkQnUOwm2tUQ/2rpVogRtqIMvUbMLpQalthr7coscE
gqc0mUs/Iw9+VkgqUL8muJ1T4iILm1GL+NW/fjZQQxlucAvE/2jfWned1n1cwG
KVUIUiQJSQWJICjoPhf5Vxl5i0rE2NNxYVJN912Y3rNv44szFXzEvAPEt1ZI83
SErIQBWHUl5RnCrbpqhOfB7KHWUYf7FCpYH6jf4jz+ooyMuU7Zn1djckMjwxTx
ITlEP75Du6mj2a+NKEClHqYsK2+JpZqpszxPQx9SVfEW/1oU/QoQfnNjDBw4Y7
mKrgSxJgQgkxj/dImWvAnRvDr7qWaD81RICfDfz/JN4Safofv9f37DfumQGPQr
P4jl9hsO6KNnKQOENOzCZguxgoVUfpsEohyRKmY5KDslgzL5bIS8JY7nVdlUZe
55eRthcMRyHlnGgGn6hthY6bHYFJ+xhMjtQPHSGK+M4VRVIFOmogxGbWVBTPlG
BhWrW4N8hCtJvw0uiOQpeTWk2bZnqIH4v7kUl8com5f/WWpJrrhZibi8BbUyet
u1SEAq+ur0JjMXrNLKgrYMqaZ7UWqsDTmk8Lur3nNP0wfaM43UEwaACOuvSQIG

KJ+l+gkEcEC9ml4COiAfL3Ughm8diiO81YHF+LdDtSSftZqog2VNlP6ubUM+6T
vI5ckFdCpNBAnice98Hc1qE4QvSMsPypjJQjVyGuQ6CjKJGz1FxsE+7QXNaItW
NOIpPW+wQRh8TGLflks+Nr4Oz+10SM/nilxWQ1XX4IgWAyrZrIlASYk2UC/1PF
+OhKaiCfwUrq8VYSnpoM6RlFqKlXVpOhHPTLcCxdvPGSWY1p0KmysTNlnPSe7F
Alj5GsNNZ15o+EdYwRFYylqyWWEdnhVbpTC2v9MxWcTlk6KSkm1WgqLWE8mC2c
GR8rLIEjcJXh/qUO4BhdIRQRJJyauNSFgytENdYCTLBmExsqbf0FY2m6dWZHR9
hjaeSsiD/ghUwTYXkSoJfrI2CKr5FhmmypotGFSOtiRaZBwmT1PkajuIESOvcb
R5sZyy6HVztdx8tqq2eke1O+V63aNTxd545aKBy/05FFQ0dAxMLGwcAIRgBMVw
BpPF5nB5fIFQJJZIZXKFUqXWaHWZ9h85mswWq83ucLrcHq/Pr6CorKKqpq6hqa
Wto6unb2BoZGxiamZuYWllbWNrZw8HG7bBjCHjPjPqJ2NW2+OMaXcddSzEPV+a
imKYpbtRNemW7bjcHq/PD4AQjKAY3vFWeli3Tqvda6+TmMlic7g8vkAoEkukMr
lCqVJrtDq9wWgyW6w2u6MtEl1uj7cffX4FRSXl9rG0iVVNXUNTS7sdMnX19Dsg
x9CoiRi3iamZuYWllbWNrZ09TCCSyBQqjc5gstgcLo8vEIrEEqlM7uDo5Ozi6m
bAoCErDFtplRGrrbHWOuttsNHoCzLuainvoICvErseiRptXatq27B2F6dc3Q3p
Ehrt+P4oCXneyP1GPWrMH++atM5l03fSbyitOYPOeV2XNxK9kIMhUBiyW7eR0A
XLnenwJMJ+NOqmsF7W6fBb5HdPbZlO8zKF82wNW+7U2JsOhY3bp34/VrLF3F4H
p/DH+BYwb5xvAfOGCSd2l27SGbZpHI7g1GVILt2Dpx4A7A7PcjtqkJXOkllK/8
A/jhW843jJOw5M7E99jsuu4f7REs2lvHJrgHbHc7pYP+AmLRNijo0uswZZ5w3J
zl/bkDCHbtqnavGGm+eNbS9cIcFO5GcAeWMat+RNutoSg9XXAAzNZBP72BSX3r
czD4dwl2oym5gD0gt0rACv+Pnd0bN1e4NGMdk1hfe3xZHYorUCbu2V91b20umn
yZfBkndu/Ib3W0TKIQ6KCALdbiyuE0FgK69fEsAbeV54dc5+0Q/ekXOVBOPURe
Az5TuY7XAv1LhlyL1m4RIEO6zg0Gvxnt24xhBIFLcmcnheG8lThmh8bR22hgnh
u2bI2nQZrF+4JBOwnL3mKzj1VhgcgUQ5Y7V4o2b67hfPdwSH9E9weR5eIfT/Oe
Hglh66ewXk9YkuYP3F+MmaxcaIzOstg68NUtdndYAVpBYl9KAO9PgP+4AAzKcm
jZyFl5P/d+fqZa2782OA6RCY55e/Pu/7M/pBg5rqxdeVXvn2Z8PiDsB2vwfnh6
nWP3E//KLWIMT6EAAAAA==) format('woff2');}@font-face{font-
family:'Computer Modern Math';font-style:italic;font-
weight:400;font-
display:swap;src:url(data:font/woff2base64,d09GMgABAAAAAEA4AA
4AAAAAerAAAD/dAAEAAAAAAAAAAAAAAAAAAAAAAAAAAAABmAAgUwIagmcDBEI
CoG/XIGVegE2AiQDgzgLgV4ABCAFiHIHg30MgTIb9mNFIyq3AwUpUJ88KurlXv
UeUZRP0nDF/8cEKkPWAk834KoT4EERhlesUQmhTTi6p5OuXKtq26/63Qj/jOM4
DlYqZS2xr1/PqGf26IrF5zGXEWL69Z/jf+8IhBgzwFh3hMY+yR2epvPv3V0uFz
+N6EWbNm0aqUnEmlq0CtQEK6U6GL9IoYiOMZgLg21MFGafYVNjZkwFomrI6pkl
KRCKQqEwjsJGmSVYjOaTsiEL80YnYf8dVfe/qqZ12cikU4Tv5NQy7MmwZDr6mF
KXMRuwArkpHwBlUKbOYpO6u2Tapq/KVzooXHl7bk0hsNsrQulrW0EhFyexf1WV
dpJhKg0ph/x0PjKluTANpKSky0rDTwUsWsOsYcY2Hege/AbcrOMQ0iG45voH0/
o/oBDnnzWtf+8q7dykluN22JTYAXKYF9XW7HirspytyvETJVfnH93t4QzFA7RE
2WGSWwkfP51/ram+P5vSqiXj/JJbJk6Z6C358LTDMtI8PT4/eP7f2xOIcVBml+
Ot/iDciSjA4Mv2qpbu2SwNgwZgJejRGZnl36azyIBU8RiwX/5PZ/+2987KHxg6
TA9cpqjTNOM70lozI+2zJO8+e+QHtnzeWWv3geR9ZAdMDwKAFQCuvR8Y2xQVcZ
2iS9WmKNPVSZXpj736fy8KC2RWQOPhhAac0RRpu4FEY2Pn26gysYIi0jFtNhwY
rnU3+A+n021VXluW6yksCCHHPPcZyyUujjiE8GiWQ8sIodvn0Ai/1/5+2qFdg+
viMGLAEAJyY//29THcf5GXvRezZ4ACyv17AMF5PHvwQPBzHwdBO66+OfjFHRjS

LzsH+p7w7Xeem8RDuB/y4f5Qw63p6FnvcKDd9wgEgR+9CwRxPlwOQ0hm0aA5Hv
VwLQ+HUmiernIdLnH97J+Dc9VcO9fPTXPr3DEvzMNZzD/z62/3H2rBbbv49Qy8
/GXn3zK3z9GXq/6Vvv7Snrjn1Mtk2O/maRK3xF9//HMMlfoSCNCyzYFljx5f/+
+JjGlmxcuCqvhfq+gH413HP5hHZvrwK9f4KGv5L2MVUz4IZf1IgQ9HkPTmA7G7
X55F+7S/hAVS9plwxGNccOmby7axllA7oaB0AwQkGFxFAHJ1URGGWLEVEaDfRT
bOFq4lTYTgMrzRumwdJ2F+W4JiJ8w29kSvlwM8NsbgaOwuW1t2kG6unF5GISNu
oHVdME0IJfyODnId8wtoBslk6YEPgtzp9mANevD4lGj6Ndh1kaP5iFjxlkjo7L
fvyZrUZkxcn0jfHLLsernsRZFVI/a0W8fERcylXHA4vgMWSNtiAYQy1BUHUMJr
VgIl5aDHTRnCQTHnhFKeUH0faauusUyf15/FI0LmsFs54kmvah+VIKvyAH0yKK
c4rcgDRiAasQf5nHry3RF3Z6ztTGpFLOkcIKeZP3LoQlF7Jgo635twHc3JG1r7
4InE/R9Zydk1GUTTlg9TE2fqRa2CvjsPyojjjGitinxQwHKA4GwDvqPAnYDQcq
VIeLQg+K6SP4xGAFDOSjk/QqW3sbgaX1UVha69PiEGwZIER+6enT4gGpxOnhir
BrKw3UQjYJTMLbci5SGkyct2cvv0pa1KJGRe5+ldkQwS0DKK0MgZSCmuO74rWR
CkTL01YvWerShDhscK+8WNigRiDqCM74cpXqbulIQJU5ooGpARidTcEI5Q6Uvm
RpIBqRtdsg3kBil8+OYz+tPpDUo6WW4uXW9t+Q9f14GxcYa+LIMMUe13l6JdBM
2RzBpZCofLsg0SexBsCYjEonIS80TDpW3w3iCGVrpnDZ4BAVwkYreolHESuCly
Q3I2TiZvNkPQaekISKhmAKrbp4xFkcHlmyflDiNK3VHihKtDOFTurAhZnC6ccz
IAc4cJWmZmMreX+9e1ohwZYsuKiowBUDIYVAwBNUNBw3igZRjoGB/0DAcDE2x0
QIowkVj98M0LaoKPtwwPB+d3OVRu/b58waE26++7xqHgXmbbImj7a1IDLNGIHJ
rQ0IyGFjS0oqENDXPQ0I6GuWiYh5Y4gWbaQmE2UFnRSlG1maXXonJW7h4usDCz
cOjPdwOYjC5pU+KavL+PD1NGAvDYq2aWTpW6m0b3MPN0c+QfKFftTdI+AJMi4C
n9DcUo1DspgQ1BJjTa2oTke5Du6l7KWi7STcbflRKgwYRFlVAQefgZlaI+31Qs
69Eg4Lxrz3JkWOQ3UXtDyPyWWXyXtk08pRF+23z07uluKFPdPHPPZitASE7FFq
BxRXS+4SyWAEUsd5LTgSaVqVdSWUGNuoWYOxcDdn+w7Om78buonTJD03RqT93H
7ZKQSGa3qowDfiIS5kAfefZX0yooQy/kMfYTLwXRgAofd0Mda1EwskOScVwz0Q
g5/tqKhAz1QQNmSpQUXoWcfHleRPKwDdE0FhYSoc7HaV/ALVqxShAJbp6XVFWa
tYZDqyMKQe5Yk28q5VwNQMRky83rV8j4Go1YU1orkPPRbgEWtbd79wPGzuXImn
dKpnziz9UeRQmcnQrfc4ZGPpvp1AdBLSUnN6ygn5qZejRaZ5dulJJX9L/xzEzf
eQBhbLevOghWZL/dIbAyJ86aSvsQU1mPDDHWdFkdjpyq1EvyVyPFk50A2u8saF
DLWpB8BFk4Ea7n/p7Y/xaTWiZm9GH2xY2sGVs181bJQnQjVFFKI2M1Ps9bmTuc
SonAOo999YcD9DZ3T7OHlQxfnfhiPTIESgFLodEAll9A43VB8pGgxjiZm2RhE6
Z1wMZCphkDBI09JhiA5BFAMMVUpYE4YRlSkD0CCLYwzbZiF4KK1mHbDID2I1UB
cjNRPJgBJrCYyMIklpIDY5gp1IJ6BKDGonQgxphBCswjAC0WbZPD0IyDJ+jaFv
AsaF+R0oEiBaEFiCxsj5EJ8qgTU0A6WCkyKku9aJB183MfQCgKmCkLgGo9ITbW
lthY6laJA3bOAfoCYChAjtuh4WSFsxUuVrg6CrYCYC9AHvrQ8bTGyxpva3wcBW
8B8Csg+WMgf97edxBi+wM6Nl68WGNVXUm8I3IyXX5mT6hiIQAsZPQRKxz2HrTG
IRD3DoKsL5B2CTwM/umYYhh49uGlRBBh0Y5qbcg7h62l89J1CTAaU0OKAhaQt0
MQBy0l1X9WAjrEnOX/LRvobkJ9QShbwYsWU8V2iilmg4IKlo651CtGBm4MF+3I
rGlVqoKqYq14TJHu62jc+L9uX5HCoS4P0g4y7mXqxGXiiqHSiQRDp5XldE1TOO
3vG+xvqWNxpa4gnJPnGIkuWiQTziwr15ptNaVkoMWvkUkK0jlh9yhDBziyQkfm
x8rJxq5kivFKiGJTOeUtKxHxRGiLCPFwZ/nEUBcB0Tw5furCj0Qe0f1AEAIILn
Ym1D9dPL2C8aMUtnlDUvMlzDlnb8QFOSI/q4k8BKfFpJAYsbTcH4eWlbI7x4Jz
574/E4YzOQZ60X6wpS3zN7DINcMMG99aBOHJEFLdF/cRnygOPAHnshcctgYuhR
18ns+oY5Jc2SglSPbakRnBRGaQQzyTnG+K3F2KzHhGPXsXJCfS4dzllTOSG25r

CUTpEfKTgyhbzieaPRqBahYtWAKj4RsjJG31MzbnS8CsSMn0kB1ygyhsuf5LWn
qr+FzYqlfUr8oVcqq2R+ThMEukPkKrQ3Zacg7R3A4U0o8V9cs5l0HjYLFrkSZX
fdf5zPgZdecsTfngQPvNw5Gh5MIHRvGJgy+RuJ52KRCJpkTh08aJXF2Q3NGgtP
mEOtE/jSjqWcQIZEvaOGjbHfc/y9NMyxCwkKhM4KLG6ipWlowMKgIGGepNDmY4
n63VDcdSNC+pXyWQ/nIaWT3rkaIa1HDnW/SsQSHWuv8eM0i9SIsfLZfG71QEoO
iw84AB0bRI1VUMJ/7FG0OPkFH1KgWL+WWXTuLLtICeufOQfORiJiWK5xrwSyVD
6kSJ1K9R+WKHwuPeaZsG+T8h2f3ax1POqTlnkCPAiXNxwfrmhAFoY2NZPZd5Zn
0VSG+VLRDvKGez6BI30RJCCUk3vOo9iv3N9YhTLAKpyyftZ5sqIGDtF0/h3gfk
l6hI069JAgNpM7FvGsb7yc2UIGJYSw+vOkwY85sJG07ooUiKLKfuvm3Ux5vo3F
mt5Q6Rn15DlDHqNjW/3svdVpP4Gq3UOiOK5u0UXWdmpm8+JqlCPgJH7uWaMIfm
Xd+Xs99Glm1sg9VojCppZU4jV1J7bza94vKwaGOPytccxbJ4BNxrIeahwupqZq
CB1Wr3YnOjVH6ZcxB9/GvyPS8hpKfGnBevcaBZBKGixd4XKH00vFkev5Cu9O7o
9pSePRf7ol8NuLRpmTaHeU9s2p5erPNz6fZ9jdbeB9SX0sOScIDguLrKyZsGaZ
DCuNq2FPZuXdNKEfqvS5Iy0AwsccUiOTbVVUOOYvIdFrA4R/6lPYzhJiIGncQi
QMIbutgS2bEc5enQZnM3wMYQ6fb98mLSmS2EtQtuRPe3x+oUotsvlAVyhwnWu6
Ii+zl27EKQg5wwEPaLR69L2Uo/bl16HuYXSI++GJ23CUmyzia6QVe7RV2Kfw7D
Iu82fly53ArQ4MPhwH664nfsX0IiUaqPBIU6gnKDfhwZZxR7+uZnm8XSinObZt
uZTtuOHPC8cNDpVPFDC21ksuAM9+a/O5s3lG03dUJqHVrBq6PsXndhWv6Vaw4i
D9/hP+te7wOWemwJD6KlIaOl0L4Sh16vcr4DrprPod3oHxTBBByEKFDCxzff3d
rMKL+rv/JuVYf/GLwA6ripTtWyAZvmxJh7dPWeDV73DTJ2TrttKMZ2Eug2TP6E
ZJt+aa9feicrXbAZhznQx6UX2G1UP2E+IP4Ylv3lr83kyRMNipMJ9RtGmeByvT
Tx68XKDirfdFo25rhuEniyFMsqXS1/xiJMf19FH5DTGM6ONb6yksPEjwC7Lm+d
p35m04t7hgNaxkQ3D7AbpcgYJZH0QX0DuG+J/5pBjMO63fUrJOnPoChyE68Gj+
By5Skp5EFFCA6huao2mYZAvhWImkUwpj8PR0QH6RKFsVo87oOer894GqwDQCyn
DPIHEpbp4HKEF+kQuQsaB0z6thvPc2eLlExREsBfHCvd1RKZEr611/fD7vt/1c
kGz2dAb4oFWSlNv7Vmp7FYJiStFWXKYaOXhnfiuQxyhH0VubwSr1S8jvFUHBJf
b8WziN1BDnvfFqL0TJ6aUea2vf35p59V+AW1Uc+zjA6URxpoIGJbmpwcHycTHY
A/wEDF5jvSn4M4uJS9YM5HhGl737UEwaPZDVVmxSlheIMzshhJgV6U/8s5T2Oc
BzqJegbrmnXvnk28GciJvv6rMtmN7+NFL+t9hwvO2Y8Iw87P1TvHLZGCskMizs
NpHG/AWvkjV6/eajD5Gj9irvisPwIKoAQAP9q8I431zBJjeGq/7d4xp3LOX02M
ETGyaAeCxBPm0J09CA0uFo/kE4BATXWJLJbhPdXEphVhLn2z8BHexWbcNlxdx/
GskMpfpEXBsYGABhR8B5JHLl5LfLCiM7SSuc6bsQeg1/tNR3UsSfNLvwqLMlLt
FQ8Nf17pq6+OYAm22RH4nzHb1GnOVnn6iGuiWaKvTLFpUBjPGy0HFwN22HR3fW
iEiI7pv089qKhXP8aXqbvlNdGtg+Bo6ghpiXNzt3TRvCja5TK/DhY3QxRdIDZR
viwQETeQ0buk8GCg7WDNgn+sxMSgn5h5/DOHVhehVFo66slZcTTJCZMnh18r/k
DGk0HUYH5Xj9xcsOHD38455apRyHrqOJYRxCrBLbbav3BIgZ+pQivG6a66fKe0
sScl/KLXL69JyjBrgxvUdh2nMxs31HHaGADq2PxRQjoJ7NIQZYyoOS8GJiFsWt
L9E/CXw+RulBKFAulh6tXcUjlq0R3Sb06zvyDXHNEdbL0ZBzoUQfuabfD0ZMg+
twXcApJeKdQ5bDoQ1Ivh5vza4QuRNrvkACPJp6z5gD5FbE+DUjGWWtUsiJsKS3
OBOIgepH4sL1PWlhMgb/oVEPNqGup4PGdddB4SpiSyNeODCowZk7IkhTBv9Kmf
OBRJLyapKJTMQ0HxIZIhVDxRLVyoUMLhNBY/Tx61mMv1F+0sT2UgzUXLoW6NSV
BEPMgoXY5qpcKL34v9smV+DfKocS2GRJga9xzGd/mstAlOWAR9Rqu6I3fw8kZW
EGsttUdyJdQMGCh4WyW7LvusnH9viF+l98dFjyVSZRdewDCWOXdWdgsJxyeYbg
WHFpa2MOjrcxmnNlK+ncC0rglotHdx6qKczqL+igPRW8PaXim+o+r0/cigS5Qy

K7Xfip1nhEkGIapDH8+1zbjsu55GFsd/X5m327neqHU/cQWo/eXfihm1nVHbBw
6lzvi2vQmV7fLE1SZionoXNHAPd9nR7DBiJjT7Qpo+GK7be/A0TD6kftYwVgaB
46Df+Rdfy0LCFxSETdGq2hcFhovbM6ZP2cEHTeeanI5pnKe0plvCXZ/6syscT8
e4ScLRCClVkivOMSBvudXKDi0KSYmkCqXSZR0g/VeptQx7MaBiyAY9/Zi07LXL
Z5W8BCe9aq5cpbIU5r4tR08LdHyaPg0KDE9n5QJ3shWvU9FhFcjgKHPwwKQpMf
2ANgOb1yeUiSURqW3Ea9LXzxuP7C/qSV0RjrC/H0uzgjIoUTuMLm/LVbV8SSdg
nl1mpdhID0mk3j0SN+t0sPG7GA7pClzSWvxI1az7EIkQKqcDAGrUBrWdZ6rr92
1ql+5RJ3lX4gfz7680jvWeymW99oEDrN3YMEIliDFCZQR+NfxU04NRorTISZuZ
JhZ7jncaYTzb8sj2MJjMNOohYrlQv4a4NGeqi0k27wal1OkjUypcCHSIXeORrS
6uB9q64xRrKVdmNx7GI4IlmpYIxek+J0JX0OwP9Z8u2v6QhFCujFsSWYOkM9BU
lidPpI47ddZ2nXQXndDHcMQUant+qoF6v0tLQvWXyiQkUWqP6HsGjEZ1/auP+E
pJyA7K/3NtKV4xnh2tJCAtiSRHOPG1+y/AXv7TQ36tXs1++zs6dtJi08++WLwo
YXZNUy+bI6PygqOBmK+pUpKIHBfTLy3GisZ2kvLblpmJy3GjgZqqTXUWe/N7cQ
qx7tXGWyEDkFmpLWp/qH1hww+eo+t6U36OiFcYX/KaQYp1GvuphkGIO9lIEJl3
CfmCb+jTimVMW0P9zGGzi3bj5eUgqP2bnL9eLZdjIXBXl2mz20VNlkcbOa57TT
yOl4z2Rc4DeEA7WsC+ryPUVhjfP9x+Z35DVhY4N59K0xbfgACcZ27f1881SaHp
hfyx2Bo7shJL4m9kc7S19q05dzT15qjnca2YuqbKQ4GaDfY7+e6NRQfCHhHt3t
6j5RNQgGYNFumyV8zVhjJHt4iQ4BbRY7ZT0jUcBaSkBHTkQnCQORkVBiZ89qs+
wyjT11hW1e5etEu5HVce/zTedbKFScN64AR0dtt+1BKZ4cfIU9VReM84+7/5Ue
kxsd//WEEOd3bHqsfziuXgh57TnnYYnJHqhZixpQhVDoJDIczsQbdM3MkLve4u
TxicuauO1UhwPb0oE0FiR6b78bosdGaVBINmMocliuS212/ZPtdUt86oZaxMi1
mh2bUVOtWOOQZnyCOTDJ/jNEuXtM3rkE5JxTmti6LLcJ8mTbuPld8h/T2cv6GV
3LhLjuPv8VPHr3xLJAeN2mns2RNmeBwrx3d5M6A76g+BSi9dmr6yNnSfGYTWxs
KieC6qj6TpkqqRvh1xZtRvOI73yIBpqcPwh0ZHNezuqHl9THMjnpu2MeOBjIvO
JWZ/vFFj84rPtV++9iAqiALOq6OgEd3IJe068Kt47lIxLQ5ba/FJt9ol/Svy5D
N8Q30HY04dr99fz18sPSCamYoQ7Y7ACasuCMfZlHOnWqJhqZzQKwBh3/FivCuB
3B6SGFu3U2eyQI4cCWgOuHBTKViRsbPt2Fk+QgsWVrWcxPMd7swqw8Ck5xwWvQ
V5l4sNrYsd0UwtosXz1Ruj0GFeMkjjQlingnDh7TO0rXOB5em57vLBG607ghPW
PtqjpozTfIs9D+QEPvoglZEN4ugJjAeVYTQ6tIqlflZ/p2/rCpSyZDIxJYOSIp
xEQl401WEiXRr8JPpvlBx89uFmhRPPwIxDWaRy+GIg0HLoPd1+zKZ7buNRWihP
NxzMkZhd1fEd3uKcWWnQm1sty0tMDTelfLJpjYg8h3aFFVoGk/9ze5ZEDNAtLo
YRQ3LYzGg52FpiDzKzQyi9xJEg1tF0EQU0F1Ze1J2p9cgUCwvksHfnS3ZGXZLm
5lQDrujNk99Z574mirrBnMZbnbzIZ6DnkNyUqUtT2xKRIVHhW4tmyPqHMTzuRi
71POsYMQp5pFt9VMKxZ43G7ASF6eDcQfYkIA1uQWA0jGoQeiYcx6Fb2fL53icg
DCSXNXCaDg2SLdEZNorld6YYpEl/rPWinPvX0jGRfl1mjdYadX+pVtlF+/17xo
Uqa4UU5EintoLOkN9pvQueF7v/7YOQ0pmWbTOdxdCxRdWpZfOBjDOFqQ6Fvr5x
tcXIoRi0aewGQRI14MdKTyiTKGLPRLvDLBROBc4R1Eo7aT9ZN5p2OSzVW7sVBN
LECnNXrxXWsy42jSiCzdCCc1FecLG0xKt7sW3UvXvC5wOYbzozSkmXWswbpROr
pgzlSi01y2qZga/r2qbu5SWbTpOa0uwI0jtbdCqg5hLLN/NzGD6aUfodfup0k5
zyz/npLYBbZnEIMmQ9BAuFytbWGfV2ub/dbScUDcZxIR0A7XvU5nDMuzzHhRQk
P+B6Oy3YyUhGMpAqR9S/jg+X8AbtjCtQLSqPYi6ZMAn6DU1oRCcIyOasOWZiot
+wrXz3Ye09pJMSjKegAnK9M8cTzg+yOon1zBTpPyWlDOTt/AJKz69f+Z9jWjnc
xTFbYYFV8bPDvojfzF4ajkKzVaQ5Wjf8qrXMm3ybWhDzGX4xavs+UmZz/RF5mR
JjUzws9mNGKYdi5DdBhIanybo5et8mFsxoza5e+JrVEuZJ8OVMqBxBd9ElTFNR

3oTa4KkF0gcy1W2mIOBVcgFahXwi+Ya6Zklos1E6qnm91X4LjboxpD8QdiZzgd
+9dJreMSVK3fKR4dKMG6DKXhoEFCE25whbPy5nuKXSorUsslvls7lE+FQa+raP
hu3uLMbydP8C+jpra6KHQdFcF5zv8T6356gHsr021rduaNXwzNQ3r7wa4lsdR4
fnDOgeWC/aONYkqHMF5IE5m8F4/+gjWocYz+7zUsoBeNXlKG/yxHGyJIn6lJMk
USKiu0G8C9CiEoKcVPrQ1VZwzdUjdUA9iJU0VoxWxTNjVe/xKJZ1VRkHO65K2Y
8Rq0eq4eFxIBi9tDYrEftt6vUkyB7bfh6BAEnISPi2sb++aSh0NZX4i98n/aN6
esF+CBvCVLCS3cP4kBv3X1aoW+fvL//xujeTubasiEyiPozDLv/AUufpa6/QTN
2uUlx0/auSORZC5XmwLO1502cF30X0+wmim5i0sd0KVq44uyQTEv3ITFY2Jqw6
GXb8UFaHSrqFvm6XnwTmzesGjzMkTR+J9ER6dj/vNdJ09upWr9Fk9Lb6g0B5PW
2lPGZuULQ30gtE2YGvXO5gZXr5Ffi1DsUpPz9mDyY8p5Y3OisvFB00J2yBYRnJ
0rfpoGZRCebvCWrqGl54PWQSSxo6NhT+DrItmxQuW3tJ5K9M1S939VVUlX8fKu
S16I6rlR9gu/oGtji8M6VFyVUaA6UIPkrfHf9ZSX/cSWYemF9Wf0MwP1P2P3yp
+ohScVaueP/QJz5EUKElN9934RZk9EWfLJMRPu/pj7bcvytuzH0k9H5dCI//Kv
bbUOOzwMvz8cSDqDTYe/ASr9EYWRzrsXhSgh3mmIU+kJN3AFUil6f3vViV/BaG
7KkroMDqw0UeEgt9Hx7qj4d7wm/xKMfkxgceemvWSGI5YsJ3cXm6uWlZ8kOF3H
cp5tj01fVnvPc/66z3OLjZmVD+6gMu8ZiGUlzuczkWVaUirpyPHos537JOTPB6
Fv4Oy8qzTcIpl+Icq3j/jBCbvzcv6PCNzvHb8oINxMvfvK1prVr0bcs9GSt4Sf
gSXC9onRMubvUOx67/a2N5VVNFser9s0/UmHMfmRX5ojmivD/vKMq568JgH3k7
/jdZB+ioG8+ihVWURxqG/8suaaFiGyiyDWgCmgAVr2JtSL4zIqJSQLXkhPjvv2
uZ5LWiyy+oKtFSvupVWsizF7T+hzD20J1/OZvbrxiEhnDDakC53GOYsFuMKvcy
lNPvhtYjL4cEA5NeM9fa7Yvv8+OJutKYv6ahbgTvPnPS2OcHcHOCE2Ia+JWYea
t5OGQq/H9PBmKOSCQvFVM1fyeTRzfHPN5wsTz/q6JE4xQEm3YSYTKMjoWg9m4f
1j8g3HY+HVsSKn5raUHVspJuyQY+297em8a7Jbbjt3UtJ31VQQmLwwpF2+6d7Y
zdPLR9gupcZpmJj+WHNo40q25wWxKPbCGf2g6Q2YULmJSP+3lyJiUjQrCBxvSe
lZbJBhcE3MRzt0JjRU3k4vU9nqc0Y22d/UNzzMsnoJXtKKAKa/sZh7rydbvjjd
kwBis5PacEx6P0zlIE7fYn9yyR+nzegq6ca/BHTgg78fyzMWwmA7l6bXDV93C3
SuCf+7ljN4zDtK8b4yXckAXiNnKbjCa1KRUaoXWlikRRpuY/GO0OQkuefUHma7
Z2XJ5WGNP84X9DRs1bUQixBmuC3oAw23JQPjp+wpBFC9ypvEGUp3oDyCnxAPYw
0LZBjNiygYqmUFEYFfCqdLFXYqtbmiHXOAtx6w1zpUcSvAv+12EFwnB+jCfu5y
E1F+khkIy8eimj38wGSg3rDGvEyWNRVR6NAVlyPeZnpw7tvzT+HMrUnhH7cqee
JhdItrzc2bEGw0raC5AcxRIeftNL1izCVWXJVf3qKCST4ogGVsIW1xgmyKmGvG
dNofSmj+uAxyNdasW0+VmPrgAhW9+lzEC0hKWcvgCPd57vm3RbLBXfQvDevTBS
8ZkENie7xXMynkXItlmkVBb4UoCQblp0lH98mqQs2/IRPjr+siECc1VZmb9HlY
EYEgcqpLin1OLnIWDvXm6zcZ6Hcb0/vofkWawFNew+oOr4cyHycgFSkHgJq/4c
QP5H8g4/3h5WwDS4vJrJyKucXqM0CuKlvvXCheZS4owBfrrFHFUZtxu34d37jn
N7RUT4sSfHxItVGXlNIvSOaBohQoQW6xolE+QvZmY+tTasaW2ubqhymzeYCup1
/P2rkMGr3bZY4GLl1LuYtgqE/J7wqZA4zx5s4NUmlmpKVIMVBu/7Atl6QgQPA4
uUiANlifv8OF85aCHi6FkiTZDEC9dA8/PW9t/jR8Kk8W0fa2WaROOl05QwixY6
vCahR8WfoINFSUG+SN4ZsNTaYGJ5SES2ACOXgghJ1tJ9tOJwIw64OHGqTuoSy6
7vRtBrDa8O2ULQQ30vVV6Q+cz7Nm5ECaL+q+yUq+F/faUBsaQNta8qWJdrJYz/
AVLFYCJpCGhi0JgxyYeHActEEaR4g1iMIgDs26PaxyCqdLqvec6MRPNgMGH3Fa
XKFyWgorTdd/99UBBKdncbPRgvgxQbI+XTss2B8P2iPQ04QwKSVSkd1YYVoHXb
yB3Pli3obZaIO7lZw0sEhBlFda3bhR0ATAgiZDiciyB5IIMLgrA3Nq8h/LqjKO
JWIdHV2M1YX/zWihS/VvSKVP0LK6fyYQVKkySKSGOwRh0AEqlAOcgwEZjZ3yin

pcaL7X+tegHfia5ITg3JR9ipp0jbnh7J5O4VW1pmypIJz3BwdGkkWF+31iwgF3
rGn/p7RWfeWw2hntOvkOzP7yzr+eWd2oc7YmV93BWK6lumGkuOapfg0MkYrtXk
qSegE+ebau7jMSrFw55QcBnZ3QN16aZ0q6XJZobT8AYqbPdt6Y5BpEvulxdr2m
mch+BkGhoLCIrvJMMH8V3NIhMf/eQd26i5QjNCl84jm2/5cszL5wEGgzc/YbB9
sHsxX6DEBu0zYdM2lbPCa7R80nyyoWjJY9JpgXEy6F5e5FK2O6Zfkq4X6oczEz
y8IFKLLTgfEFB7tDztlNok1Pe8E4G4+y7o5QPi81G30CtvW1NyBOsPWGPiYLPB
/J5S6l7yc2NWzIipqXcvWcUOX9jNF4kNilnVl4Tnts/dx8OG9dxGmA8SJO4uZZ
aaXPThiAjDQXGteA8NjkdhPwqDcCN0uU9Y+Bsb1xtwOz8UqvOX9rfXGL7ZYtxk
Ap+sfDRbfqz6ko2ao/K+wSuffDLNl+BI+RcsJQnB+pga8bXU1gtf/jreuE+h8J
UkNpqMjxfFizxlo2sdUaGLLxq+7t6smF/2aVbCCDCEx2HXadYC83qjDNzvGNqq
ADQBgcA+0zYieGeYwng6QzB8e/s9hs6BVhEiycKYrp0NfZkafG0TQCAYYKY3Ic
RNSrBuogjim+dViie+Hzr6g3NDGNqej/JPvHrcdIKBC2AMZiCSQ7ErtpeeKUut
HVHGezYLcxtTVocIYlroVT0n/0jhUzW6tmu29iSXdwrreKSCVHrOuJ7D+evhvz
pA8bmS87QOK7iTjAMqTPKzaCGsgGleAkv9Q26FrN6VmVSAoIxuxBKbpjsKRGTb
6BPtq50KCOEmLDaJP5sLl/8ExSaSwb3f0XU8xbNkqZqwovE7qThEBox8UzqYHx
bU6q+6x737X6KFkhWZuM/5RzMjj3we31gDtTO0HHHMQMpfzitjHI14X4XwiEji
c8HX6hbHLZ8xfO517Cq6aSuT25t2aeVUftDNHSGrdU2AIPloB8Qq3VebELHhWT
FF7XojeE1BcO7jSoTolkuFiArmI7Q7gaEQ3QwxTUmBKR2sTv+QhaFDr34JLWND
2xLa/YtPSrbK0xAjTGHKQUaWfpfIX25Ixz8+fF5ETvFWCaY9VVlxqrf1Pty6nB
MNMH+BoIze7JKWLlBzrf8m70HOjIv6LRJZPWTQJ2u7GGoyR4gSPrz4a/DoOTAt
pJ8De4BYyM/Nz2C2MlGI3MAfuJppyJeEPMqZuXu+zqIaVA7aJ5JfLJg5zgYXuA
OE+KfqS0zCSBVFmYTXp4VM3p+uXrRmdO3IDgHxEymENc6ISBqGld690A3Y5V4R
6Ch6ZmHDobkDqiH1KLZExYrQJC9gvPAd6XHCGA2TGfIGYW07NFYvyCYVdTDLIA
BYCDLXsHSWvxNYCeMXX4QQuFWWAqb2JdBYDM8ENbGVO2dGBNjxrZK4jcrTzt0W
pOvTLgJHaRx1sOmLg8bauQYAGUnyn68SdODWd0MEb2Vo4uerfZ6xf/WS2rjD8o
1ex39oeyntOYuLb+nItVcF/9Y2iTKIDG+7E+wSArEADzMAyWoOHVUfraxO+8au
vm+R7upG0ypdJ3Cf9pzWYQwfZU2ZQLXR9OdAohqeAxXvkWYzMO1QLVG5pi8qjY
bN+gARIaP4nb2GFh2docna3yYQ9NY8U+mZQ4aoLHBYG/t4FSn7cbGkMrQVQRce
s1QceHOJJEkuzW21FDgLtfENw3X8hMT8zjS23Lggx/n7mlRCzxdrcbCjur6eCj
OlL4RhpOgVVJy+6TibRbgAr59tKn6oK9Tyjb64PHrJRvMe/Iywob2v2TbBNG5Y
3imAZEjezbsRKSVnIq2sOANZAdBcbyg8kDcWy9jdLZLRsqXpGnNQfOk9CGCMR2
++wrECn7zhFlghOFIV0ByAkMJCY5+/BmJeynEBmOzKZZbsH3O4+UZdqSY/z8pt
YsW6+UvJfrEfLi0gMVE/c95ZJp+1PLnFeGC+Ofa7UV/2z/0Vq4tQk5TGMwjf6L
Z1jq6d1vaT1Y8elfjjHeHaFwq6snmv53qaSeoNraRWLFv1nMU6FYmvro/J1KXJ
XbPf/KaA58Chrx5Q9hBBrrmmvCg8a96OXyPI0VuJq5wz4qBTUemmVgUknuQy42
bD+vpTBvEK5b764HHi+fESqDMh8OAt0VJ/cZHCuaHgUkIumRVpxa/kpSWMtOxR
V+QHO/ZQqyKrorZQ8mklNS08qSRhZcsP3wIGInkoJp1uUmXJmw326hyJD+TP5M
+EhSpSessDyrQjqCXTPnsk7bJJbSQLOn6Cm3F7Wt2iJvsJTYmGmhC8oFOL7r89
VwCXgiSPL51epIv+qraAl3hxhArwSML1vOf/OX81BL6dV+b8aNROqugB2vKTT2
Da+iriPVX8joSPL+RUgxX07UHvTRofW3eJRZ9jtGgN9Dylonp7HhObXnYOvpf4
69effljPSu7C7RrqwgLdpKCkZNqn7rhuvdI56BRkcUnnyWqhBH7ibmVfTsM+mX
iFWBhzWPYkwzqMQWAUrhNQbEFYkCoQo/8DMIt6jAnXHE+IKEvmu4xxu4SIGBft
9etgMaJ70/2D+yk2m995YRD+bys1vkf2YkuF0hp1g4vvFFtUwgPjis5o5iOLSF
4rn5rW1e64UQCDO19M9NBJeLYe+ZpLEa5BgRmUJqVaFLQqVbbb1hjo1xwTjx/7

dA9LXeml3Dy3oe6UXrnaLjJu/B8RuhP+GwZrslPv0KJUjlJhuflQGrCkNIugPt
3k+6sElVe/CHzYaAHBwfzJYFWTJlf0nIJ6aCaY/mEpN/Z1tD1a+bUlutC2I9ss
8PEIdaG0ffeueInjYxvFxhTmbFaS4Kl6UrV/8pl6OoZr2SyCP597aFpX33zwhb
8wq/RSvkKb0t/TuzerLn4rBOXeBWDYyPl4Y5gQVUWQQkeLkIgBKmJcSgyMIeLW
iZd1qCBXJZ4KsszjDvVoSgUb6mjL4RbRJNVIRk1wJ43znuLHU1E7QBTemmZb6A
QE+hyzwJtouXmhTyEWEtvdCE7wURXd8pjkIzEPieiDzdNc7R2rRTZxSb39DsnZ
kOhhiaHYnwnG+raFy9PR85R73SIpmyv78Lq92tXG7bFT2OR19ABFFv7+LOAIyP
LFucKG81+ynGx+iy/U/UzvpvSO4rIdr6Z66CAj1b6iItyE8YPVACmaLMjpNFPf
HVx4+41Pf2ARE+oa2LL0iRWMoQ7bHMzT8H1Ke+S6CTYlZnC4/F6Xb0N9nsWyU0
FTpu2mHRDs9WYRZzXEnHcUU81MwvqY6XihREzFXLfnqmCPN4sWOgaXmvdY9vNR
7JggsCWT1RUVBwp3YC7tqG2oQyGNC3iK0EUZWhQxHbqi0JqntzJ6+/IJvsw2ze
XR3nrafJlu+uFpKS11SuurUVGetvenW79SUAOhnHST46uwax9VX8WZ6Waq4WTN
XJcv7qulDLutvReThIrmbzCquvc55zYrmuQ3m3M0FtLO/7RgrUXjO+XRNL9lv2
wNld97WiYvxo4fltZIJRWfEWqlZ59GEc2dXboKFXqq/6HlQQXsgq+szbfuLXSv
ScoSkoKg/t1FZy3BRG2tRtlqG3bsmbxmvSV+bZLM/c/C6B2TLol9kcUhjim0KN
BMGtaWnMLK2Syv4g5nJCJ8mW++6dRxc50r12ky5OroScpJgFsupyXJLjLHe07r
lCkfU+QFI4/mpiQ0DlSSxk+LIjVnzq0nK2b2KPTlWkeelSNSbsAhnz0gtVx9g2
Zly5vL4bhYeZWiwB86owxIsbOHpTXqnq29P0RSX7QXNtx7jdpkCijyt8+SVvmv
BE6u7wg2K67HgMcTL/D6C69OGyETPmAwr8ulAPapfcXq576q2BL5GVj25xyyns
I+fegoT3bVqfe/kwv8/LtHEU4kyWK7a488293SHHnxsfNyez5t/3xnWVrMCIHi
k6ZEPCEin9gc2j8hzXdESUmW1/FvxXX9nQ3hu57iibc9SuvGm970sTjP/vxDWA
piuiCLuQUiZCxcMMdYVJvoVBzui7eRZXhQXOH64rab2iMrMfPb82EseSyiIkX4
ppnB3D4cZJcv88IIeCToMo3xMdLtR8AtD61rgKqgyovwGgD/gU+xVVTwcTY2Eh
LAx/HjcJS/cTgvU9gtmRBwqVDfN6sUyvVvj7XmBKqGZQvk+tthdS7YuLGgxmjS
6FX5hI8IFpxihH8kLSOCQRLFzfwgDQJF9RGIzwJahb6LVFsvGGTJLET76TBiAR
OcJkaHPfrSbBpiCKYLQJZ/JP2xApvVG8DFk4zG1Z/Ih+J2RrD3r4IH75hFAYQI
NFsQOSUexOSDKMK/77mvs1s+gAtAFueL+3lwzmZ8x1Rlis4ySaNJImEiCHjAoS
sjoRYR8XdFfx3MkhJTsDXozRit7EO8QRTRHUBKCPEgBhQipJtETEaT6fKh/nEL
WH70xrTk7LHkH2CBycgIjwK9ynrpFEyldRQHx//GBusxNWtUG5Kv3XzOzasR/r
v4sI+fF1bgTQ+29z+zHdpQJ7mxYQCwnn8ui+blmNpLQceDyU6ssszakWwRIqwI
ZD+DaQ30mT2Ih8qFJHp1nEE0fo9RqzUV89VvgSKZeALLl+MJE4+5uqJrsfe2J2
esNEJb59fWrkxzIjO9Fz+YGo0QrAAWodonDWVnIkHrvrhJiC0xi3W1bghSiFIY
24asZ0PsqqF16+QdiiVUs244t/j24RrbF9fmbhJhfSsl7UqJ6LusEbRWR/eqUY
vasEEV4rFHm/HZjE//3aaNmTThO9E3hyJMs3g/GlZp+1vBtSd/PPb3qF3SpBEI
NRuXtK5x/w6YttGjoN4FADXVRFvbAt6y15474tXOFj66t1N0TNjsOiv86Ini14
te52Mf95rtItsMMjWxvaKNdf+5vLuKXgX3fzewTT04ulrboPblceKYIzeMKetG
v1R0zlzaxuYIEZLNIpidvxvkWeQ2xZU517dQha9CwdysPc+1pVqkuvxeO5xZcw
3W6jGLUFCv0vk51hVw8BjkBDh/XOiGxaUZfUJbP4vruLod5a2Lf66T31g2FM7b
5ky6YqWZvoZWX91D+QjHiDlgvCl2eL6cMyqMaczSbG7ijO3tGX7NbftYZ0NOYK
+nqynQ3BCsD3uWhKl9IrsTC/GTFrY4WsckYpj+YeQ8GmiJyXyLeDpXnaqzOiRh
+aBWuyLWfQ8fTnxnxF4eb5CnxIwA67zlWjU/JI/aAqm63PpoCwfKwiu56t8vjf
af8npxAUzyBMuO+KXysDe2ScaE/styIgsj185Wq9N/713y3DR8Hq1tile5Sxdd
PFOVGNq5s1u6V9NuDRXX/sNSJWO2ykxE2q1UUugrNewn1a0rVKlmPw9HibIKn6
djYBc6hFWYrnE0G/i2d3Hl4SwC2MS1RQcTWn6IjCqHylIooRXydjp2poCAzeg/

DFV6XYmiP5UwKWVw1Mf6etqvEGsSf1Vn2dZ3VSiIr7Qs8QxEIEKKPX05fZgVuV
/zi4pcuP7TS0T5vTMOQ7S0Ny9ti197VsPPI1POeqCNUf0d8XXPInM2S2MGFeV+
XrdKBg2w9ytSOYW/52tCh6zDxaZTv9GXdRpMa+kgfteCbFIiOFnhyUKIgYYWkh
u+81DcwNUieR2+RHvgr/K6+XGRVfSyjF/a2eXxpc64PY6/j286MDNsMNXt6yqc
kIU/WIHlbJCRqYcFJMmDYyxPh0AwQXQTyTE+ndx/TqTbQowZBh6bn+zfw9Yrix
bXGTetzf3pBweh/n7Pf6soTWs3g5/Geal1X6tf3sGY/YGbfCy+pGCt5w35oq2P
1Tk2JTB6x8DlmWn0sGgmX7Oe4qSEmD/zGWfu2jVPgZd4mXdHBzUG8abS1cZIx8
ix7xT8/HPRNa79lv+vWlcVuHRusNiYpzdN1IggVzcUEscksI3fM9g+jXVodm7
KLvD5wwJn3hE5pZiG3JvtPQelhNxIVYJb78mB8nwcEkSZyZ+HO21YpWLLieJq4
ytebfvY+rmsJQ34ounykhe1xFwGwEMHXidf9OLJy0FdlWJht5MoYr95xoGRk/y
YPcGCRPMVQcXM/IbZcS/C1TUkZbQuIZpCt1pcRY7BS04+eR9/fwfCEi+RD1gO4
Xdcr2ET7fD68DZKnDx8emDoZ1zi/+DK0NfirAq/Bhl4aRrIjsWQv6gH/SxFVUG
/kHsV2HR5rueZZKXTP9LSV1i4b4oA4U4M1f9s16tTAZJ9iU2Yt5l3ll2Si+tYY
7WUBIM1qGHsljd4Smx+LCCRwufQAiul+uzXfe4btmLNolPigdzNn7y8gHzZiHy
E0hjcN5hx72aCJ1UNlPgNmGi6ZgulyKMsCIExW6kwjdb+CV0rUPf8k+qbLHLsL
9kFhpRkT3zT4igE9V65xVPh2za+BZKqjixkXuqgdvkcsBQviGroWiGnzL3eTbV
qRkl+XAU8uslDoUEnT/4/D3zJuAdILPF1e2DYeUDjhnqb8LDFvIsK7ut0xG0iL
CQoicI9yKgZpnDDz3/k0Exx4hEp+PXGMenOftnXetU0ZSYsKE3hIb9gp53rneV
sIa/krU+ThQtbNdHe7DyN406Yf9l1t/6wtk2CXwEld9XX5v1t8/e3iovkorMrR
3qlcru2el/O3+J64zmcEXjB4peIjj+MGfzppgo/mGEbKHkOmJrI3GDKIRIVfmy
+KVDCUlQ6YI6pKIPxN23/maMjbccQIRF95BXZNyPDRvthfYekv1Ogp/mY+Ot2E
PCcLo9VLxwRTbtnX0F2r6tkjJ8laysqmMLxzKZ6Zis/1ou0Vq5bZjpmlDmXZ9T
cm4b23YYf0RsNArdN9ykbu3LAKfbN5SNpkTim3M2eHyuvFeTN17vL5JLcp9OPg
BD+hsMtzFNN2BvPqq6irNS8a/j1evadDxKeplsFSJRONaau8gxJWXae5Mx91/7
/e9c58z/3sFGr69fysQuoRzvvltaIpAXYb8+UlbZWEMbiP76xrpoG/0cp4YNt3
LHmJchqaTkDojjXAG6tSvG1Gv/5/NJWByd/V25aDtP1YQFNC6KSA2os4PO0c7X
193kuGh3Z5s1kZcl8R0vY2vS6QsPy/HgEkaIrnrU16A6fG5evbgdIB/8mkoiFM
mnmNlNMxt3c2tsTcaAXHpFMncVy63OC7I5v8tEqQtn7Qtzxe//EFiN8K/ioxZk
S7Uts241R+aEExi/t/0ohl3/sF1mZvT5s7b6MKWQgEQsFTcZJQxfeONmBGAs89
OKKTowQuS5vJFSpljV1ZxiWuLrUTf5fG0tXUL5IsViwFKSLMwreADu5l/LrJNi
fZb+kL5vwXIf9TSLSEG9prn0ULtWrkjoGkDLG/VFxIoD+YeZXm/j6/v7+MIzAi
EW7hY28mTeq+zixO+V0zR26Kq7ewUGi9BxVT2cA2DevfPuHKeGbRpkCyUXcZgn
L8Lu74kFdY8Prn1AmZa33X/yrXvDweaRbS4ReeN/juWNFSc1aoAQe11FwLvGJw
zdSC9acf/EJXk7J8SVhDyj7Mf5ojHk9xfIPU+pz9gzngfn9UEjXyMKD+YmN4Bc
tXxxDYMfvNdKxgG48jtw6vEGK93oV7bvPzjCHiNRxRJFFxnWUwnNfxDg8Cu+U6
nIMz61e+6m/uRrPgMPQaou6JUlH4b4zscKHodXDcqKk1Wa7Owiz84vXgZIwaOF
j8Gu/fesdKt2xIVnBYtmnK8xUXewjbWPchvqG3NEdAZTLbpFSYEurX6dcUtrVH
BoFVtSEqnEuZLgih5q/adubvnJSFUD1bBUlaq1hd+NLZX5bEapNAvDAMGQUznL
mgKlsfIm/Mw9kWBuWYHjjR+hoVSXzN/bJU2wSVtdGVCp6rFWuWbr42+Wz5jvZM
UaR1oqyfLwCaGQiqLPn4KMCK4QynwWew9nTXOfjvO2vrUnN/btSqPs6V9HDiB4
TZOpvXufw1Ztqm6rbftGNFzxGmG3fS4jGcJsc3UJua7KoyeFdOmD2N1r/pPcWy
HRZk2/mpqkTK/69eMt62/I6d1awCPsakat1scrjTJpFiEe/eT2XePgJd5Rb76s
dwvzR8NiaVOlWTPDQjrcYwcUH4Y9R/ezGKBIbHBoLM7HKis8GruqrCarLdFSwS
D4eihc4bIrYWQZ/CVO08exeJiBYLOwOB2tzaHS/eUcAIkjndUXw9eg5hw5RDdF

```
wIwOp5UkQMFBIazNx/K0IiiQpWNOsmhJNvjdNIX3peOl1BRlXbzQUXnMORoImt
doyMHlwNoYKMlncd/V+YwEKsq1WXNIA1tRvgC61Gp44NDKypje5RwOtEITDnuV
bK4KeddChnSMkwuJD73tLv7hmFVJjhNqiIiZAtqtFMEhWZTnlPKpF+AEbkKOGq
UYFqGI1Tp0IkhoUVBJ5q1JV454RanmLpX5WCvA0vKlXYvKcOISz8F7N1U6uVhF
gQQ6OZp8LFjuVplU7YCfS1CilfrwUHhbtLIw6CEwyOUCQOBo4L4WrLFxK8xhEk
EqpEzk6HdinIVQ8FklAXn+njLCBfbYlSmgeBd1ofISJSh7qimVQ6LOKp3iYKC3
d5b6Dzq6b3qsS1b1H5chX4IgaO3hNcevftoQ3dD0LAjCwSC3FV7lXEPHnJLAOJ
kHdbrCXzzqcHAKfzXAPzk7tmA+5qHW9xiwlDkeRC8w93dcT/NMFWM/ZuhGhqGq
q9cy5GHqOpRPkt37Dro5xTxms/ffPN3AOIoMfr8THVDUGLZ3qWqUd46QbBVlw/
TMr3J5Scbcw56jTMUhY16Rk6+V5n3C+rjSjs7V66sAciWPbVZg24bNZ0z5fW10
zJhnsHldcUbQrULnM3TPtMeZ83UzLzML0AM/Moyfi2j+QZZ+rpjR0zxiqKJR2l
5F2X7UoFCic0wtoSgGifI4S16hyYPy82PAvc3p5aqPe3jvwnZS5IICkSCXaQWq
PObpW8rQSoYcQ1SL7i6ejlN1O7oryNqMbYZjqpxuLpgD7AmgM7c+oIQn3Ag5HI
JrSjtNS7srhnRNYFzSBhsCCBL7kOFigHHjxTCxBy9GBDx9McoU9GKeQGwHY9aL
Je1vww9WqmjOCFppzKxxI4YMm8Cx65Ob1TxcKS5XsJdu7OSEDFjtIFd4THODA7
KPyvy6rms+PSch/qluRVgm64W6Bxv9tR792F5uVn0FM+e36dIoJ8SyjrZ7LDOy
1iJWjuqJq9q4IU1BtJPyJ6/AobxdwXJfYbLF6HH2Mytrbk4iK3M5schBsZw0XC
CHZf4KvnoK+v1ZxskrV+z3VP6S0EyV6edizV/OQo5mcqZqC/MfUiO3LMPC+kuh
b9ezGAwv8zGeNkIu5qTYkahyim7qx5Xx8pwmPZaPLp2orVgoI/Xbr3aW6RwH5f
SC4x4Hc0pI0vmrA8pJrfspaA6Z2nZanQZjmo05DOK/fQMO5I9TEZ6T3ujgbU6P
KS1HWE6hlyz9suiesMYvaSjauIKwRBzlmJWFKV6tr3RyzHOrnRTWLRP6+79Qs4
gGxwim3+TFjTzWrbg74tNJY7DkFJRU1DS0LKxsctjlyuOQr4BTYR397/XwKlKs
RKky5SpUqlItJCwiKqZOXL0GjZo0S0hKScvIatGqTbsOnRZZHBggAAU8gAE+wI
EACIEIiIEESIEMECFDhQ4TNvIoooowq6miidVV00ccQLsaYYo4l1tiSE3tykxdH
8rcnx4jL5XfxorF8eY/pQXjLi3iZHJjowRpieW9/D9xWcKJ46Rpq++XZtXpk2c
oVSCJHkMRm94v95bwHeQRo72jIG8LTvIesVPgDvQL0knOFdJET18/Fh4/MzqUx
NtaDLQtko00JBOEVG54p/soHpvA1Hc+V2OrgkZt3sRmZiMkOZyQ4JvtIsejy+p
KBoR7MR+BGzyM3Zvbc6HORK1Z8Ztjtdd+yx1sK1X+HTqb8UQB2XR0B9zee9HQU
4h9flxaG8vYrnxmX4bBweWYBptoyguP/pdOAYKIVRGdCPr6b/AEhoNAHC1i4bD
viW2hbQj+z1x/GoXkOCl1pjuYsd3gUnwM2xAt9mZdJ3/AxK07uwNK87+Di32/s
/NAp/4nX36HQCQAA) format('woff2');}";
    {
      const style = document.createElement('style');
      style.textContent = window.MATH_FONT_CSS;
      document.head.append(style);
    }
```

js/report.js

```
'use strict';
window.Report = (() => {
  const esc = (v) =>
    String(v).replace(
      /[&<>"']/g,
      (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;',
'"': '&quot;', "'": '&#39;' })[c]
    );
  const { Q, Z, O } = LP;
  const num = (q) =>
    q.d === 1n
      ? esc(q)
      : /* HTML */ `<span class="fraction"
title="${esc(q.decimal())}"
          ><span>${q.n}</span><span>${q.d}</span></span
        >`;
  const named = (s) => esc(s).replace(/^x(\d+)/, '<i
class="math-var">x</i><sub>$1</sub>');
  const A = (j) => /* HTML */ `<i class="math-
var">A</i><sub>${j + 1}</sub>`;
  const rel = (s) => ({ '<=': '≤', '>=': '≥', '=': '='
})[s];
  const domain = (s) => ({ pos: '≥ 0', neg: '≤ 0', free:
'tự do' })[s];
  const phase = (s) =>
    ({
      direct: 'Đơn hình trực tiếp',
      bigm: 'Phương pháp M',
      phase1: 'Pha I · Tìm phương án khả thi',
      phase2: 'Tối ưu hàm mục tiêu gốc',
    })[s];
  const methodName = (r) =>
    !r.kinds.includes('giả')
      ? 'Đơn hình'
      : r.method === 'two'
        ? 'Đơn hình hai pha'
        : 'Phương pháp M';
  function cost(c) {
    if (c.eta.zero()) return num(c.delta);
    const e = c.eta,
      v = e.sign() < 0 ? e.neg() : e;
    return (
```

```
          (e.sign() < 0 ? '−' : '') +
          (v.eq(O) ? '' : num(v)) +
          'M' +
          (c.delta.zero()
            ? ''
            : (c.delta.sign() > 0 ? ' + ' : ' − ') +
num(c.delta.sign() > 0 ? c.delta : c.delta.neg()))
        );
      }
      function expr(coeff, names) {
        const parts = [];
        coeff.forEach((v, j) => {
          if (v.zero()) return;
          const mag = v.sign() < 0 ? v.neg() : v;
          parts.push(
            (parts.length ? (v.sign() < 0 ? ' − ' : ' + ') :
v.sign() < 0 ? '−' : '') +
              (mag.eq(O) ? '' : num(mag) + ' ') +
              named(names[j])
          );
        });
        return parts.join('') || '0';
      }
      const cn = (q) =>
        q.zero() ? '' : (q.sign() > 0 ? ' + ' : ' − ') +
num(q.sign() > 0 ? q : q.neg());
      const vector = (x) =>
        /* HTML */ `<span class="vector"
          >(${x.map(num).join('<span
class="separator">;</span>')})</span
        >`;
      const originals = (n) => Array.from({ length: n }, (_,
j) => 'x' + (j + 1));
      // Đề bài gốc
      function problem(p) {
        return /* HTML */ `<section class="solution-section">
          <h2>Bài toán ban đầu</h2>
          <div class="math-block">
            <div class="objective-line">
              f(x) = ${expr(p.c,
originals(p.n))}${cn(p.constant)} <span class="arrow">→</span>
              <b>${p.sense.toUpperCase()}</b>
            </div>
            <div class="constraints-list">
```

```
              ${p.a.length
                ? p.a
                    .map(
                      (row, i) =>
                        /* HTML */ `<div>
                          <span class="row-tag">R${i +
1}</span>${expr(row, originals(p.n))}
                          ${rel(p.rel[i])} ${num(p.b[i])}
                        </div>`
                    )
                    .join('')
                : '<div>Không có ràng buộc tuyến tính bổ
sung.</div>'}
            </div>
            <div class="domain-line">
              ${p.domains
                .map((d, j) => named('x' + (j + 1)) + ' ' +
domain(d))
                .join('<span class="separator">;</span>')}
            </div>
          </div>
        </section>`;
      }
      // Dạng chính tắc và cơ sở xuất phát
      function normalization(r) {
        const first = r.steps[0],
          full = LP.zeros(r.names.length);
        if (first) first.basis.forEach((k, i) => (full[k] =
first.b[i]));
        const artificial = r.kinds.map((k, j) => (k === 'giả'
? j : -1)).filter((j) => j >= 0);
        const substitutions = r.p.domains.flatMap((d, j) =>
          d === 'free'
            ? [
                `${named('x' + (j + 1))} = ${named('x' + (j +
1) + '+')} − ${named('x' + (j + 1) + '−')}`,
              ]
            : d === 'neg'
              ? [`${named('x' + (j + 1))} = −${named('x' + (j
+ 1) + '’')}`]
              : []
        );
        return /* HTML */ `<section class="solution-section">
          <h2>Dạng chính tắc</h2>
```

```
          ${substitutions.length
            ? /* HTML */ `<p class="math
substitution">${substitutions.join(';   ')}</p>`
            : ''}
          <div class="math-block">
            <div class="objective-line">
              f${r.method === 'bigm' && artificial.length ?
'<sub>M</sub>' : ''}(x) =
              ${expr(r.objective,
r.names)}${cn(r.p.constant)}${r.method === 'bigm'
                ? artificial
                    .map((j) => (r.p.sense === 'max' ? ' − M'
: ' + M') + named(r.names[j]))
                    .join('')
                : ''}
              → ${r.p.sense.toUpperCase()}
            </div>
            ${r.canonical
              .map((row, i) => /* HTML */ `<div>${expr(row,
r.names)} = ${num(r.rhs[i])}</div>`)
              .join('')}
            <div class="domain-line">Tất cả các biến trong hệ
trên ≥ 0.</div>
          </div>
          ${r.method === 'two' && artificial.length
            ? /* HTML */ `<p class="math">
                Pha I: W = ${artificial.map((j) =>
named(r.names[j])).join(' + ')} → MIN.
              </p>`
            : ''}${first
            ? /* HTML */ `<div class="initial-basis">
                <p>
                  Cơ sở xuất phát: <span class="math">J = {
${first.basis.map(A).join(', ')} }</span>
                </p>
                <p class="math">x<sup>(0)</sup> =
${vector(full)}</p>
              </div>`
            : ''}
        </section>`;
      }
      // Bảng đơn hình và thông tin biến vào / ra
      function step(r, index) {
        const t = r.steps[index],
```

```
          big = t.cost.some((c) => !c.eta.zero());
        const table = /* HTML */ `<div
          class="table-wrap"
          tabindex="0"
          aria-label="Bảng đơn hình ${index + 1}, cuộn ngang
để xem đủ"
        >
          <table class="tableau">
            <thead>
              <tr>
                <th rowspan="2">Hệ số<br />c<sub>B</sub></th>
                <th rowspan="2">Cơ sở</th>
                <th rowspan="2" class="rhs">Phương án</th>
                ${t.cost
                  .map(
                    (c, j) =>
                      /* HTML */ `<th class="${j === t.enter
? 'enter-col' : ''}">${cost(c)}</th>`
                  )
                  .join('')}
              </tr>
              <tr>
                ${r.names
                  .map(
                    (n, j) => /* HTML */ `<th class="${j ===
t.enter ? 'enter-col' : ''}">${A(j)}</th>`
                  )
                  .join('')}
              </tr>
            </thead>
            <tbody>
              ${t.a
                .map(
                  (row, i) =>
                    /* HTML */ `<tr class="${i === t.leave ?
'leaving-row' : ''}">
                      <td>${cost(t.cost[t.basis[i]])}</td>

<td><strong>${A(t.basis[i])}</strong></td>
                      <td class="rhs">${num(t.b[i])}</td>
                      ${row
                        .map(
                          (v, j) =>
                            /* HTML */ `<td
```

```
                                class="${!t.active[j]
                                  ? 'inactive'
                                  : i === t.leave && j ===
t.enter
                                    ? 'pivot'
                                    : j === t.enter
                                      ? 'enter-col'
                                      : ''}"
                                >
                                  ${t.active[j] ? num(v) : '×'}
                                </td>`
                            )
                            .join('')}
                        </tr>`
                    )
                    .join('')}
            </tbody>
            <tfoot>
              ${big
                ? /* HTML */ `<tr>
                      <th colspan="2">δ<sub>j</sub></th>
                      <td>${num(t.value.delta)}</td>
                      ${t.delta
                        .map((v, j) => /* HTML */
`<td>${t.active[j] ? num(v.delta) : '×'}</td>`)
                        .join('')}
                    </tr>
                    <tr>
                      <th colspan="2">η<sub>j</sub></th>
                      <td>${num(t.value.eta)}</td>
                      ${t.delta
                        .map((v, j) => /* HTML */
`<td>${t.active[j] ? num(v.eta) : '×'}</td>`)
                        .join('')}
                    </tr>`
                : /* HTML */ `<tr>
                    <th colspan="2">Δ<sub>j</sub> =
z<sub>j</sub> − c<sub>j</sub></th>
                    <td>${t.phase === 'phase1' ? 'W' : 'f'} =
${cost(t.value)}</td>
                    ${t.delta
                      .map((v, j) => /* HTML */
`<td>${t.active[j] ? cost(v) : '×'}</td>`)
                      .join('')}
```

```
                  </tr>`}
              </tfoot>
            </table>
          </div>`;
          const pivot =
            t.enter >= 0 && t.leave >= 0
              ? /* HTML */ `<p class="pivot-line">
                  <span class="math">${A(t.enter)}</span> vào,
                  <span
class="math">${A(t.basis[t.leave])}</span> ra.
                </p>`
              : '';
          return /* HTML */ `<section class="solution-section
tableau-section">
            <div class="step-heading">
              <h2>Bảng ${index + 1}</h2>
              <span class="badge">${phase(t.phase)}</span>
            </div>
            ${table}${pivot}
          </section>`;
        }
        const status = (r) =>
          ({
            optimal: 'Nghiệm tối ưu',
            infeasible: 'Bài toán vô nghiệm',
            unbounded: 'Hàm mục tiêu không bị chặn',
            limit: 'Chưa có kết luận',
          })[r.status];
        // Kết luận và kiểm tra nghiệm
        function conclusion(r) {
          return /* HTML */ `<section class="solution-section">
            <h2>${status(r)}</h2>
            <div class="notice ${r.status === 'optimal' ?
'success' : 'warning'}">
              <p>${esc(r.conclusion)}</p>
              ${r.x
                ? /* HTML */ `<p class="answer math">
                    ${r.status === 'optimal' ? 'x*' : 'x⁰'} =
${vector(r.x)}<br />f(${r.status ===
                    'optimal'
                      ? 'x*'
                      : 'x⁰'})
                    = ${num(r.value)}
                  </p>`
```

```
              : ''}
          </div>
          ${r.status === 'optimal'
            ? /* HTML */ `<p>
                ${{
                  unique: 'Nghiệm tối ưu duy nhất.',
                  multiple: 'Bài toán có vô số nghiệm tối
ưu.',
                  unknown: 'Chưa kết luận tính duy nhất của
nghiệm tối ưu.',
                }[r.multiplicity]}
              </p>`
            : ''}${r.alternate
            ? /* HTML */ `<div class="math-block">
                Một nghiệm tối ưu khác: y* =
${vector(r.alternate)}<br />x(λ) = (1 − λ)x* + λy*,
                 0 ≤ λ ≤ 1.
              </div>
              <p class="muted">Một họ nghiệm tối ưu với 0 ≤
λ ≤ 1.</p>`
            : ''}${r.ray
            ? /* HTML */ `<div class="math-block">
                d = ${vector(r.ray)}<br />x(t) = x⁰ + t·d,
 t ≥ 0<br />f(x(t)) = ${num(r.value)} +
                t·(${num(LP.value(r.p,
r.ray).sub(r.p.constant))}) →
                ${r.p.sense === 'max' ? '+∞' : '−∞'}
              </div>`
            : ''}${r.x
            ? /* HTML */ `<h3>Phương án đầy đủ (theo thứ tự
cột)</h3>
                <p class="math">${vector(r.full)}</p>`
            : ''}
        </section>`;
      }
      // Ghép toàn bộ lời giải và xuất HTML độc lập
      function all(r) {
        return /* HTML */ `<div class="report-cover">
            <h1>${esc(r.p.title)}</h1>
            <p>${methodName(r)} · ${r.pivots} lần đổi cơ
sở</p>
          </div>
          ${problem(r.p)}${normalization(r)}${r.steps.map((s,
i) => step(r, i)).join('')}${conclusion(
```

```
            r
          )}`;
      }
      function standalone(r, css) {
        return (
          '<!doctype html><html lang="vi"><head><meta
charset="UTF-8"><meta name="viewport" content="width=device-
width,initial-scale=1"><title>Lời giải đơn
hình</title><style>' +
          (window.MATH_FONT_CSS || '') +
          css +
          '</style></head><body class="exported"><main
class="report">' +
          all(r) +
          '</main></body></html>'
        );
      }
      // A compact, independent print theme keeps exported
HTML self-contained on file://.
      const exportCSS = /* CSS */ `
        * {
          box-sizing: border-box;
        }
        body {
          font:
            15px Arial,
            sans-serif;
          color: #303030;
          line-height: 1.65;
          margin: 0;
          background: #f5f5f5;
        }
        .report {
          max-width: 1200px;
          margin: 24px auto;
          background: white;
          padding: 30px 40px;
        }
        .solution-section {
          padding: 22px 0;
          border-bottom: 1px solid #ddd;
        }
        h1 {
          font-size: 25px;
```

```
  font-weight: 500;
}
h2 {
  font-size: 19px;
  margin: 0 0 14px;
  font-weight: 600;
}
h3 {
  font-size: 15px;
  margin-top: 24px;
}
.section-label {
  font-size: 10px;
  letter-spacing: 1px;
  color: #888;
}
.muted,
small,
.report-footer {
  color: #777;
  font-size: 12px;
}
.report-footer {
  padding-top: 20px;
}
.table-wrap {
  overflow: auto;
  margin: 14px 0;
}
table {
  border-collapse: collapse;
  white-space: nowrap;
  width: 100%;
  text-align: center;
  font-variant-numeric: tabular-nums;
}
th,
td {
  border: 1px solid #d8d8d8;
  padding: 10px 12px;
}
th {
  background: #f4f4f4;
  font:
```

```
    12px Arial,
    sans-serif;
}
td {
  font:
    17px 'Computer Modern',
    Cambria,
    serif;
}
thead small {
  display: block;
}
tfoot {
  background: #edf3fc;
}
.rhs {
  border-right: 2px solid #ccc;
}
.enter-col {
  background: #eef4fd;
}
.leaving-row td {
  background: #fdf1ed;
}
.leaving-row .enter-col {
  background: #e3edfb;
}
.pivot {
  background: #2d70b3 !important;
  color: white;
  font-weight: bold;
}
.inactive {
  background: #fafafa;
  color: #aaa;
}
.fraction {
  display: inline-flex;
  flex-direction: column;
  vertical-align: middle;
  text-align: center;
  line-height: 1.2;
  font-family: 'Computer Modern', Cambria, serif;
  margin: 0 0.15em;
```

```
          min-width: 1em;
        }
        .fraction > span:first-child {
          border-bottom: 1px solid currentColor;
          padding: 0 0.12em 0.12em;
        }
        .fraction > span:last-child {
          padding: 0.12em 0.12em 0;
        }
        .separator {
          margin: 0 0.45em;
        }
        .vector {
          white-space: nowrap;
        }
        .math,
        .math-block {
          font:
            19px 'Computer Modern',
            Cambria,
            serif;
          line-height: 2;
        }
        .math-var {
          font-family: 'Computer Modern Math', Cambria,
serif;
          font-style: italic;
        }
        .math-block {
          padding: 12px 0;
          margin: 10px 0;
          overflow-x: auto;
        }
        .objective-line {
          font-size: 21px;
          margin-bottom: 9px;
        }
        .row-tag {
          font: 10px Arial;
          color: #999;
          margin-right: 15px;
        }
        .domain-line {
          margin-top: 10px;
```

```
  font: 13px Arial;
}
.step-heading {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
}
.badge {
  font-size: 11px;
  color: #888;
}
.pivot-line,
.table-note {
  font-size: 13px;
}
.notice {
  border-left: 3px solid #2d70b3;
  background: #f1f6fd;
  padding: 14px 18px;
  margin: 16px 0;
}
.notice.warning {
  border-color: #c74440;
  background: #fff5f4;
}
.answer {
  font-size: 25px;
}
details {
  border-top: 1px solid #ddd;
  padding-top: 12px;
  margin-top: 18px;
}
summary {
  cursor: pointer;
  color: #2d70b3;
  font-size: 12px;
}
.check {
  color: #388c46;
}
.report sub {
  line-height: 0;
  font-size: 0.7em;
```

```
}
@media (max-width: 700px) {
  .report {
    margin: 0;
    padding: 18px;
  }
  .step-heading {
    display: block;
  }
}
@media print {
  @page {
    size: A4 landscape;
    margin: 12mm;
  }
  body {
    background: #fff;
    font-size: 10pt;
  }
  .report {
    margin: 0;
    padding: 0;
    max-width: none;
  }
  .solution-section {
    padding: 14px 0;
  }
  h2,
  h3 {
    break-after: avoid;
  }
  table {
    font-size: 9pt;
  }
  th,
  td {
    padding: 6px 5px;
  }
  td {
    font-size: 12pt;
  }
  thead {
    display: table-header-group;
  }
```

```
      tfoot {
        display: table-row-group;
      }
      tr,
      .notice {
        break-inside: avoid;
      }
      .table-wrap,
      .math-block {
        overflow: visible;
      }
      details > * {
        display: block !important;
      }
      * {
        -webkit-print-color-adjust: exact;
        print-color-adjust: exact;
      }
    }
  `;
  return {
    esc,
    num,
    named,
    A,
    rel,
    domain,
    phase,
    methodName,
    cost,
    expr,
    vector,
    problem,
    normalization,
    step,
    conclusion,
    status,
    all,
    standalone,
    exportCSS,
  };
})();
```

js/session.js

```
    // Lưu đề và các tab; không lưu HTML hay tự chạy lại bộ
giải.
    'use strict';
    window.WorkspaceSession = (() => {
      const KEY = 'simplex.workspace.v1';
      const isText = (value) => typeof value === 'string';
      const textArray = (value, size) =>
        Array.isArray(value) && value.length === size &&
value.every(isText);
      const validId = (value) =>
        Number.isSafeInteger(value) && value > 0 && value <
Number.MAX_SAFE_INTEGER - 1;
      function draft(tab) {
        const p = tab?.raw;
        if (
          !validId(tab?.id) ||
          !p ||
          !isText(p.title) ||
          !['max', 'min'].includes(p.sense) ||
          !isText(p.constant) ||
          !Array.isArray(p.c) ||
          p.c.length < 1 ||
          p.c.length > 20 ||
          !textArray(p.c, p.c.length) ||
          !textArray(p.domains, p.c.length) ||
          !p.domains.every((v) => ['pos', 'neg',
'free'].includes(v)) ||
          !Array.isArray(p.a) ||
          p.a.length > 40 ||
          !p.a.every((row) => textArray(row, p.c.length)) ||
          !textArray(p.b, p.a.length) ||
          !textArray(p.rel, p.a.length) ||
          !p.rel.every((v) => ['<=', '=', '>='].includes(v))
||
          !['bigm', 'two', 'geometry'].includes(tab.method)
        )
          throw new Error('Dữ liệu phiên đã lưu không hợp
lệ.');
        return {
          id: tab.id,
          raw: {
            title: p.title,
```

```
            sense: p.sense,
            constant: p.constant,
            c: p.c,
            domains: p.domains,
            a: p.a,
            b: p.b,
            rel: p.rel,
          },
          method: tab.method,
          dimensions: textArray(tab.dimensions, 2)
            ? tab.dimensions
            : [String(p.c.length), String(p.a.length)],
          scroll:
            Array.isArray(tab.scroll) &&
            tab.scroll.length === 2 &&
            tab.scroll.every((v) => Number.isFinite(v) && v
>= 0)
              ? tab.scroll
              : [0, 0],
          lastExampleIndex:
Number.isInteger(tab.lastExampleIndex) ? tab.lastExampleIndex
: -1,
        };
      }
      function decode(data) {
        if (
          data?.version !== 1 ||
          data.enabled !== true ||
          !Array.isArray(data.tabs) ||
          !data.tabs.length ||
          !Array.isArray(data.closedTabs) ||
          data.closedTabs.length > 10
        )
          throw new Error('Không đọc được phiên làm việc đã
lưu.');
        const tabs = data.tabs.map(draft),
          closedTabs = data.closedTabs.map(draft);
        const ids = [...tabs, ...closedTabs].map((tab) =>
tab.id);
        if (new Set(ids).size !== ids.length ||
!tabs.some((tab) => tab.id === data.activeTabId))
          throw new Error('Danh sách tab đã lưu không hợp
lệ.');
        return { tabs, closedTabs, activeTabId:
```

```
data.activeTabId, nextTabId: Math.max(...ids) + 1 };
      }
      function load(storage = localStorage) {
        const value = storage.getItem(KEY);
        return value === null ? null :
decode(JSON.parse(value));
      }
      function save(tabs, closedTabs, activeTabId, storage =
localStorage) {
        const data = {
          version: 1,
          enabled: true,
          tabs: tabs.map(draft),
          closedTabs: closedTabs.map(draft),
          activeTabId,
        };
        storage.setItem(KEY, JSON.stringify(data));
      }
      function clear(storage = localStorage) {
        storage.removeItem(KEY);
      }
      return { load, save, clear };
    })();
```

js/solver.js

```
    /* Exact simplex engine. Browser JavaScript only; no
packages, network or build step.
     * installLP is self-contained so the UI can run the
engine inside a Blob Worker.
     */
    function installLP(scope) {
      'use strict';
      class Rational {
        constructor(n, d = 1n) {
          n = BigInt(n);
          d = BigInt(d);
          if (!d) throw new Error('Mẫu số phải khác 0.');
          if (d < 0n) {
            n = -n;
            d = -d;
          }
          let a = n < 0n ? -n : n,
            b = d;
          while (b) [a, b] = [b, a % b];
          this.n = n / a;
          this.d = d / a;
          Object.freeze(this);
        }
        static parse(value) {
          if (value instanceof Rational) return value;
          const s = String(value).trim().replace(/−/g, '-
').replace(/,/g, '.');
          if (!s || s.length > 100) throw new Error('Hệ số
phải có 1–100 ký tự.');
          if (s.includes('/')) {
            const p = s.split('/');
            if (p.length !== 2) throw new Error('Phân số
không hợp lệ.');
            return
Rational.parse(p[0]).div(Rational.parse(p[1]));
          }
          const match = s.match(/^([+-
]?)(\d+(?:\.\d*)?|\.\d+)(?:[eE]([+-]?\d+))?$/);
          if (!match) throw new Error('Số không hợp lệ: ' + s
+ '. Ví dụ: -2, 0.5, 3/7.');
          const exp = Number(match[3] || 0),
            parts = match[2].split('.');
```

```
          const scale = (parts[1] || '').length - exp;
          if (!Number.isSafeInteger(scale) || Math.abs(scale)
> 50)
            throw new Error('Số mũ/thang thập phân tối đa
50.');
          let num = BigInt(parts.join('')) * (match[1] === '-
' ? -1n : 1n);
          return scale >= 0
            ? new Rational(num, 10n ** BigInt(scale))
            : new Rational(num * 10n ** BigInt(-scale));
        }
        add(b) {
          return new Rational(this.n * b.d + b.n * this.d,
this.d * b.d);
        }
        sub(b) {
          return this.add(b.neg());
        }
        mul(b) {
          return new Rational(this.n * b.n, this.d * b.d);
        }
        div(b) {
          return new Rational(this.n * b.d, this.d * b.n);
        }
        neg() {
          return new Rational(-this.n, this.d);
        }
        sign() {
          return this.n < 0n ? -1 : this.n > 0n ? 1 : 0;
        }
        zero() {
          return this.n === 0n;
        }
        cmp(b) {
          const v = this.n * b.d - b.n * this.d;
          return v < 0n ? -1 : v > 0n ? 1 : 0;
        }
        eq(b) {
          return this.n === b.n && this.d === b.d;
        }
        toString() {
          return this.d === 1n ? String(this.n) : this.n +
'/' + this.d;
        }
```

```
        decimal() {
          const sign = this.n < 0n ? '–' : '',
            a = this.n < 0n ? -this.n : this.n;
          const rounded = ((a * 100000000n * 2n) / this.d +
1n) / 2n;
          const whole = rounded / 100000000n,
            frac = String(rounded % 100000000n)
              .padStart(8, '0')
              .replace(/0+$/, '');
          return sign + whole + (frac ? ',' + frac : '');
        }
        toJSON() {
          return { $q: this.toString() };
        }
      }
      const Q = Rational,
        Z = new Q(0n),
        O = new Q(1n);
      const zeros = (n) => Array(n).fill(Z),
        copy = (a) => a.map((r) => r.slice());
      class Cost {
        constructor(eta = Z, delta = Z) {
          this.eta = eta;
          this.delta = delta;
        }
        add(b) {
          return new Cost(this.eta.add(b.eta),
this.delta.add(b.delta));
        }
        sub(b) {
          return new Cost(this.eta.sub(b.eta),
this.delta.sub(b.delta));
        }
        mul(b) {
          return new Cost(this.eta.mul(b),
this.delta.mul(b));
        }
        cmp(b) {
          return this.eta.cmp(b.eta) ||
this.delta.cmp(b.delta);
        }
        toString() {
          if (this.eta.zero()) return this.delta.toString();
          const m = this.eta.eq(O) ? 'M' :
```

```
this.eta.eq(O.neg()) ? '−M' : this.eta + 'M';
          return (
            m +
            (this.delta.zero()
              ? ''
              : (this.delta.sign() > 0 ? ' + ' : ' − ') +
                (this.delta.sign() > 0 ? this.delta :
this.delta.neg()))
          );
        }
        toJSON() {
          return { $cost: [this.eta, this.delta] };
        }
      }
      const CZ = new Cost();
      function revive(key, value) {
        if (value && typeof value === 'object') {
          if ('$q' in value) return Q.parse(value.$q);
          if ('$cost' in value) return new
Cost(...value.$cost);
        }
        return value;
      }
      function parseProblem(raw) {
        if (!raw || !Array.isArray(raw.c) ||
!Array.isArray(raw.a))
          throw new Error('Dữ liệu bài toán không hợp lệ.');
        const n = raw.c.length,
          m = raw.a.length;
        if (n < 1 || n > 20 || m > 60) throw new Error('Hỗ
trợ 1–20 biến và tối đa 60 ràng buộc.');
        if (!['max', 'min'].includes(raw.sense)) throw new
Error('Chiều tối ưu phải là max hoặc min.');
        if (
          !Array.isArray(raw.b) ||
          raw.b.length !== m ||
          !Array.isArray(raw.rel) ||
          raw.rel.length !== m ||
          !Array.isArray(raw.domains) ||
          raw.domains.length !== n
        )
          throw new Error('Kích thước dữ liệu không khớp.');
        if (
          raw.rel.some((s) => !['<=', '=', '>='].includes(s))
```

```
||
          raw.domains.some((s) => !['pos', 'neg',
'free'].includes(s))
        )
          throw new Error('Quan hệ hoặc miền biến không hợp
lệ.');
        const p = {
          n,
          m,
          title: String(raw.title || 'Bài toán quy hoạch
tuyến tính').slice(0, 150),
          sense: raw.sense,
          c: raw.c.map(Q.parse),
          constant: Q.parse(raw.constant ?? '0'),
          b: raw.b.map(Q.parse),
          rel: raw.rel.slice(),
          domains: raw.domains.slice(),
          a: raw.a.map((r, i) => {
            if (!Array.isArray(r) || r.length !== n)
              throw new Error('Sai số hệ số ở ràng buộc ' +
(i + 1));
            return r.map(Q.parse);
          }),
        };
        return p;
      }
      function rawProblem(p) {
        return {
          title: p.title,
          sense: p.sense,
          c: p.c.map(String),
          constant: String(p.constant),
          a: p.a.map((r) => r.map(String)),
          b: p.b.map(String),
          rel: p.rel.slice(),
          domains: p.domains.slice(),
        };
      }
      function value(p, x) {
        return p.c.reduce((v, c, j) => v.add(c.mul(x[j])),
p.constant);
      }
      function feasible(p, x) {
        if (x.length !== p.n) return false;
```

```
        if (
          x.some(
            (v, j) =>
              (p.domains[j] === 'pos' && v.sign() < 0) ||
(p.domains[j] === 'neg' && v.sign() > 0)
          )
        )
          return false;
        return p.a.every((row, i) => {
          const s = row.reduce((v, a, j) =>
v.add(a.mul(x[j])), Z).cmp(p.b[i]);
          return p.rel[i] === '=' ? s === 0 : p.rel[i] ===
'<=' ? s <= 0 : s >= 0;
        });
      }
      const same = (a, b) => a.every((v, i) => v.eq(b[i]));
      class Limit extends Error {}
      class Solver {
        solve(raw, method = 'bigm', alternatives = true,
options = {}) {
          this.p = raw.n !== undefined && raw.c[0] instanceof
Q ? raw : parseProblem(raw);
          this.method = method;
          this.record = options.record !== false;
          if (!['bigm', 'two'].includes(method)) throw new
Error('Phương pháp không hợp lệ.');
          this.deadline = options.deadline || Date.now() +
45000;
          this.pivots = 0;
          this.bland = false;
          this.visited = new Set();
          this.out = {
            p: this.p,
            method,
            status: null,
            multiplicity: 'unknown',
            notes: [],
            names: [],
            kinds: [],
            steps: [],
            pivots: 0,
            conclusion: '',
          };
          const r = this.out;
```

```
          try {
            this.normalize();
            const hasArt = this.art.some(Boolean);
            if (hasArt) {
              this.phase = method === 'two' ? 'phase1' :
'bigm';
              this.max = method === 'two' ? false :
this.p.sense === 'max';
              this.cost = this.regularCosts();
              for (let j = 0; j < this.N; j++)
                this.cost[j] =
                  method === 'two'
                    ? new Cost(Z, this.art[j] ? O : Z)
                    : this.art[j]
                      ? new Cost(this.max ? O.neg() : O, Z)
                      : this.cost[j];
              if (this.iterate(true, method === 'bigm') ===
'infeasible') {
                r.status = 'infeasible';
                r.artificialSum = this.artSum();
                r.conclusion =
                  'Tổng biến giả nhỏ nhất bằng ' +
                  r.artificialSum +
                  ' > 0. Miền ràng buộc của bài toán gốc
rỗng.';
                return r;
              }
              this.cleanup();
            }
            this.phase = hasArt ? 'phase2' : 'direct';
            this.cost = this.regularCosts();
            this.max = this.p.sense === 'max';
            this.visited.clear();
            r.status = this.iterate(false, false);
            r.full = this.current();
            r.x = this.original(r.full);
            r.value = value(this.p, r.x);
            if (!feasible(this.p, r.x))
              throw new Error('Kiểm tra nội bộ: phương án
không thỏa bài toán gốc.');
            if (r.status === 'unbounded') {
              r.conclusion =
                'Có phương án khả thi nhưng hàm mục tiêu
không bị chặn ' +
```

```
              (this.max ? 'trên (+∞).' : 'dưới (−∞).') +
              ' Không có nghiệm tối ưu hữu hạn.';
            return r;
          }
          r.conclusion =
            'Mọi ước lượng Δⱼ trên các cột đang xét đều ' +
            (this.max ? '≥ 0' : '≤ 0') +
            '. Phương án hiện tại tối ưu.';
          if (alternatives) this.alternatives();
        } catch (e) {
          if (!(e instanceof Limit)) throw e;
          r.status = 'limit';
          r.conclusion = e.message + ' Chưa kết luận tối
ưu, vô nghiệm hay không bị chặn.';
        }
        return r;
      }
      check() {
        if (Date.now() > this.deadline) throw new Limit('Đã
chạm giới hạn 45 giây.');
        if (this.pivots >= 600) throw new Limit('Đã chạm
giới hạn 600 phép đổi cơ sở.');
      }
      normalize() {
        const p = this.p,
          r = this.out,
          origins = [],
          signs = [];
        const add = (name, kind) => {
          r.names.push(name);
          r.kinds.push(kind);
        };
        p.domains.forEach((d, j) => {
          if (d === 'free') {
            add('x' + (j + 1) + '⁺', 'tách');
            origins.push(j);
            signs.push(O);
            add('x' + (j + 1) + '⁻', 'tách');
            origins.push(j);
            signs.push(O.neg());
            r.notes.push(
              'Biến tự do x' +
                (j + 1) +
                ': đặt x' +
```

```
                  (j + 1) +
                  ' = x' +
                  (j + 1) +
                  '+ – x' +
                  (j + 1) +
                  '–; cả hai biến mới ≥ 0.'
              );
            } else {
              add('x' + (j + 1) + (d === 'neg' ? '’' : ''), d
=== 'neg' ? 'đổi dấu' : 'gốc');
              origins.push(j);
              signs.push(d === 'neg' ? O.neg() : O);
              if (d === 'neg')
                r.notes.push('Đặt x' + (j + 1) + ' = –x' + (j
+ 1) + '’ với x' + (j + 1) + '’ ≥ 0.');
            }
          });
          const structural = origins.length,
            rel = p.rel.slice(),
            factor = p.b.map((b) => (b.sign() < 0 ? O.neg() :
O));
          this.b = p.b.map((b, i) => {
            if (b.sign() < 0) {
              rel[i] = rel[i] === '<=' ? '>=' : rel[i] ===
'>=' ? '<=' : '=';
              r.notes.push(
                'Ràng buộc ' +
                  (i + 1) +
                  ': nhân hai vế với –1, quan hệ mới ' +
                  rel[i] +
                  ', vế phải = ' +
                  b.neg() +
                  '.'
              );
              return b.neg();
            }
            return b;
          });
          const slack = Array(p.m).fill(-1);
          rel.forEach((s, i) => {
            if (s !== '=') {
              slack[i] = r.names.length;
              add('x' + (r.names.length + 1), s === '<=' ?
'phụ' : 'dư');
```

```
            r.notes.push(
              'Ràng buộc ' +
                (i + 1) +
                ': ' +
                (s === '<=' ? 'cộng ' : 'trừ ') +
                r.names[slack[i]] +
                ' ≥ 0 để chuyển sang đẳng thức.'
            );
          }
        });
        const regular = r.names.length;
        this.basis = Array(p.m).fill(-1);
        const base = p.a.map((row, i) => {
          const a = zeros(regular);
          origins.forEach((j, k) => (a[k] =
row[j].mul(signs[k]).mul(factor[i])));
          if (slack[i] >= 0) {
            a[slack[i]] = rel[i] === '<=' ? O : O.neg();
            if (rel[i] === '<=') this.basis[i] = slack[i];
          }
          return a;
        });
        for (let j = 0; j < regular; j++) {
          let row = -1,
            unit = true;
          for (let i = 0; i < p.m; i++)
            if (!base[i][j].zero()) {
              if (base[i][j].eq(O) && row < 0) row = i;
              else {
                unit = false;
                break;
              }
            }
          if (unit && row >= 0 && this.basis[row] < 0)
this.basis[row] = j;
        }
        const artCols = [];
        this.basis.forEach((col, i) => {
          if (col < 0) {
            this.basis[i] = r.names.length;
            artCols.push(r.names.length);
            add('x' + (r.names.length + 1), 'giả');
            r.notes.push(
              'Ràng buộc ' +
```

```
                (i + 1) +
                ': thêm biến giả ' +
                r.names[this.basis[i]] +
                ' ≥ 0 để tạo cột đơn vị.'
            );
          }
        });
        this.N = r.names.length;
        this.c = zeros(this.N);
        this.map = Array.from({ length: p.n }, () =>
zeros(this.N));
        origins.forEach((j, k) => {
          this.c[k] = p.c[j].mul(signs[k]);
          this.map[j][k] = signs[k];
        });
        this.a = base.map((row, i) => {
          const a = zeros(this.N);
          row.forEach((v, j) => (a[j] = v));
          if (this.basis[i] >= regular) a[this.basis[i]] =
O;
          return a;
        });
        this.art = Array(this.N).fill(false);
        artCols.forEach((j) => (this.art[j] = true));
        this.active = Array(this.N).fill(true);
        r.canonical = copy(this.a);
        r.rhs = this.b.slice();
        r.objective = this.c.slice();
        r.notes.push(
          'Tất cả các biến trong bảng đều không âm. Biến
ngoài cơ sở bằng 0. Cột A_j tương ứng với biến ghi dưới tên
cột.'
        );
        r.notes.push(
          'Giữ chiều ' +
            p.sense.toUpperCase() +
            '. Dùng Δ_j = Σ c^B_i a_ij − c_j: ' +
            (p.sense === 'max'
              ? 'MAX tối ưu khi Δ_j ≥ 0; chọn Δ_j âm nhỏ
nhất.'
              : 'MIN tối ưu khi Δ_j ≤ 0; chọn Δ_j dương lớn
nhất.')
        );
        if (artCols.length)
```

```
            r.notes.push(
              this.method === 'bigm'
                ? 'Phạt biến giả bằng ' +
                    (p.sense === 'max' ? '−M' : '+M') +
                    '. Với Δⱼ = δⱼ + ηⱼM, so sánh ηⱼ trước rồi δⱼ. M là ký hiệu, không thay bằng số hữu hạn.'
                : 'Pha I: W = tổng biến giả → MIN. Nếu W* > 0: vô nghiệm. Nếu W* = 0: khử biến giả rồi tối ưu f ở pha II.'
            );
        }
        regularCosts() {
          return this.c.map((c) => new Cost(Z, c));
        }
        artSum() {
          return this.b.reduce((s, b, i) =>
(this.art[this.basis[i]] ? s.add(b) : s), Z);
        }
        deltas() {
          return this.cost.map((c, j) =>
            this.basis.reduce((z, k, i) =>
z.add(this.cost[k].mul(this.a[i][j])), CZ).sub(c)
          );
        }
        objective() {
          return this.basis.reduce(
            (z, k, i) => z.add(this.cost[k].mul(this.b[i])),
            new Cost(Z, this.phase === 'phase1' ? Z :
this.p.constant)
          );
        }
        improving(c) {
          return this.max ? c.cmp(CZ) < 0 : c.cmp(CZ) > 0;
        }
        iterate(feasibility, bigm) {
          while (true) {
            this.check();
            this.validateBasis();
            const key = this.basis.join(',');
            if (this.visited.has(key)) this.bland = true;
            this.visited.add(key);
            const d = this.deltas();
            if (feasibility && this.artSum().zero()) {
              if (!bigm || this.basis.some((j) =>
this.art[j]))
```

```
                this.capture(
                  d,
                  -1,
                  -1,
                  'Tổng biến giả bằng 0. Đã tìm được phương
án khả thi của bài toán gốc.'
                );
              return 'optimal';
            }
            if (
              feasibility &&
              bigm &&
              !d.some(
                (v, j) =>
                  this.active[j] && !this.art[j] && (this.max
? v.eta.sign() < 0 : v.eta.sign() > 0)
              )
            ) {
              this.capture(
                d,
                -1,
                -1,
                'Không thể giảm tiếp tổng biến giả; tổng biến
giả = ' +
                  this.artSum() +
                  ' > 0. Bài toán gốc vô nghiệm.'
              );
              return 'infeasible';
            }
            let s = -1;
            for (let j = 0; j < this.N; j++)
              if (this.active[j] && !this.art[j] &&
this.improving(d[j])) {
                if (s < 0 || (!this.bland && (this.max ?
d[j].cmp(d[s]) < 0 : d[j].cmp(d[s]) > 0)))
                  s = j;
                if (this.bland) break;
              }
            if (s < 0) {
              if (feasibility && !this.artSum().zero()) {
                this.capture(d, -1, -1, 'W* = ' +
this.artSum() + ' > 0. Bài toán vô nghiệm.');
                return 'infeasible';
              }
```

```
            this.capture(
              d,
              -1,
              -1,
              'Các Δ_j đang xét đều ' +
                (this.max ? '≥ 0' : '≤ 0') +
                '. Điều kiện tối ưu được thỏa mãn.'
            );
            return 'optimal';
          }
          const row = this.leaving(s);
          if (row < 0) {
            if (feasibility) throw new Error('Lỗi nội bộ:
hàm phụ không thể không bị chặn.');
            const ray = zeros(this.N);
            ray[s] = O;
            this.basis.forEach((k, i) => (ray[k] =
this.a[i][s].neg()));
            this.out.ray = this.original(ray);
            this.capture(
              d,
              s,
              -1,
              'Cột A' +
                (s + 1) +
                ' cải thiện mục tiêu nhưng mọi a_is ≤ 0.
Không có dòng ra; θ không bị chặn.'
            );
            return 'unbounded';
          }
          const theta = this.b[row].div(this.a[row][s]);
          const note =
            'Đưa A' +
            (s + 1) +
            ' (' +
            this.out.names[s] +
            ') vào cơ sở, Δ = ' +
            d[s] +
            '. ' +
            (this.bland
              ? 'Dùng Bland để tránh lặp cơ sở. '
              : 'Chọn ước lượng cải thiện mạnh nhất; nếu
hòa, lấy chỉ số nhỏ nhất. ') +
            'θ = min{b̄_i/a_is | a_is > 0} = ' +
```

```
              theta +
              '. A' +
              (this.basis[row] + 1) +
              ' ra cơ sở. Phần tử trục = ' +
              this.a[row][s] +
              '.' +
              (theta.zero() ? ' Bước thoái hóa (θ = 0): mục
tiêu có thể chưa đổi.' : '');
            this.capture(d, s, row, note);
            this.pivot(row, s);
          }
        }
        leaving(s) {
          let r = -1,
            best = null;
          this.b.forEach((b, i) => {
            if (this.a[i][s].sign() > 0) {
              const t = b.div(this.a[i][s]);
              if (r < 0 || t.cmp(best) < 0 || (t.eq(best) &&
this.basis[i] < this.basis[r])) {
                r = i;
                best = t;
              }
            }
          });
          return r;
        }
        pivot(r, s) {
          this.check();
          const v = this.a[r][s];
          this.a[r] = this.a[r].map((a) => a.div(v));
          this.b[r] = this.b[r].div(v);
          this.a.forEach((row, i) => {
            if (i !== r) {
              const f = row[s];
              this.a[i] = row.map((a, j) =>
a.sub(f.mul(this.a[r][j])));
              this.b[i] = this.b[i].sub(f.mul(this.b[r]));
            }
          });
          if (this.art[this.basis[r]])
this.active[this.basis[r]] = false;
          this.basis[r] = s;
          this.out.pivots = ++this.pivots;
```

```
        }
        cleanup() {
          for (let i = 0; i < this.b.length; ) {
            this.check();
            if (!this.art[this.basis[i]]) {
              i++;
              continue;
            }
            if (!this.b[i].zero()) throw new Error('Biến giả
còn dương khi kết thúc pha khả thi.');
            const s = this.a[i].findIndex((v, j) =>
!this.art[j] && !v.zero());
            if (s >= 0) {
              this.capture(
                this.deltas(),
                s,
                i,
                'Biến giả cơ sở bằng 0: dùng một hệ số khác 0
để đẩy biến giả ra. Đây là bước khử biến giả, không dùng phép
thử tỷ số.',
                true
              );
              this.pivot(i, s);
              i++;
            } else {
              this.capture(
                this.deltas(),
                -1,
                -1,
                'Dòng ' + (i + 1) + ' trên các cột thực là 0
= 0. Loại ràng buộc phụ thuộc này.'
              );
              this.active[this.basis[i]] = false;
              this.a.splice(i, 1);
              this.b.splice(i, 1);
              this.basis.splice(i, 1);
            }
          }
          this.art.forEach((v, j) => {
            if (v) this.active[j] = false;
          });
          this.visited.clear();
        }
        current() {
```

```
          const x = zeros(this.N);
          this.basis.forEach((k, i) => (x[k] = this.b[i]));
          return x;
        }
        original(v) {
          return this.map.map((row) => row.reduce((sum, a, j)
=> sum.add(a.mul(v[j])), Z));
        }
        capture(delta, enter, leave, note, cleanup = false) {
          if (!this.record) return;
          this.out.steps.push({
            phase: this.phase,
            note,
            a: copy(this.a),
            b: this.b.slice(),
            basis: this.basis.slice(),
            active: this.active.slice(),
            cost: this.cost.slice(),
            delta,
            value: this.objective(),
            enter,
            leave,
            max: this.max,
            bland: this.bland,
            cleanup,
            theta: this.b.map((b, i) =>
              enter >= 0 && this.a[i][enter].sign() > 0 ?
b.div(this.a[i][enter]) : null
            ),
          });
        }
        validateBasis() {
          this.basis.forEach((k, i) => {
            if (this.b[i].sign() < 0) throw new Error('Vế
phải âm trong cơ sở khả thi.');
            this.a.forEach((row, t) => {
              if (!row[k].eq(i === t ? O : Z)) throw new
Error('Cơ sở không phải ma trận đơn vị.');
            });
          });
        }
        alternatives() {
          const p = this.p,
            r = this.out;
```

```
            if (p.m >= 60) return;
            const face = rawProblem(p);
            face.a.push(p.c.map(String));
            face.b.push(String(r.value.sub(p.constant)));
            face.rel.push('=');
            face.constant = '0';
            try {
              for (let j = 0; j < p.n; j++)
                for (const sense of ['max', 'min']) {
                  this.check();
                  face.c = Array(p.n).fill('0');
                  face.c[j] = '1';
                  face.sense = sense;
                  const other = new Solver().solve(face, 'two',
false, {
                    record: false,
                    deadline: this.deadline,
                  });
                  if (other.status === 'limit') return;
                  let candidate = null;
                  if (other.status === 'unbounded') {
                    candidate = other.x.map((v, k) =>
v.add(other.ray[k]));
                    if (same(candidate, r.x)) candidate =
candidate.map((v, k) => v.add(other.ray[k]));
                  } else if (other.status === 'optimal' &&
!same(other.x, r.x)) candidate = other.x;
                  else if (other.status !== 'optimal') throw
new Error('Mặt tối ưu không khả thi.');
                  if (candidate) {
                    if (
                      !feasible(p, candidate) ||
                      !value(p, candidate).eq(r.value) ||
                      same(candidate, r.x)
                    )
                      throw new Error('Nghiệm tối ưu thứ hai
không hợp lệ.');
                    r.alternate = candidate;
                    r.multiplicity = 'multiple';
                    return;
                  }
                }
              r.multiplicity = 'unique';
            } catch (e) {
```

```
        if (!(e instanceof Limit)) throw e;
      }
    }
  }
  scope.LP = {
    Q,
    Z,
    O,
    Cost,
    CZ,
    Solver,
    parseProblem,
    rawProblem,
    value,
    feasible,
    revive,
    zeros,
  };
}
installLP(globalThis);
```